# Drones, control y visión para principiantes

Cómo se construyó la simulación del Tarot Ironman 650 que busca incendios forestales por sí solo

ROS 2 Jazzy · Gazebo Harmonic · ArduPilot SITL · YOLO26

Dr. Jairo Olguín Roque
FES Acatlán, UNAM

Octubre de 2026

# Índice general

**Cómo usar este libro** viii

**I Antes de despegar: lo básico** 1

**1. El proyecto de un vistazo** 2

1.1. Qué hace la simulación . . . 2
1.2. Arquitectura . . . 2
1.3. Convenciones . . . 3
1.4. Equipo usado . . . 3

**2. ¿Qué es un dron y cómo vuela?** 4

2.1. Drones, multirrotores y cuadrirrotores . . . 4
2.2. Las partes . . . 5
2.3. ¿Por qué vuela? El empuje . . . 5
2.4. Cómo se mueve: subir, alabeo, cabeceo y guiñada . . . 6
2.5. Configuración "+" y "X" . . . 8
2.6. El Tarot Ironman 650 . . . 8
2.7. Para qué sirve un dron en un incendio forestal . . . 9

**3. Física sin miedo** 10

3.1. Vectores: flechas con tamaño y dirección . . . 10
3.2. Las leyes de Newton aplicadas al dron . . . 10
3.3. Empuje: el cuadrado de la velocidad . . . 11
3.4. Inercia de rotación y par . . . 11
3.5. Marcos de referencia: ¿desde dónde mido? . . . 12
3.6. Rotaciones y ángulos de Euler . . . 13
3.7. Las ecuaciones del dron en una página . . . 13

**4. ¿Qué es una simulación? Gazebo** **15**

4.1. ¿Por qué simular? . . . 15
4.2. Gazebo . . . 16
4.3. Cómo se describe un robot: SDF . . . 17
4.4. Lo que se ve y lo que choca . . . 17
4.5. Plugins . . . 18
4.6. Tiempo simulado y factor de tiempo real . . . 18
4.7. SITL y HITL . . . 19

**5. ROS 2 para principiantes** **20**

5.1. La terminal en cinco minutos . . . 20
5.2. ¿Qué es ROS 2? . . . 20
5.3. Nodos, tópicos y mensajes . . . 21
5.4. Servicios y parámetros . . . 22
5.5. Archivos launch . . . 22
5.6. El puente entre Gazebo y ROS . . . 22
5.7. Paquetes y espacio de trabajo . . . 22

**II El control: mantener el dron donde queremos** **24**

**6. Control automático desde cero** **25**

6.1. ¿Qué es controlar? . . . 25
6.2. Lazo abierto y lazo cerrado . . . 25
6.3. El controlador proporcional (P) . . . 26
6.4. El término derivativo (D): frenar a tiempo . . . 26
6.5. El término integral (I): no rendirse . . . 26
6.6. Cómo se lee una respuesta . . . 27
6.7. Control en cascada: así se controla un dron . . . 28

**7. Modos deslizantes y super-twisting** **29**

7.1. El problema: perturbaciones que no conocemos . . . 29
7.2. La superficie deslizante . . . 29
7.3. El control clásico: empujar con todo . . . 30
7.4. El chattering . . . 30
7.5. Super-twisting: robusto y suave . . . 31

7.6. Ganancia adaptativa: no exagerar . . . 31
7.7. Cómo se usa en el dron . . . 32

**8. Del papel a la computadora: discretización 33**
8.1. La computadora ve a saltos . . . 33
8.2. Un ejemplo para entender: explícito contra implícito . . . 34
8.3. El problema con los modos deslizantes . . . 35
8.4. La discretización implícita del super-twisting . . . 35
8.5. Tres versiones en el código . . . 36

**9. El control del dron en la simulación 37**
9.1. Modelo . . . 37
9.2. Controlador PD en cascada (línea base) . . . 37
9.3. Super-twisting adaptativo (STA) . . . 38
9.3.1. Discretizaciones . . . 38
9.3.2. Parámetros . . . 39
9.4. Resultados . . . 39
9.4.1. Observaciones . . . 40
9.5. Agregar un controlador . . . 40

**III Ver el mundo: visión computacional 41**

**10. Qué es una imagen para una computadora 42**
10.1. Una imagen es una tabla de números . . . 42
10.2. Cómo forma una imagen una cámara . . . 43
10.3. Campo de visión . . . 44
10.4. Del píxel al mundo: rayos . . . 45

**11. Redes neuronales y YOLO sin complicaciones 47**
11.1. Programar contra aprender . . . 47
11.2. La neurona artificial . . . 48
11.3. Convoluciones: buscar patrones locales . . . 48
11.4. Detectar objetos: cajas y clases . . . 49
11.5. YOLO . . . 49
11.6. Entrenar: datos, etiquetas y épocas . . . 49

11.7. Cómo se mide si funciona . . . 51
11.7.1. IoU: ¿le atinó a la caja? . . . 51
11.7.2. Precisión y recall . . . 51
11.7.3. mAP . . . 51
11.8. Sobreajuste y por qué importan los datos . . . 52

**12. El detector de incendios de la simulación 53**
12.1. Por qué ajustar un modelo . . . 53
12.2. Captura del dataset . . . 53
12.3. Entrenamiento . . . 54
12.4. El nodo detector . . . 55

**IV Construir la simulación 57**

**13. Instalación paso a paso 58**
13.1. ROS 2 Jazzy y Gazebo Harmonic . . . 58
13.2. Espacio de trabajo . . . 59
13.3. ArduPilot SITL . . . 59
13.4. Visión: YOLO26 . . . 60
13.5. Herramientas de CAD y generación de mundos . . . 60
13.6. Comprobación de la instalación . . . 60

**14. Dónde está cada cosa: el espacio de trabajo 62**
14.1. Módulos del paquete `cuadrirotor_sim` . . . 62
14.2. Scripts . . . 63
14.3. Archivos de configuración . . . 64

**15. Cómo se metió el Tarot 650 en Gazebo 65**
15.1. Del CAD a las mallas . . . 66
15.2. Parámetros del vehículo . . . 67
15.3. Motores y mezclador . . . 68
15.4. Sensores . . . 70
15.5. Perturbación de viento . . . 71

**16. Cómo se construyó el bosque 72**

16.1. Mundos disponibles . . . 72
16.2. El mundo `bosque_grande` . . . 73
16.2.1. Generación del terreno . . . 74
16.2.2. El DEM y la clase `Terreno` . . . 75
16.2.3. Árboles en un solo modelo . . . 76
16.2.4. Fauna . . . 76
16.3. El incendio . . . 76
16.4. Cómo arma el mundo el *launch* . . . 77

**V La autonomía: buscar el incendio sin piloto 79**

**17. Conceptos de autonomía 80**
17.1. Percibir, decidir, actuar . . . 80
17.2. Máquinas de estados . . . 80
17.3. Planear la cobertura: recorrer todo el bosque . . . 81
17.4. Ubicar lo que se ve . . . 81
17.5. GPS: de metros a latitud y longitud . . . 83
17.6. LiDAR: medir distancias con luz . . . 84
17.7. Evitar obstáculos . . . 84

**18. La búsqueda del incendio, pieza por pieza 86**
18.1. Máquina de estados . . . 87
18.2. Ruta de barrido en serpentina . . . 87
18.3. Seguimiento del relieve . . . 88
18.4. Mapa de cobertura . . . 88
18.5. Estimación de la posición del foco . . . 88
18.6. Verificación: solo con llamas se confirma . . . 89
18.7. Órbita . . . 89
18.8. Aviso con las coordenadas GPS . . . 90
18.9. Evasión de obstáculos . . . 91
18.9.1. Relieve: primero subir . . . 91
18.9.2. Árboles, rocas y animales: LiDAR y histograma polar . . . 91
18.10 Parámetros del planificador . . . 93

**19. ¿Qué es un piloto automático? ArduPilot y MAVLink 95**

19.1. El software del controlador de vuelo . . . 95
19.2. Modos de vuelo . . . 95
19.3. Armar el dron . . . 96
19.4. El EKF: adivinar bien dónde estoy . . . 96
19.5. MAVLink: el idioma de los drones . . . 96
19.6. Estación de tierra . . . 96

**20. Volar la búsqueda con ArduPilot** **98**
20.1. El vehículo para ArduPilot . . . 98
20.2. Parámetros de ArduCopter . . . 99
20.3. El nodo `mision_ardupilot` . . . 100
20.4. Las ventanas del SITL . . . 100
20.4.1. Comandos útiles en el prompt de MAVProxy . . . 101
20.5. La trayectoria en el mapa . . . 102
20.6. Cierre limpio . . . 103

**VI Manos a la obra** **104**

**21. Tu primer vuelo, paso a paso** **105**
21.1. Paso 0: preparar la terminal . . . 105
21.2. Paso 1: despegar . . . 105
21.3. Paso 2: mirar por dentro . . . 106
21.4. Paso 3: una órbita con viento, y medirla . . . 107
21.5. Paso 4: la búsqueda autónoma del incendio . . . 107
21.6. Paso 5: volar con ArduPilot y darle órdenes . . . 109
21.7. Qué hacer si algo falla . . . 110

**22. Referencia de uso** **111**
22.1. Antes de lanzar . . . 111
22.2. Órdenes de lanzamiento . . . 111
22.3. Argumentos del *launch* . . . 112
22.4. Tópicos . . . 112
22.5. Herramientas de visualización . . . 113
22.6. Campañas y métricas . . . 113
22.7. Regenerar los recursos . . . 113

**23. Cómo se verificó, qué falló y qué falta** **115**

23.1. Pruebas unitarias . . . . . 115

23.2. Resultados de la búsqueda . . . . . 115

23.3. Problemas encontrados . . . . . 116

23.4. Pendientes . . . . . 116

**A. Glosario** **118**

**B. Referencia rápida** **122**

**C. Configuración del mundo grande** **123**

**D. YAML del Tarot generado** **124**

**E. Créditos de los recursos** **125**

# Cómo usar este libro

## ¿Para quién es?

Para cualquier persona que quiera entender, desde cero, cómo funciona un dron que vuela solo: estudiantes de licenciatura, ingenieros de otras áreas o curiosos. No necesitas saber de drones, de control automático ni de inteligencia artificial. Sí ayuda saber usar una computadora con Linux y recordar algo de física y matemáticas de bachillerato; cuando haga falta algo más, lo explicamos en el momento.

El libro cuenta una historia real: cómo se construyó, en tres días de octubre de 2026, una simulación para el monitoreo de incendios forestales. En ella un dron Tarot Ironman 650 despega, recorre un bosque de 420 por 420 metros, encuentra un incendio con su cámara, avisa sus coordenadas GPS y se queda vigilándolo. Cada pieza de esa historia es una excusa para explicar un concepto.

## Cómo está organizado

**Parte I. Lo básico.** Qué es un dron y cómo vuela, la física mínima que hace falta, qué es una simulación y qué es ROS 2, el "sistema nervioso" que conecta todas las piezas.

**Parte II. El control.** Cómo se consigue que el dron vaya exactamente a donde queremos aunque sople el viento: control PID, modos deslizantes, super-twisting y el tema central del proyecto, la discretización implícita.

**Parte III. Visión.** Qué es una imagen para una computadora, cómo funciona una cámara y cómo una red neuronal (YOLO) aprende a reconocer fuego y humo.

**Parte IV. Construir la simulación.** La instalación y cómo se metieron en Gazebo el dron, sus motores, el bosque, las montañas, los animales y el incendio.

**Parte V. Autonomía.** Cómo el dron decide qué hacer: recorrer el bosque, evitar árboles y montañas, estimar dónde está el fuego, confirmarlo y avisar. También cómo volar con un piloto automático real (ArduPilot).

**Parte VI. Manos a la obra.** Un tutorial para hacer tu primer vuelo, la lista de órdenes y cómo se verificó que todo funciona.

Al final hay un **glosario** con todas las palabras raras.

Si solo quieres usar la simulación, puedes ir directo al capítulo 21 ("Tu primer vuelo"). Si quieres entenderla, léelo en orden: cada parte se apoya en la anterior.

## Los recuadros

**Recuerda**

Lo más importante del tema, en pocas palabras. Si solo lees esto, ya te llevas la idea.

**Consejo**

Trucos prácticos que ahorran tiempo o problemas.

**Ejemplo de la vida diaria**

Una comparación con algo cotidiano para entender un concepto abstracto.

**Para curiosos (te lo puedes saltar)**

Detalles matemáticos o de programación. Si es tu primera lectura, sáltalos sin culpa.

**Cuidado**

Errores que es fácil cometer y que pueden romper algo o hacerte perder horas.

**Problema encontrado y cómo se resolvió**

Problemas reales que aparecieron durante el desarrollo y cómo se resolvieron. Son de lo más valioso del libro: así se trabaja de verdad.

**Pruébalo tú**

Algo que puedes hacer en la simulación para comprobar lo que acabas de leer.

## Convenciones de escritura

- Las órdenes que se escriben en la terminal aparecen en recuadros grises:

```
ros2 launch cuadrirotor_sim simulacion.launch.py
```

- Los nombres de archivos, programas y tópicos van en letra de máquina: `busqueda_incendio`.
- `~` significa tu carpeta personal (`/home/tu_usuario`).
- Las distancias están en metros, los ángulos en radianes o grados (se indica) y el tiempo en segundos.

# Parte I

# Antes de despegar: lo básico

# Capítulo 1

# El proyecto de un vistazo

Este libro documenta, paso a paso, cómo se construyó la simulación de monitoreo de incendios: desde la instalación del software hasta la misión autónoma en la que un cuadrirrotor Tarot Ironman 650 recorre un bosque, encuentra un incendio con una cámara y una red neuronal, envía sus coordenadas GPS y lo orbita. Todo el desarrollo se hizo entre el 21 y el 23 de octubre de 2026 en el espacio de trabajo `~/ros2_ws`.

## 1.1. Qué hace la simulación

- **Planta.** Gazebo Harmonic simula la física del dron: cuerpo rígido, cuatro motores de primer orden con empuje $f_i = k_f \omega_i^2$, una cámara, una IMU y un LiDAR 3D. Hay dos vehículos: un cuadrirrotor genérico en "+" y el Tarot 650 en "X", cuya geometría viene de su CAD.
- **Control.** Tres alternativas intercambiables: un PD en cascada (línea base), el super-twisting adaptativo (STA) con tres discretizaciones (explícita, semi-implícita e implícita) y el piloto automático ArduCopter 4.7.2 en SITL.
- **Mundos.** Tres escenarios: `incendio` (plano), `bosque` (85 árboles) y `bosque_grande`: un valle de 420 × 420 m rodeado de montañas, con 1700 árboles, 140 rocas, 21 animales, cielo y un incendio con humo.
- **Percepción.** YOLO26n ajustado con 6000 imágenes del simulador detecta *fuego* y *humo* en la cámara del dron.
- **Autonomía.** El nodo `busqueda_incendio` despega, barre el bosque en serpentina siguiendo el relieve, evita obstáculos con el LiDAR, estima la posición del foco, lo confirma con las llamas, avisa con las coordenadas GPS y lo orbita.

## 1.2. Arquitectura

La figura 1.1 muestra los nodos y los tópicos principales. La cadena completa es *imagen* → *detección* → *estimación* → *referencia* → *control*. El planificador nunca usa la posición verdadera del foco: solo lo que reporta el detector.

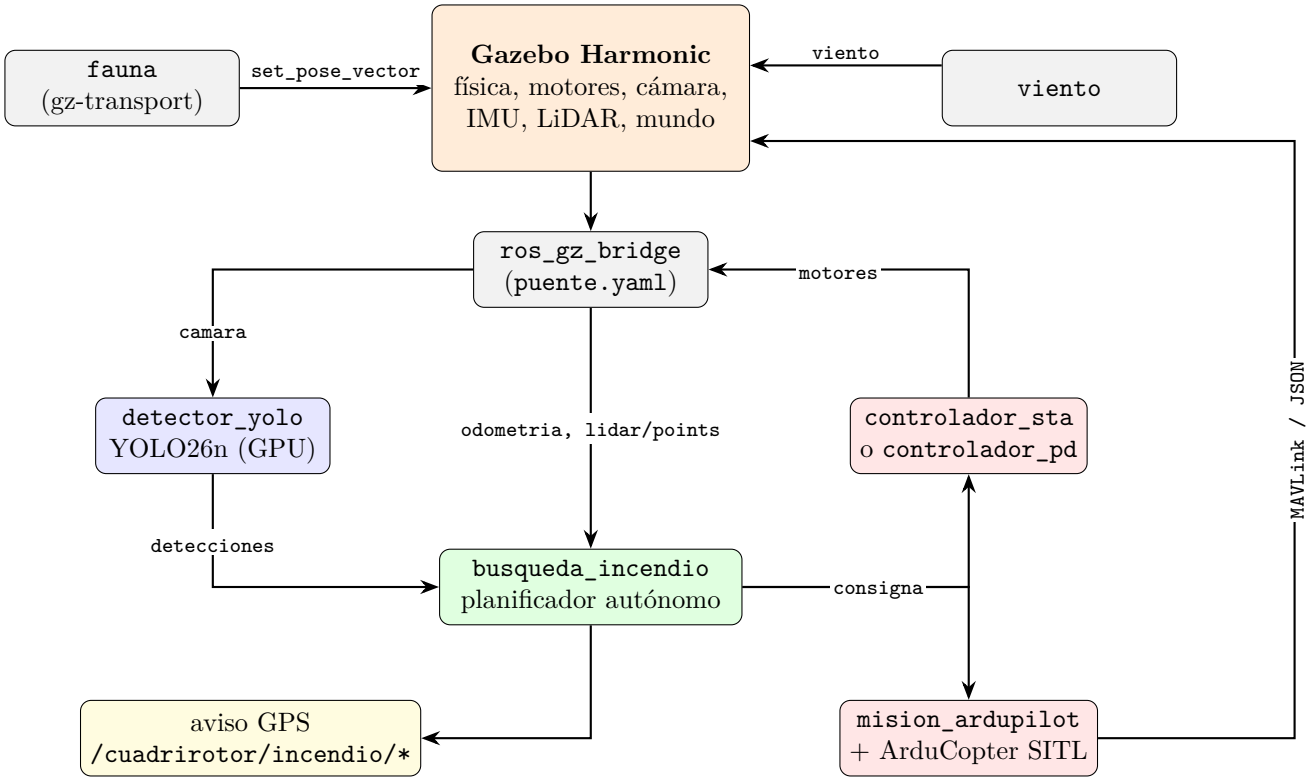



**Figura 1.1:** Nodos y tópicos de la misión de búsqueda. Con `controlador:=sta` o `pd` la consigna la sigue el controlador de ROS 2; con `controlador:=ardupilot`, la reenvía `mision_ardupilot` a ArduCopter por MAVLink.

## 1.3. Convenciones

- **Marco del mundo:** ENU ($x$ al este, $y$ al norte, $z$ arriba), en metros. ArduPilot usa NED: $N = y$, $E = x$, $D = -z$ y la guiñada NED es $\pi/2 - \psi$.
- **Marco del cuerpo:** $x$ al frente, $y$ a la izquierda, $z$ arriba.
- El modelo del dron se llama siempre `cuadrirotor` dentro de Gazebo, sin importar el vehículo; así los tópicos no cambian.
- Rutas: el paquete principal está en `~/ros2_ws/src/cuadrirotor_sim`; salvo que se diga otra cosa, las rutas relativas son respecto a él.

## 1.4. Equipo usado

Ubuntu 24.04 (Noble), ROS 2 Jazzy, Gazebo Harmonic (gz-sim 8), ArduCopter 4.7.2 y una GPU NVIDIA RTX 5060 Ti (YOLO corre en la GPU y Gazebo renderiza con EGL de NVIDIA). Con este equipo el mundo grande corre a tiempo real (factor $\approx 1.0$; $\approx 0.8$ con el LiDAR).

# Capítulo 2

# ¿Qué es un dron y cómo vuela?

**En este capítulo**

Las partes de un cuadrirrotor, por qué vuela, cómo sube, avanza y gira con solo cambiar la velocidad de sus cuatro motores, y cómo es el Tarot 650 que usamos.

## 2.1. Drones, multirrotores y cuadrirrotores

"Dron" es el nombre popular de un **vehículo aéreo no tripulado** (UAV, por sus siglas en inglés): una aeronave sin piloto a bordo. Puede ser un avión de ala fija, un helicóptero o, el caso más común, un **multirrotor**: una aeronave con varias hélices horizontales. El de cuatro hélices se llama **cuadrirrotor** o **cuadricóptero**.

¿Por qué cuatro? Porque es el número mínimo con el que se puede controlar todo el movimiento sin piezas móviles más que los propios motores: no hay alerones, timones ni "plato cíclico" como en un helicóptero. Eso los hace baratos, robustos y fáciles de reparar; a cambio, son inestables por naturaleza: sin una computadora que corrija cientos de veces por segundo, se voltean.

**Recuerda**

Un cuadrirrotor no tiene superficies de control. Todo lo que hace (subir, inclinarse, girar) lo consigue cambiando la velocidad de sus cuatro motores. Por eso el *control* es el corazón del dron.

## 2.2. Las partes

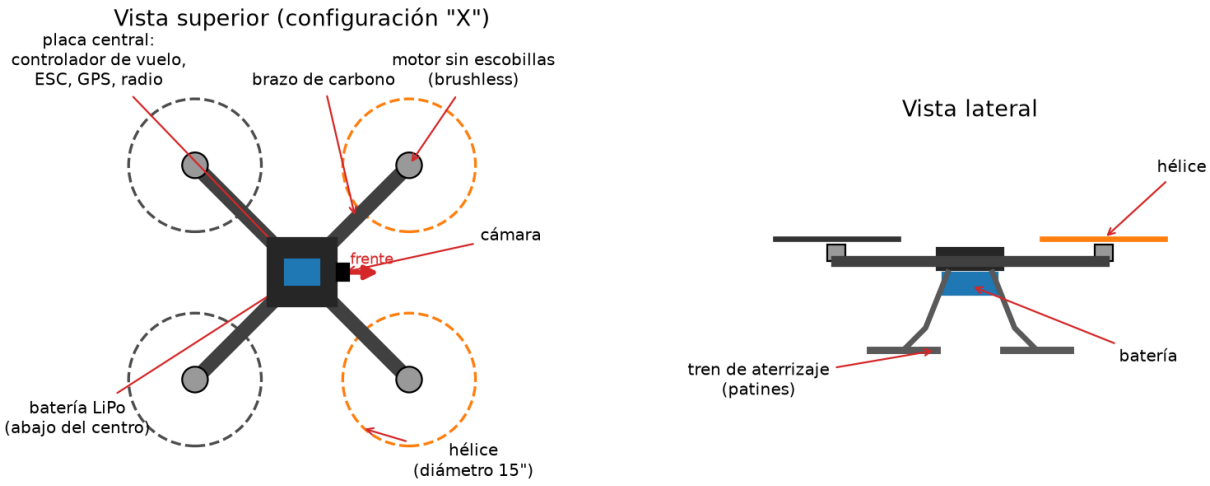



**Figura 2.1:** Partes de un cuadrirrotor como el Tarot 650.

**Marco (frame).** La estructura: una placa central y cuatro brazos. En el Tarot son de fibra de carbono, ligera y rígida. Los brazos del Tarot se pueden plegar para transportarlo.

**Motores sin escobillas (brushless).** Motores eléctricos trifásicos muy eficientes. En el Tarot se usan T-Motor MN3508. Su constante KV dice cuántas revoluciones por minuto da por cada volt.

**ESC (controlador electrónico de velocidad).** Uno por motor. Recibe una orden ("gira a tanto") y genera las corrientes trifásicas para lograrlo.

**Hélices.** Convierten el giro en empuje. Se describen con dos números: diámetro y paso, en pulgadas. Las del Tarot son de $15 \times 5$: 15" de diámetro (38 cm) y 5" de paso (lo que avanzarían en una vuelta si se "atornillaran" en el aire).

**Batería LiPo.** Polímero de litio: mucha energía para su peso. Se describen por el número de celdas en serie (4S, 6S: 3.7 V cada una) y su capacidad en mAh.

**Controlador de vuelo.** La computadora a bordo: lee los sensores y decide la velocidad de cada motor cientos de veces por segundo. Puede llevar ArduPilot o PX4. En nuestra simulación ese papel lo hacen nuestros controladores en Python o ArduPilot en SITL.

**Sensores.** IMU (acelerómetros y giroscopios: cómo se mueve y gira), barómetro (altura), magnetómetro (brújula), GPS (posición) y, según la misión, cámaras y LiDAR.

**Tren de aterrizaje.** Patas o patines para posarse sin dañar la cámara ni la batería.

## 2.3. ¿Por qué vuela? El empuje

Una hélice que gira empuja aire hacia abajo; por la tercera ley de Newton (acción y reacción), el aire empuja a la hélice hacia arriba. A esa fuerza se le llama **empuje**. Crece con el *cuadrado* de la velocidad de giro: al doble de velocidad, cuatro veces más empuje.

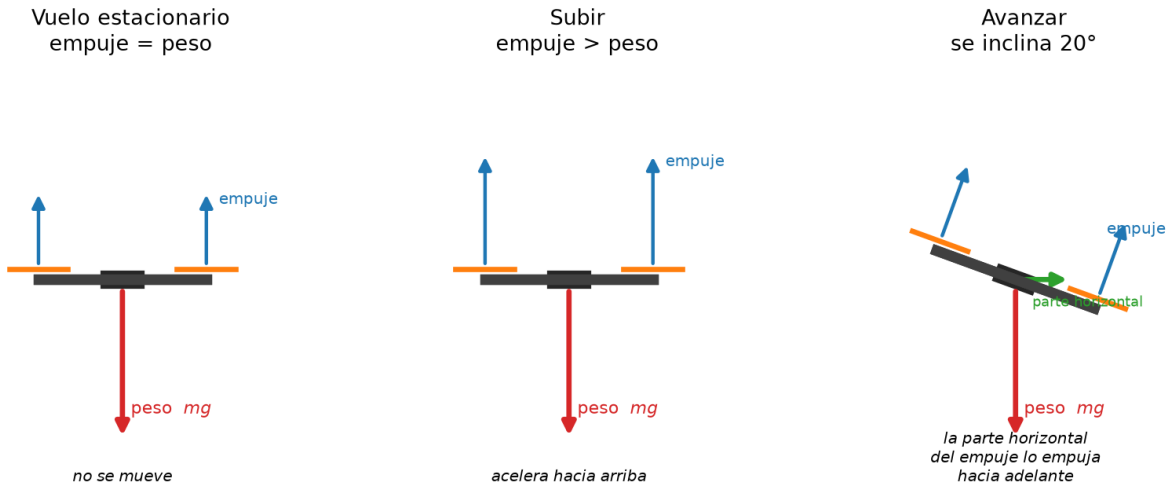



**Figura 2.2:** Las dos fuerzas principales: el empuje de las hélices (azul) y el peso (rojo).

- Si el empuje total es igual al peso, el dron se queda quieto en el aire: **vuelo estacionario** (*hover*).
- Si es mayor, sube; si es menor, baja.
- Para avanzar, el dron **se inclina**: el empuje ya no apunta hacia arriba, y su parte horizontal lo empuja hacia adelante (figura 2.2, derecha). Un cuadrirrotor no tiene "motor hacia adelante": avanza inclinándose, como una persona que se inclina para empezar a caminar.

**Ejemplo de la vida diaria**

Sostén un libro con la mano extendida, plano. Para moverlo hacia adelante sin empujarlo, inclínalo un poco: la fuerza de tu mano "hacia arriba" ahora tiene una parte hacia adelante. Así avanza un dron.

## 2.4. Cómo se mueve: subir, alabeo, cabeceo y guiñada

Un objeto en el espacio puede girar alrededor de tres ejes (figura 2.3). En aviación esos giros tienen nombre:

**Alabeo (roll, $\phi$):** girar alrededor del eje que va al frente; el dron "se ladea".

**Cabeceo (pitch, $\theta$):** girar alrededor del eje lateral; "asiente con la cabeza".

**Guiñada (yaw, $\psi$):** girar alrededor del eje vertical; cambia hacia dónde mira.

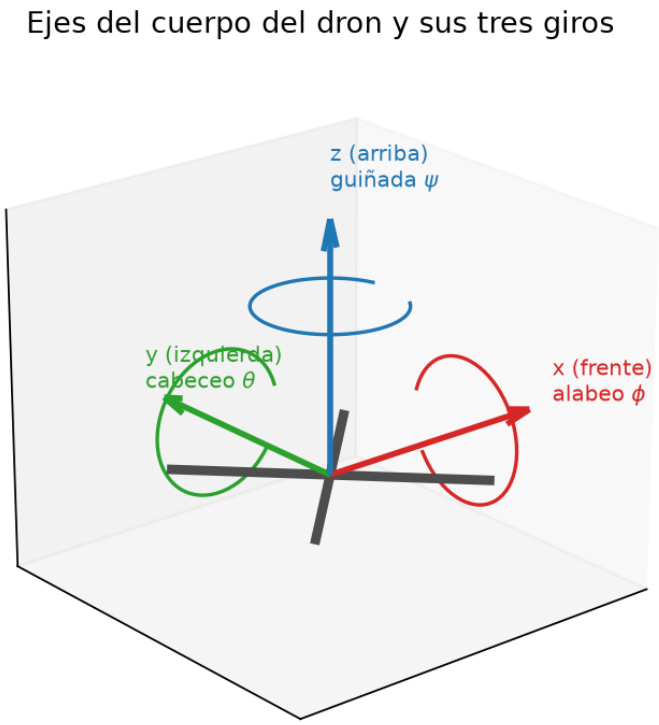



**Figura 2.3:** Ejes del cuerpo: $x$ al frente, $y$ a la izquierda, $z$ arriba.

Con cuatro motores se controla todo así (figura 2.4):

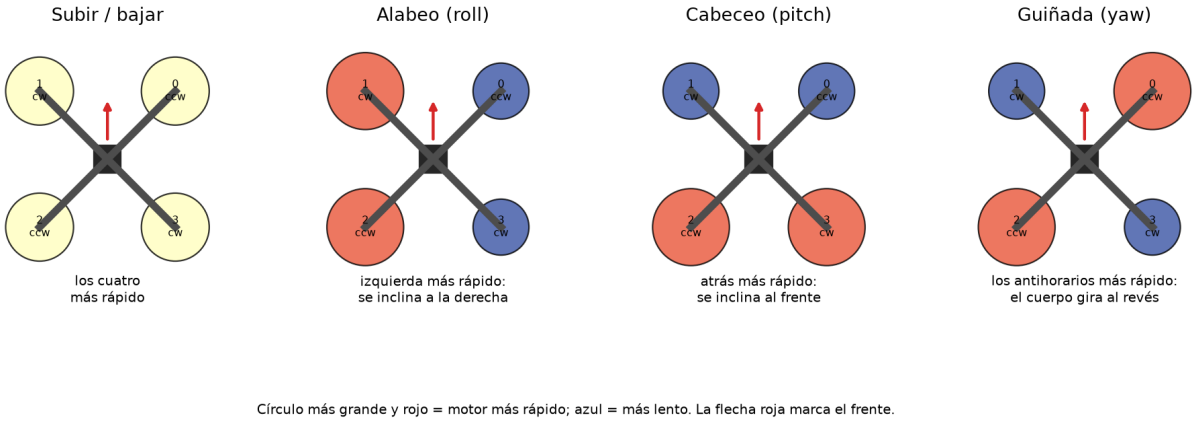



**Figura 2.4:** Cómo produce el dron cada movimiento solo cambiando la velocidad de los motores.

- **Subir o bajar:** los cuatro motores aceleran o frenan por igual.
- **Alabeo:** los motores de un lado giran más rápido que los del otro; el dron se inclina hacia el lado lento y empieza a moverse de lado.
- **Cabeceo:** igual, pero adelante contra atrás; así avanza o retrocede.
- **Guiñada:** aquí entra un truco. Cada hélice, al girar, "empuja" el aire de lado y el aire, por

reacción, intenta hacer girar al dron al revés. Por eso dos hélices giran en un sentido y dos en el otro: los efectos se cancelan. Si se aceleran las dos que giran en un sentido y se frenan las otras dos, el empuje total no cambia pero sobra par en un sentido, y el dron gira sobre sí mismo.

**Recuerda**

Cuatro motores, cuatro "perillas": empuje total, alabeo, cabeceo y guiñada. El avance y el movimiento lateral no se controlan directamente: se consiguen inclinando el dron. Por eso el control de un dron es *en cascada*: para ir a una posición hay que pedir una inclinación, y para lograr la inclinación hay que pedir velocidades a los motores.

## 2.5. Configuración "+" y "X"

Si un brazo apunta al frente, la configuración es "+"; si el frente queda entre dos brazos, es "X". La "X" es la más común: la cámara frontal no ve hélices y los cuatro motores trabajan al inclinarse hacia adelante. En la simulación hay un cuadrirrotor genérico en "+" y el Tarot en "X".

## 2.6. El Tarot Ironman 650



**Figura 2.5:** El Tarot 650 simulado, en vuelo estacionario sobre su base.

El Tarot Ironman 650 es un marco comercial de fibra de carbono, muy usado en investigación porque carga cámaras y computadoras sin problema. "650" es la distancia en milímetros entre motores

opuestos (en diagonal). Nuestro modelo:

- motores a $(\pm 0.22, \pm 0.21)$ m del centro, en "X";
- hélices de 15";
- 2.2 kg de masa total (supuesto);
- cada motor da hasta ≈19.6 N (unos 2 kg) de empuje: en total puede levantar ≈3.6 veces su peso. Esa **relación empuje/peso** da margen para maniobrar y resistir el viento.

**Para curiosos (te lo puedes saltar)**

En vuelo estacionario cada motor sostiene un cuarto del peso: $2.2 \times 9.81/4 = 5.4$ N. Como el empuje es $f = k_f \omega^2$, con $k_f = 4.24 \times 10^{-5}$ la velocidad necesaria es $\omega = \sqrt{5.4/k_f} = 357$ rad/s, unas 3400 revoluciones por minuto.

## 2.7. Para qué sirve un dron en un incendio forestal

Un incendio se combate mejor cuanto antes se detecta y cuanto mejor se conoce su ubicación. Un dron puede recorrer zonas de difícil acceso, ver por encima de las copas, detectar el humo desde lejos, dar las coordenadas exactas del foco y quedarse vigilándolo, sin poner en riesgo a un piloto. Para hacerlo de forma autónoma necesita tres capacidades, que son las tres partes centrales del libro:

1. **Control** preciso aunque haya viento (Parte II);
2. **percepción**: ver y reconocer el fuego (Parte III);
3. **autonomía**: decidir a dónde ir y qué hacer (Parte V).

# Capítulo 3

# Física sin miedo

**En este capítulo**

Vectores, las leyes de Newton aplicadas al dron, por qué el empuje crece con el cuadrado de la velocidad, qué es la inercia, qué son los marcos de referencia (ENU, NED, cuerpo) y cómo se describe una rotación. Al final, las ecuaciones del dron en una sola página.

## 3.1. Vectores: flechas con tamaño y dirección

Muchas cantidades físicas no son solo un número: la velocidad de un dron es "5 m/s *hacia el norte*". Esas cantidades se llaman **vectores** y se dibujan como flechas. En tres dimensiones un vector tiene tres componentes, por ejemplo $v = (v_x, v_y, v_z)$. Posición, velocidad, aceleración, fuerza y velocidad angular son vectores.

## 3.2. Las leyes de Newton aplicadas al dron

1. **Inercia.** Un cuerpo sigue como está (quieto o moviéndose en línea recta) mientras nadie lo empuje. Un dron que avanza y apaga la inclinación sigue avanzando un rato.
2. $F = ma$**.** La aceleración es la fuerza total dividida entre la masa. Para el dron:

$$m\,\ddot{p} = \underbrace{T\,R\,e_3}_{\text{empuje}}\ \underbrace{-\,mg\,e_3}_{\text{peso}}\ +\ \underbrace{d(t)}_{\text{viento}} \tag{3.1}$$

   donde $\ddot{p}$ es la aceleración, $T$ el empuje total, $Re_3$ la dirección "arriba" del dron (que cambia si se inclina) y $e_3 = (0, 0, 1)$ la vertical.
3. **Acción y reacción.** La hélice empuja aire hacia abajo y el aire empuja la hélice hacia arriba. También explica la guiñada: al hacer girar la hélice, el motor "empuja" al dron en sentido contrario.

## 3.3. Empuje: el cuadrado de la velocidad

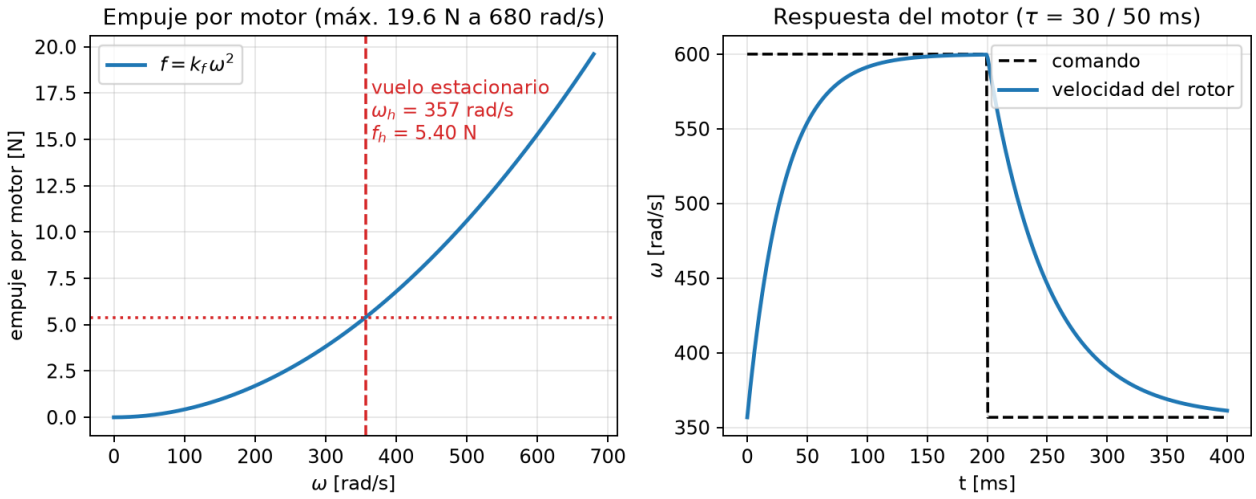



**Figura 3.1:** Izquierda: el empuje de un motor del Tarot crece como $\omega^2$. Derecha: el motor no cambia de velocidad al instante, tarda unos 30–50 ms.

El empuje de una hélice es aproximadamente

$$f = k_f \, \omega^2, \tag{3.2}$$

con $\omega$ la velocidad de giro en rad/s y $k_f$ una constante que depende de la hélice y del aire. Que sea cuadrático tiene consecuencias: cerca del vuelo estacionario un pequeño cambio de velocidad da un cambio de empuje más o menos proporcional, pero no hay una relación lineal en todo el rango, y el controlador tiene que tenerlo en cuenta (por eso calcula fuerzas y luego saca raíces cuadradas para obtener velocidades).

Además del empuje, cada hélice produce un **par de arrastre** (una "torsión") proporcional a su empuje, $k_m f$: es lo que intenta girar al dron al revés de la hélice.

**Recuerda**

Los motores no son instantáneos. En el Tarot simulado tardan ≈30 ms en acelerar y ≈50 ms en frenar. Ese retardo es parte de lo que hace difícil el control, y tiene un papel importante en los resultados del control.

## 3.4. Inercia de rotación y par

Para girar, lo que importa no es la masa sino el **momento de inercia** $J$: cuánto se resiste un cuerpo a cambiar su giro. Depende de cómo está repartida la masa: un patinador que encoge los brazos gira más rápido porque baja su momento de inercia. El dron tiene un momento de inercia para cada eje (en el Tarot: 0.034, 0.036 y 0.066 $\mathrm{kg\,m^2}$; el del eje vertical es mayor porque los motores están lejos del centro en ese plano).

Lo que hace girar es un **par** (o torque) $\tau$: una fuerza aplicada a cierta distancia del centro. Un motor de un brazo que empuja más que el del brazo opuesto produce un par que ladea al dron. La "segunda

ley de Newton para giros" es

$$J\,\dot{\omega} = \tau - \omega \times J\omega. \tag{3.3}$$

El último término es un efecto giroscópico; para entender la idea basta con "par = inercia por aceleración angular".

## 3.5. Marcos de referencia: ¿desde dónde mido?

Decir "el dron está en $(3, 5, 10)$" no significa nada si no se dice desde dónde y con qué ejes. Eso es un **marco de referencia**. En este proyecto aparecen tres:

**Mundo ENU** (Este, Norte, Arriba): fijo al suelo, con origen en el centro del mundo de Gazebo. Lo usan Gazebo y ROS.

**Mundo NED** (Norte, Este, Abajo): el mismo lugar, pero con otros ejes. Lo usa la aviación y ArduPilot. Pasar de uno a otro es solo cambiar $x \leftrightarrow y$ y el signo de $z$.

**Cuerpo:** pegado al dron y que se mueve con él: $x$ al frente, $y$ a la izquierda, $z$ arriba. Los sensores (IMU, cámara, LiDAR) miden en este marco.

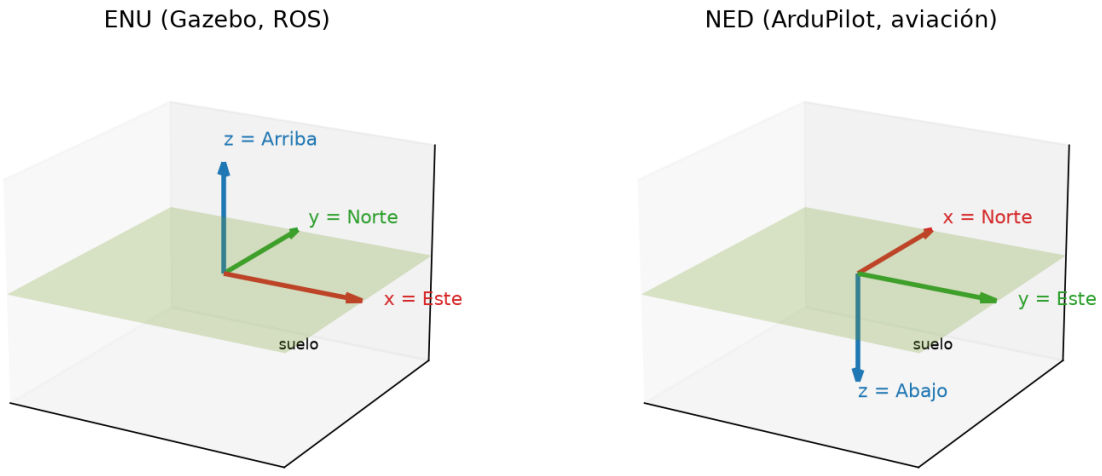



**Figura 3.2:** Los dos marcos del mundo. Confundirlos es una de las causas más comunes de errores en drones.



**Figura 3.3:** En Gazebo se pueden mostrar los marcos de cada pieza del dron (rojo = $x$, verde = $y$, azul = $z$).

**Problema encontrado y cómo se resolvió**

Durante el desarrollo, el vuelo con ArduPilot mandaba al dron 330 m lejos de donde debía ir: se restaba dos veces la posición de salida al pasar del marco del mundo al marco local de ArduPilot. El dron llegó a estrellarse contra un cerro. Se arregló midiendo el desfase entre los dos marcos en lugar de suponerlo (sección 20.1). Moraleja: **siempre verifica en qué marco está cada número**.

## 3.6. Rotaciones y ángulos de Euler

La orientación del dron se puede describir con tres ángulos, alabeo $\phi$, cabeceo $\theta$ y guiñada $\psi$ (los **ángulos de Euler**), aplicados en cierto orden. Para hacer cuentas es más cómodo una **matriz de rotación** $R$ de $3 \times 3$: sus tres columnas son los ejes del dron vistos desde el mundo. Multiplicar un vector del cuerpo por $R$ lo "pasa" al mundo. Por ejemplo, la dirección del empuje es la tercera columna de $R$, $Re_3$.

**Para curiosos (te lo puedes saltar)**

Gazebo y ROS entregan la orientación como un **cuaternión** $(q_x, q_y, q_z, q_w)$: cuatro números que representan una rotación sin las singularidades de los ángulos de Euler (que fallan cuando el cabeceo llega a 90°). El código convierte cuaterniones a matrices con `dinamica.quat_a_rot`.

## 3.7. Las ecuaciones del dron en una página

**Modelo del cuadrirrotor**

| | |
|---|---|
| posición: | $m\ddot{p} = TRe_3 - mg\,e_3 + d(t)$ |
| orientación: | $\dot{R} = R\hat{\omega}$ |
| giro: | $J\dot{\omega} = \tau - \omega \times J\omega$ |
| motores: | $f_i = k_f\omega_i^2, \qquad T = \sum_i f_i, \qquad \tau = \left(\sum y_i f_i,\ -\sum x_i f_i,\ -\sum s_i k_m f_i\right)$ |

No hace falta resolverlas a mano: Gazebo las integra por nosotros (Capítulo 4). Pero el controlador las usa para saber cuánto empuje y cuánto par pedir.

# Capítulo 4

# ¿Qué es una simulación? Gazebo

**En este capítulo**

Por qué se simula antes de volar, qué hace un simulador de robots como Gazebo, cómo se describe un robot (SDF), la diferencia entre lo que se ve y lo que choca, qué son los *plugins* y qué significan SITL y "factor de tiempo real".

## 4.1. ¿Por qué simular?

Un Tarot 650 con cámara cuesta varios miles de pesos y un error de programación puede estrellarlo en segundos. Además, para probar la búsqueda de un incendio harían falta un bosque, un incendio y permisos. Un simulador resuelve todo eso:

- los choques son gratis: se reinicia y listo;
- se puede repetir exactamente el mismo experimento (mismo viento, mismo incendio) para comparar controladores, algo imposible al aire libre;
- se puede medir todo, incluida la posición *verdadera* del dron, que en la vida real nunca se conoce exactamente;
- se puede acelerar o pausar el tiempo.

A cambio, un simulador es una aproximación: el viento, la aerodinámica o la cámara nunca son idénticos a los reales. Por eso el camino es simular, luego volar con cuidado y comparar.

**Recuerda**

La simulación no reemplaza el vuelo real: lo prepara. Lo que no funciona en simulación, casi seguro no funcionará en el dron; lo que funciona en simulación, probablemente funcionará con ajustes.

## 4.2. Gazebo

**Gazebo** (en su versión *Harmonic*) es un simulador de robots de código abierto. En cada paso de tiempo (1 ms en nuestro caso):

1. un **motor de física** (DART) calcula fuerzas, contactos y movimientos de todos los cuerpos, resolviendo ecuaciones como las del capítulo anterior;
2. un **motor de render** (OGRE 2) dibuja la escena en 3D, lo que permite simular cámaras y LiDAR;
3. los **sensores** simulados (IMU, cámara, LiDAR, odometría) generan sus mediciones;
4. los **plugins** (pequeños programas) agregan comportamientos: motores, viento, partículas de fuego, conexión con ArduPilot.

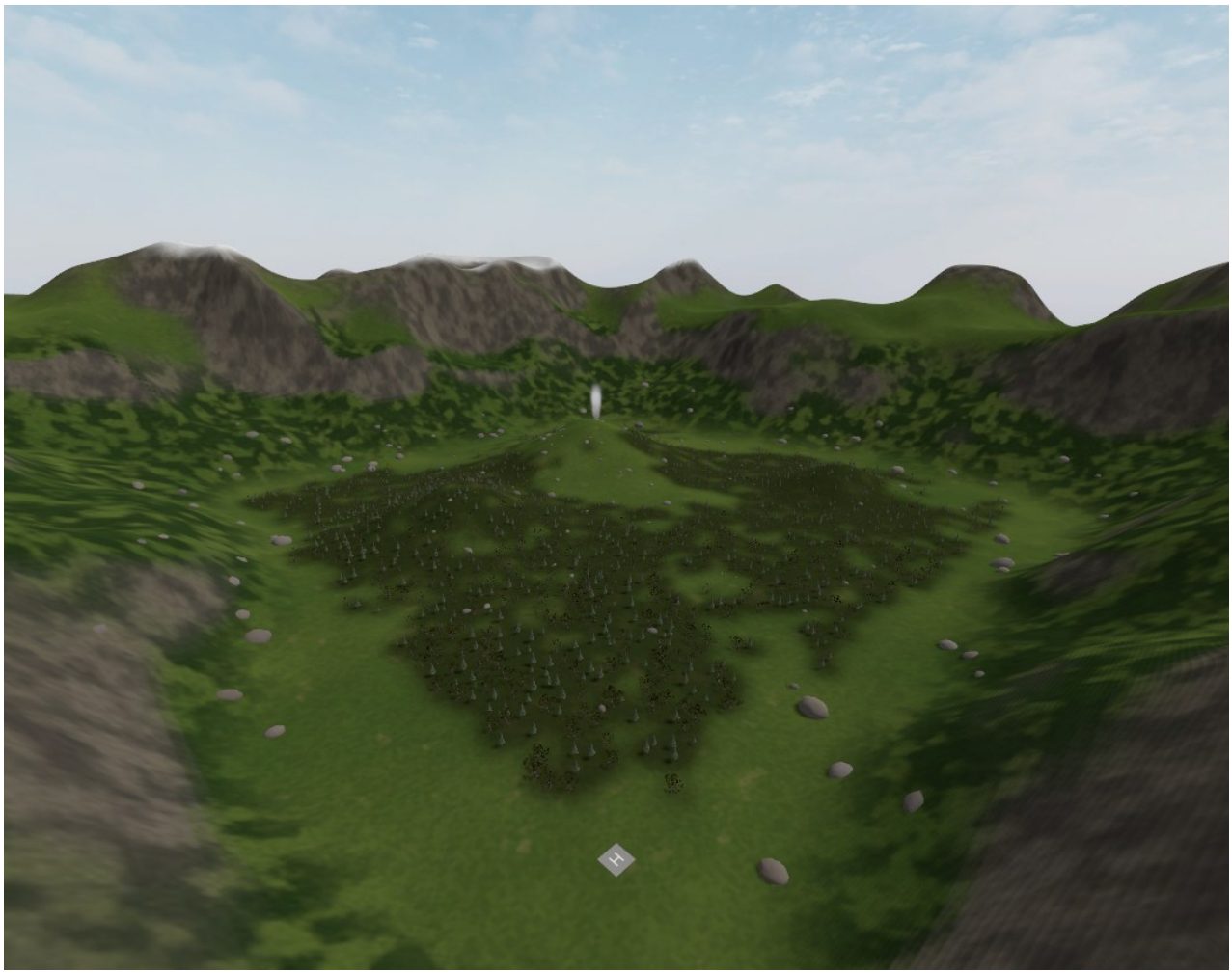


**Figura 4.1:** El mundo `bosque_grande` visto en Gazebo: terreno, árboles, rocas, cielo y, al fondo, el humo del incendio.

## 4.3. Cómo se describe un robot: SDF

Gazebo lee los robots y los mundos en archivos **SDF** (*Simulation Description Format*), que son texto en formato XML. Un robot se describe con:

**Eslabones (*links*):** piezas rígidas, cada una con masa, inercia, una forma para dibujar y otra para chocar. El Tarot tiene el cuerpo y cuatro rotores.

**Articulaciones (*joints*):** cómo se unen los eslabones; los rotores giran respecto al cuerpo en articulaciones *revolute*.

**Sensores** montados en un eslabón, con su posición y orientación.

**Plugins** que les dan vida.

Un fragmento simplificado:

```
<model name="cuadrirotor">
  <link name="base_link">
    <inertial><mass>2.12</mass> ... </inertial>
    <visual name="placas"> <geometry><mesh><uri>model://tarot650/meshes/placas.stl</uri></mesh></
    geometry> </visual>
    <collision name="cuerpo_col"> <geometry><box><size>0.16 0.16 0.06</size></box></geometry> </
    collision>
    <sensor name="camara" type="camera"> ... </sensor>
  </link>
  <link name="rotor_0"> ... </link>
  <joint name="rotor_0_joint" type="revolute"> <parent>base_link</parent> <child>rotor_0</child>
    ... </joint>
  <plugin filename="gz-sim-multicopter-motor-model-system" ...> ... </plugin>
</model>
```

## 4.4. Lo que se ve y lo que choca

Cada eslabón tiene dos formas. La **visual** es la que se dibuja: puede ser una malla 3D muy detallada, como la del CAD del Tarot. La de **colisión** es la que usa la física para detectar choques: conviene que sea simple (cajas, cilindros) porque calcular contactos entre mallas complicadas es lento y frágil.



**Figura 4.2:** Izquierda: la forma visual del Tarot. Derecha: Gazebo mostrando sus formas de colisión (en naranja): una caja para el cuerpo y dos cilindros delgados para los patines.

**Problema encontrado y cómo se resolvió**

En el mundo grande, las rocas usaban como colisión mallas OBJ sin *normales* (los vectores que dicen hacia dónde "mira" cada triángulo). Gazebo se caía con un *segfault* en cuanto algo las tocaba. Exportarlas con normales lo arregló.

## 4.5. Plugins

Un *plugin* es código (en C++) que Gazebo carga para agregar un comportamiento. En este proyecto se usan:

`MulticopterMotorModel` (de Gazebo): convierte una velocidad pedida en empuje y par de un rotor, con su retardo.

`OdometryPublisher:` publica la posición y velocidad verdaderas del dron.

`ParticleEmitter:` las llamas y el humo.

`FuerzaViento` (nuestro): aplica la fuerza del viento.

`ComandosMotores` (nuestro) y `ArduPilotPlugin`: para volar con ArduPilot.

## 4.6. Tiempo simulado y factor de tiempo real

La simulación tiene su propio reloj. Si la computadora es rápida, un segundo simulado tarda un segundo real; si no, tarda más. El **factor de tiempo real** (RTF) es la razón entre los dos: RTF = 1 significa tiempo real; RTF = 0.5, que todo va a la mitad de velocidad. Con la RTX 5060 Ti el mundo grande corre a RTF ≈1.0 (0.8 con el LiDAR). Todos los nodos usan el reloj simulado (`use_sim_time`), así que los resultados no dependen de la velocidad de la computadora.

## 4.7. SITL y HITL

**SITL** (*software in the loop*): el mismo programa del piloto automático que volaría en el dron (aquí ArduCopter) corre en la computadora y "vuela" el dron simulado. Es lo que hacemos en el capítulo 19.

**HITL** (*hardware in the loop*): el controlador de vuelo físico está conectado a la simulación. Es el paso siguiente antes de volar.

**Pruébalo tú**

Lanza la simulación (capítulo 21) y, en la ventana de Gazebo, haz clic derecho sobre el dron → *View* → *Collisions*. Verás las formas de choque. Prueba también *Inertia* y *Frames*.

# Capítulo 5

# ROS 2 para principiantes

**En este capítulo**

Qué es ROS 2, la terminal en cinco minutos, nodos, tópicos, mensajes, servicios, parámetros, archivos *launch* y el espacio de trabajo. Con ejemplos de nuestra simulación.

## 5.1. La terminal en cinco minutos

Casi todo se hace escribiendo órdenes en una **terminal** (Ctrl+Alt+T en Ubuntu). Lo mínimo:

```
pwd                     # ?`en qué carpeta estoy?
ls                      # ?`qué hay aquí?
cd ~/ros2_ws            # entrar a una carpeta (~ es tu carpeta personal)
cd ..                   # subir un nivel
cat archivo.txt         # ver un archivo
nano archivo.txt        # editar un archivo (Ctrl+O guarda, Ctrl+X sale)
Ctrl+C                  # detener el programa que está corriendo
```

**Consejo**

La tecla Tab completa nombres de archivos y órdenes; la flecha hacia arriba recupera la orden anterior. Ahorran muchísimo tiempo y errores de dedo.

## 5.2. ¿Qué es ROS 2?

**ROS** (*Robot Operating System*) no es un sistema operativo: es un conjunto de herramientas para que muchos programas pequeños de un robot hablen entre sí. En lugar de un programa gigante que lo hace todo, hay un programa que lee la cámara, otro que detecta fuego, otro que decide a dónde ir, otro que controla los motores... y ROS los conecta. Usamos la versión **ROS 2 Jazzy**.

**Ejemplo de la vida diaria**

Piensa en una cocina de restaurante. El que recibe pedidos los pega en un tablero; el cocinero toma los pedidos del tablero; el mesero recoge los platos de la barra. Nadie habla directamente con nadie: todos leen y escriben en lugares comunes. En ROS, los cocineros son los *nodos* y los tableros son los *tópicos*.

## 5.3. Nodos, tópicos y mensajes

**Nodo:** un programa que hace una tarea. En nuestra simulación: `detector_yolo`, `busqueda_incendio`, `controlador_sta`, `viento`...

**Tópico:** un canal con nombre por donde viajan datos, por ejemplo `/cuadrirotor/camara`.

**Mensaje:** el dato que viaja, con un tipo definido. `sensor_msgs/Image` es una imagen; `nav_msgs/Odometry`, una posición con velocidad.

**Publicar / suscribirse:** un nodo *publica* en un tópico y cualquier otro se *suscribe* para recibir. El que publica no sabe quién escucha.

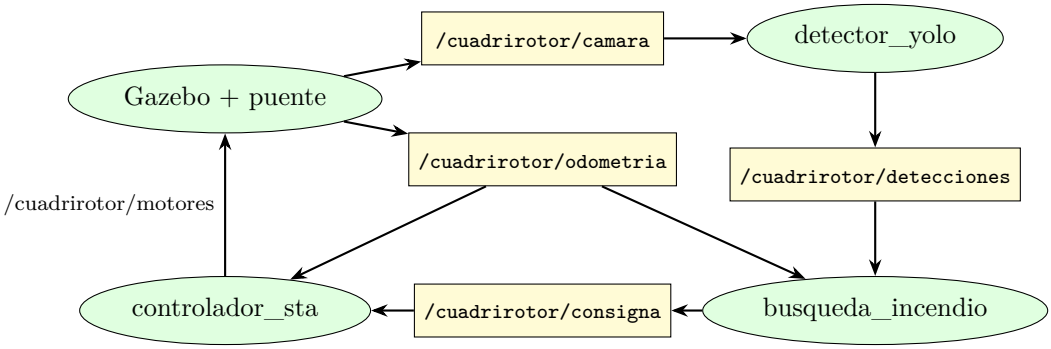



**Figura 5.1:** Nodos (óvalos verdes) que se comunican por tópicos (rectángulos amarillos) en la misión de búsqueda.

Para mirar desde la terminal:

```
ros2 node list                                  # nodos que están corriendo
ros2 topic list                                 # tópicos que existen
ros2 topic echo /cuadrirotor/busqueda/estado    # ver lo que pasa por un tópico
ros2 topic hz /cuadrirotor/camara               # ¿cuántos mensajes por segundo?
ros2 topic info /cuadrirotor/odometria          # tipo y quién publica/escucha
ros2 interface show nav_msgs/msg/Odometry       # ¿qué campos tiene un mensaje?
```

## 5.4. Servicios y parámetros

**Servicio:** una pregunta con respuesta (como una llamada telefónica), a diferencia del tópico, que es una transmisión continua. El *launch* usa servicios de Gazebo para hacer que la cámara de la ventana siga al dron.

**Parámetro:** un ajuste de un nodo, como la altura de búsqueda o la ganancia del controlador. Se guardan en archivos **YAML**:

```
busqueda_incendio:
  ros__parameters:
    altura_busqueda: 30.0     # metros sobre el suelo
    velocidad: 5.0            # m/s
```

## 5.5. Archivos launch

Arrancar diez nodos a mano sería tedioso. Un archivo **launch** (en Python) los arranca todos con sus parámetros. El nuestro es `launch/simulacion.launch.py` y recibe *argumentos* para elegir vehículo, mundo, controlador y misión:

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande
    controlador:=sta mision:=busqueda
```

## 5.6. El puente entre Gazebo y ROS

Gazebo tiene su propio sistema de mensajes (*gz-transport*). El nodo `ros_gz_bridge` traduce entre los dos mundos según la lista de `config/puente.yaml`: la cámara, la odometría, la IMU y el LiDAR van de Gazebo a ROS; las velocidades de los motores y el viento, de ROS a Gazebo.

## 5.7. Paquetes y espacio de trabajo

El código de ROS se organiza en **paquetes** (carpetas con un `package.xml`). Los paquetes viven en un **espacio de trabajo** (*workspace*); el nuestro es `~/ros2_ws`. Se compila con `colcon` y, antes de usarlo en cada terminal, se "activa" con `source`:

```
cd ~/ros2_ws
colcon build --symlink-install       # compila todos los paquetes de src/
source install/setup.bash            # hace visibles los paquetes en esta terminal
```

**Cuidado**

Si una orden de ROS dice que no encuentra el paquete `cuadrirotor_sim`, casi siempre es porque olvidaste el `sourceinstall/setup.bash` en esa terminal. Para no olvidarlo, agrégalo a `~/.bashrc`.

**Pruébalo tú**

Con la simulación corriendo, abre otra terminal, haz `source` y escribe `ros2topicecho/cuadrirotor/odometria--fieldpose.pose.position`. Verás la posición del dron cambiar en tiempo real.

# Parte II

# El control: mantener el dron donde queremos

# Capítulo 6

# Control automático desde cero

**En este capítulo**

Qué es controlar, la diferencia entre lazo abierto y lazo cerrado, el error, los controladores P, PD y PID, cómo se lee una respuesta y por qué un dron necesita control en cascada.

## 6.1. ¿Qué es controlar?

Controlar es conseguir que algo se comporte como queremos aunque haya perturbaciones. Queremos que el dron esté a 30 m de altura; el viento lo empuja hacia abajo; el controlador acelera los motores para compensar. Al "algo" que se controla se le llama **planta** (aquí, el dron); a lo que queremos, **referencia** o **consigna**; a lo que nos estorba, **perturbación**.

## 6.2. Lazo abierto y lazo cerrado

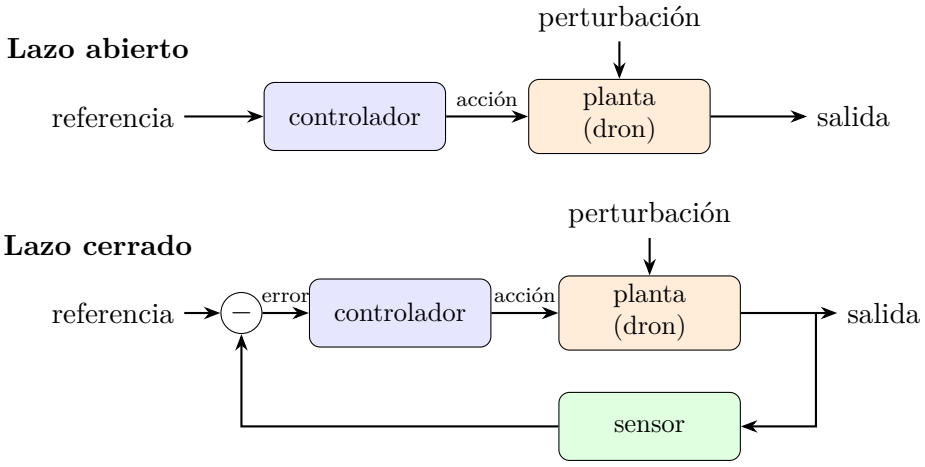



**Figura 6.1:** En lazo abierto se actúa "a ciegas". En lazo cerrado se mide el resultado, se compara con lo que se quiere y se corrige: eso es la *retroalimentación*.

**Ejemplo de la vida diaria**

Una tostadora es lazo abierto: tuesta un tiempo fijo, aunque el pan se queme. Ajustar la temperatura de la regadera es lazo cerrado: sientes el agua (sensor), la comparas con la que quieres (error) y mueves la llave (acción). Si alguien abre otra llave en la casa (perturbación), lo notas y corriges.

Un dron sin lazo cerrado no se puede volar: cualquier ráfaga o diferencia entre motores lo voltea en un segundo. El **error** es la diferencia entre lo que queremos y lo que hay, $e = \text{referencia} - \text{medición}$, y todo controlador hace algo con él.

## 6.3. El controlador proporcional (P)

La idea más simple: actuar en proporción al error. Si estoy 2 m abajo, empujo el doble que si estoy 1 m abajo:

$$u = K_p\, e. \tag{6.1}$$

$K_p$ es la **ganancia proporcional**. El problema con un dron: el control P actúa sobre la *aceleración*, y un objeto que se acelera hacia el objetivo llega con velocidad, se pasa, regresa, se vuelve a pasar... oscila para siempre (curva gris de la figura 6.2).

## 6.4. El término derivativo (D): frenar a tiempo

Hay que frenar antes de llegar. El término derivativo actúa sobre la *velocidad* del error: si me acerco rápido, freno.

$$u = K_p\, e + K_d\, \dot{e}. \tag{6.2}$$

Es como un amortiguador. Con poco $K_d$ oscila; con mucho, llega muy lento; hay un punto intermedio (figura 6.2, derecha).

## 6.5. El término integral (I): no rendirse

Si un viento constante empuja hacia abajo, el PD llega a un equilibrio un poco por debajo del objetivo: con ese pequeño error ya compensa el viento. El término integral *acumula* el error en el tiempo: mientras haya error, sigue creciendo hasta eliminarlo.

$$u = K_p\, e + K_d\, \dot{e} + K_i \int e\, dt. \tag{6.3}$$

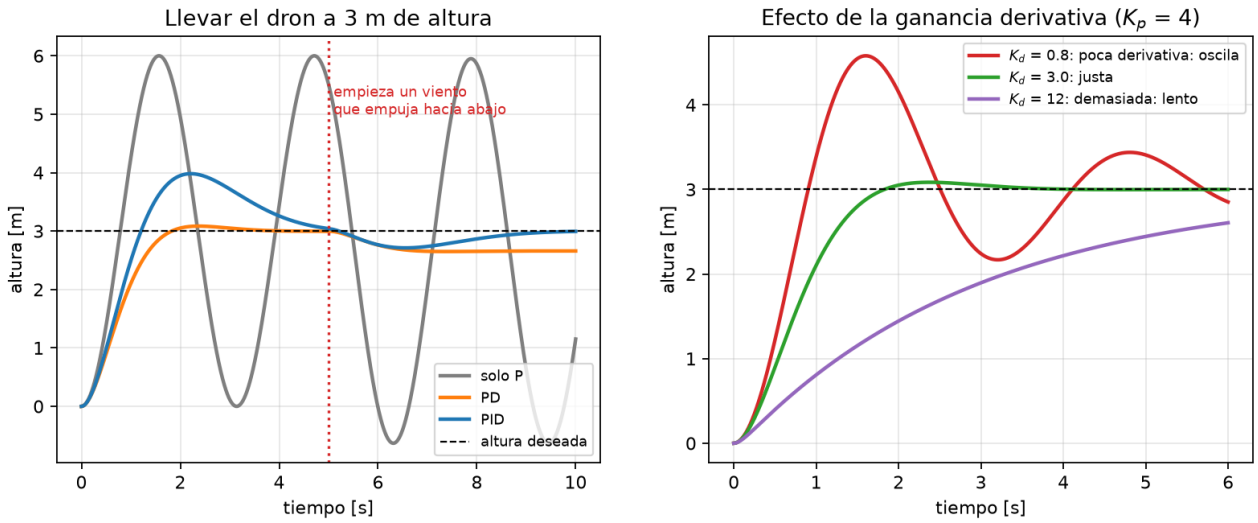



**Figura 6.2:** Simulación de la altura de una masa de 2.2 kg (como el Tarot). Izquierda: solo P oscila; PD llega pero el viento (desde $t = 5$ s) le deja un error; PID lo corrige. Derecha: efecto de $K_d$.

**Recuerda**

**P** empuja hacia el objetivo, **D** frena para no pasarse, **I** corrige lo que queda. El PID es, de lejos, el controlador más usado en el mundo, incluidos los drones comerciales.

## 6.6. Cómo se lee una respuesta

Para comparar controladores se les pide un cambio brusco de referencia (un **escalón**) y se mide:

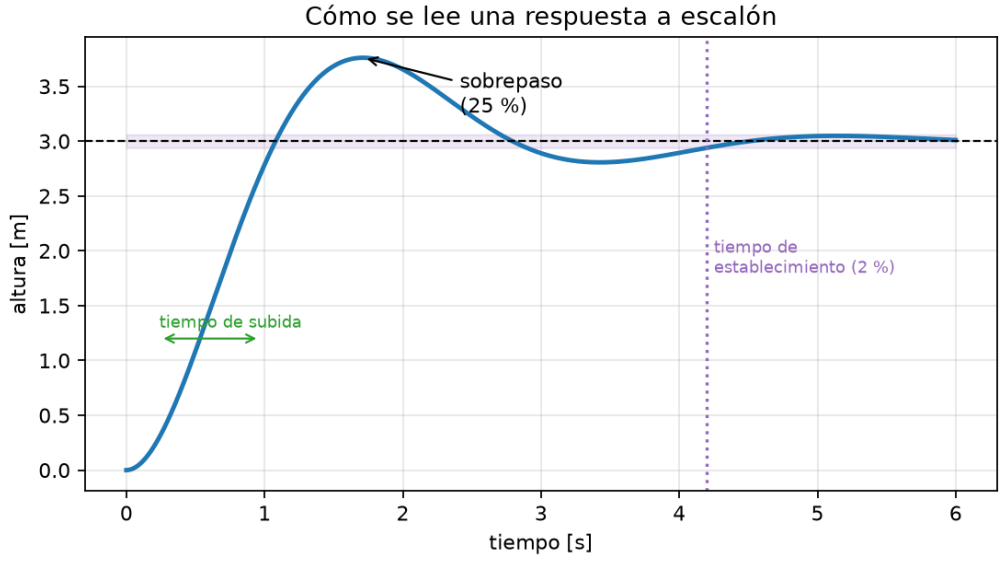



**Figura 6.3:** Medidas típicas de una respuesta a escalón.

**Sobrepaso:** cuánto se pasa del objetivo.

**Tiempo de subida:** cuánto tarda en ir del 10 % al 90 % del cambio.

**Tiempo de establecimiento:** a partir de cuándo se queda cerca del objetivo (por ejemplo, a menos del 2 %).

**Error en estado estacionario:** lo que queda de error cuando todo se calma.

En este proyecto se usa sobre todo el **error RMS** (raíz del promedio de los errores al cuadrado) durante una órbita con viento: un solo número que resume qué tan bien sigue el dron su trayectoria.

## 6.7. Control en cascada: así se controla un dron

El dron no puede "moverse a la derecha" directamente: tiene que inclinarse. Por eso hay dos lazos, uno dentro del otro:

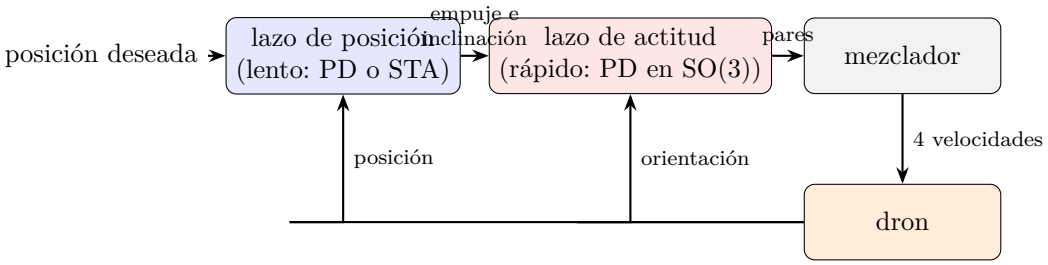



**Figura 6.4:** Control en cascada del cuadrirrotor.

1. El **lazo de posición** compara dónde está el dron con dónde debe estar y calcula qué fuerza total necesita: cuánto empuje y hacia dónde apuntarlo.
2. Apuntar el empuje significa inclinar el dron: el **lazo de actitud** calcula los pares para alcanzar esa inclinación. Es mucho más rápido que el de posición.
3. El **mezclador** convierte empuje y pares en la velocidad de cada motor.

En este proyecto, el lazo interno es siempre el mismo y lo que se cambia es el externo: el PD (línea base) o el super-twisting adaptativo, que es el tema del próximo capítulo.

**Pruébalo tú**

Compara el PD con el STA en la órbita con viento:

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque controlador:=
    pd mision:=orbita viento:=true csv:=$HOME/pd.csv
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque controlador:=
    sta mision:=orbita viento:=true csv:=$HOME/sta.csv
python3 ~/ros2_ws/src/cuadrirotor_sim/scripts/metricas.py ~/pd.csv ~/sta.csv
```

# Capítulo 7

# Modos deslizantes y super-twisting

**En este capítulo**

Qué es una superficie deslizante, por qué el control por modos deslizantes resiste perturbaciones que no conoce, qué es el *chattering*, cómo lo resuelve el super-twisting y por qué conviene que su ganancia se adapte sola.

## 7.1. El problema: perturbaciones que no conocemos

El PID funciona bien, pero sus ganancias se ajustan para un caso. Si el viento cambia mucho, o el dron lleva una carga distinta, rinde peor. El **control por modos deslizantes** (SMC, *sliding mode control*) nace con otra idea: no importa cuál sea la perturbación, siempre que no pase de cierto tamaño, el controlador la vence.

## 7.2. La superficie deslizante

Se define una variable que combina el error y su velocidad:

$$s = \dot{e} + \lambda e, \qquad \lambda > 0. \tag{7.1}$$

Si logramos que $s = 0$, entonces $\dot{e} = -\lambda e$: el error decae exponencialmente a cero solo, como $e(t) = e_0\, e^{-\lambda t}$. La recta $s = 0$ en el plano (error, velocidad del error) es la **superficie deslizante**. El trabajo se divide en dos:

1. **llegar** a la superficie, aunque haya perturbación;
2. **quedarse** en ella y "deslizarse" hasta el origen.

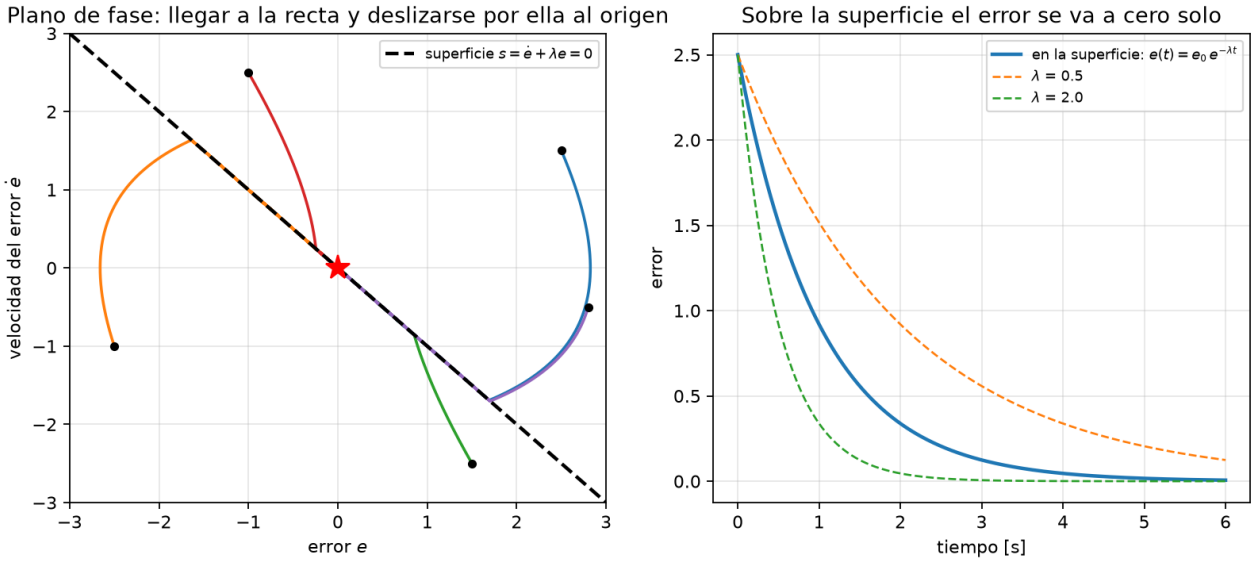



**Figura 7.1:** Izquierda: desde cualquier punto, el estado viaja hasta la recta $s = 0$ y luego se desliza por ella hasta el origen (error cero). Derecha: sobre la superficie, $\lambda$ decide qué tan rápido se va el error.

**Ejemplo de la vida diaria**

Imagina una canica en un tobogán con forma de "V". Caiga donde caiga, rueda hasta el fondo de la "V" (llegar a la superficie) y luego baja por el fondo del canal hasta la meta (deslizarse). El viento puede empujarla, pero si las paredes son lo bastante empinadas, no la saca del canal.

## 7.3. El control clásico: empujar con todo

El SMC más simple empuja siempre con la misma fuerza hacia la superficie:

$$u = -k \operatorname{sgn}(s), \tag{7.2}$$

donde $\operatorname{sgn}(s)$ vale $+1$ si $s > 0$ y $-1$ si $s < 0$. Si $k$ es mayor que la perturbación más grande, el estado siempre llega a la superficie. Es muy robusto, pero tiene un problema grave.

## 7.4. El chattering

Al llegar a $s = 0$, el control salta de $+k$ a $-k$ una y otra vez, muy rápido. Eso es el **chattering** ("castañeteo"): vibraciones que desgastan los motores, gastan batería y pueden excitar dinámicas que el modelo no considera.

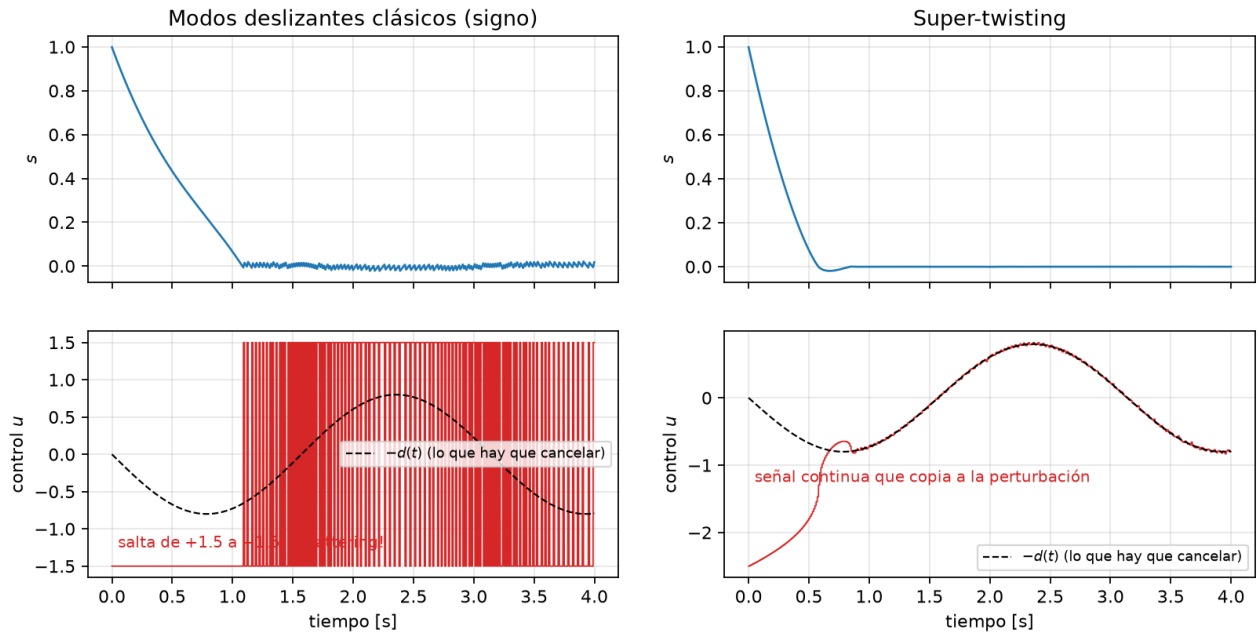



**Figura 7.2:** Izquierda: el SMC clásico lleva $s$ a cero, pero su control salta sin parar entre $+1.5$ y $-1.5$. Derecha: el super-twisting también lleva $s$ a cero, con una señal de control suave que copia a la perturbación para cancelarla.

## 7.5. Super-twisting: robusto y suave

El algoritmo **super-twisting** (STA) es un modo deslizante "de segundo orden" que esconde la parte discontinua detrás de una integral:

$$u = -k_1 |s|^{1/2} \operatorname{sgn}(s) + \nu, \tag{7.3}$$

$$\dot{\nu} = -k_2 \operatorname{sgn}(s). \tag{7.4}$$

- El primer término empuja hacia la superficie con una fuerza que se hace pequeña cerca de ella (por la raíz cuadrada).
- El segundo, $\nu$, es la integral de un signo: va creciendo o decreciendo poco a poco hasta igualar a la perturbación. Como una integral es continua, **el control $u$ es continuo**.

El resultado: $s$ y $\dot{s}$ llegan a cero en tiempo finito, aunque haya una perturbación que cambie (con derivada acotada), y sin chattering en el control continuo (figura 7.2, derecha).

**Recuerda**

El super-twisting tiene la robustez de los modos deslizantes pero entrega una señal de control continua. Necesita dos ganancias, $k_1$ y $k_2$, lo bastante grandes para la perturbación.

## 7.6. Ganancia adaptativa: no exagerar

¿Qué tan grandes deben ser $k_1$ y $k_2$? Si son pequeñas, una ráfaga fuerte gana; si son grandes, el control es agresivo todo el tiempo y amplifica el ruido. La versión **adaptativa** (Shtessel, Taleb y

Plestan, 2012) las ajusta sola:

$$\dot{k}_1 = \begin{cases} +w_1\sqrt{\gamma_1/2} & \text{si } |s| > \mu \text{ (vamos mal: subir)} \\ -w_1\sqrt{\gamma_1/2} & \text{si } |s| < \mu \text{ (vamos bien: bajar)} \end{cases} \qquad k_2 = 2\varepsilon k_1. \tag{7.5}$$

$\mu$ es una pequeña tolerancia: si $s$ se mantiene dentro de $\pm\mu$, la ganancia baja; si se sale, sube.

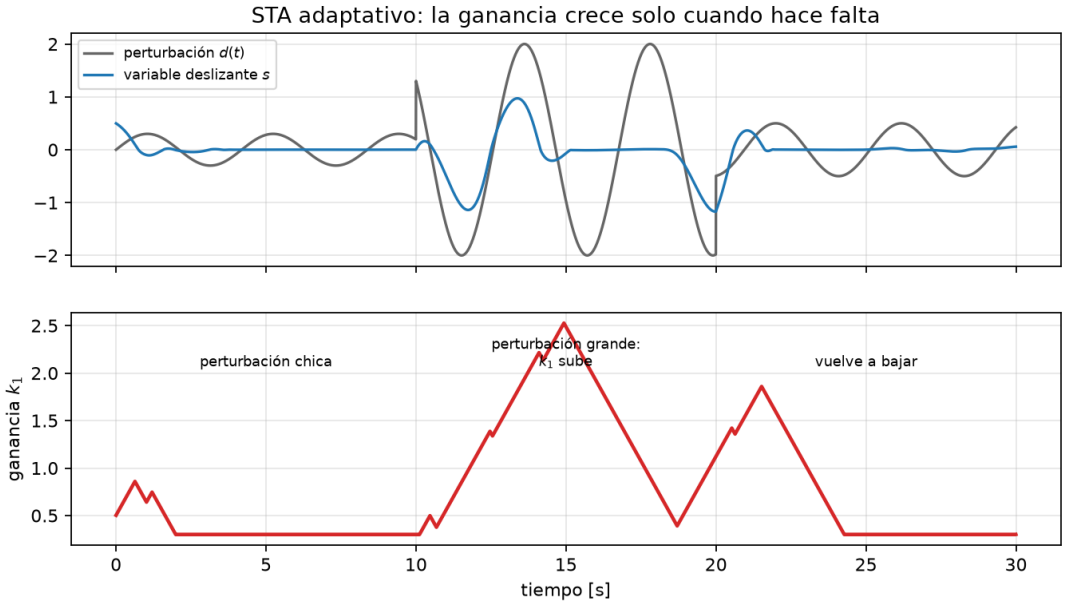



**Figura 7.3:** La perturbación crece de $t = 10$ a 20 s: la ganancia $k_1$ sube para vencerla y luego vuelve a bajar.

**Cuidado**

Elegir $\mu$ es delicado. En el dron, el retardo de los motores y del lazo de actitud produce una pequeña oscilación de $s$ que nunca desaparece (un *ciclo límite*). Si $\mu$ es menor que esa oscilación, la ganancia crece sin parar y el dron se vuelve inestable. Por eso en la simulación $\mu$ es distinto para cada eje y cada vehículo (0.3 m/s en $x$, $y$ y 0.03 m/s en $z$ para el Tarot).

## 7.7. Cómo se usa en el dron

Por cada eje ($x$, $y$, $z$) del lazo de posición:

$$s = e_v + \lambda e_p, \qquad a = a_d + \lambda e_v - u, \tag{7.6}$$

donde $e_p$ y $e_v$ son los errores de posición y velocidad y $a_d$ la aceleración que pide la trayectoria. Con esa elección resulta $\dot{s} = u + d$: exactamente el sistema para el que está diseñado el super-twisting, con $d$ la perturbación (viento y todo lo que el modelo no sabe). Los detalles están en el capítulo 9.

# Capítulo 8

# Del papel a la computadora: discretización

**En este capítulo**

Por qué una computadora no puede aplicar una ecuación "continua", qué es el periodo de muestreo, la diferencia entre discretizar de forma explícita e implícita, y por qué esto es el corazón del proyecto.

## 8.1. La computadora ve a saltos

Las ecuaciones del capítulo anterior son *continuas*: dicen qué hacer en cada instante. Una computadora, en cambio, trabaja a pasos: lee los sensores, calcula y manda la orden a los motores, y repite cada $T_s$ segundos (el **periodo de muestreo**; en la simulación, 5 ms por omisión, es decir 200 veces por segundo). Entre una orden y la siguiente, el motor recibe la misma (eso se llama **retenedor de orden cero**).

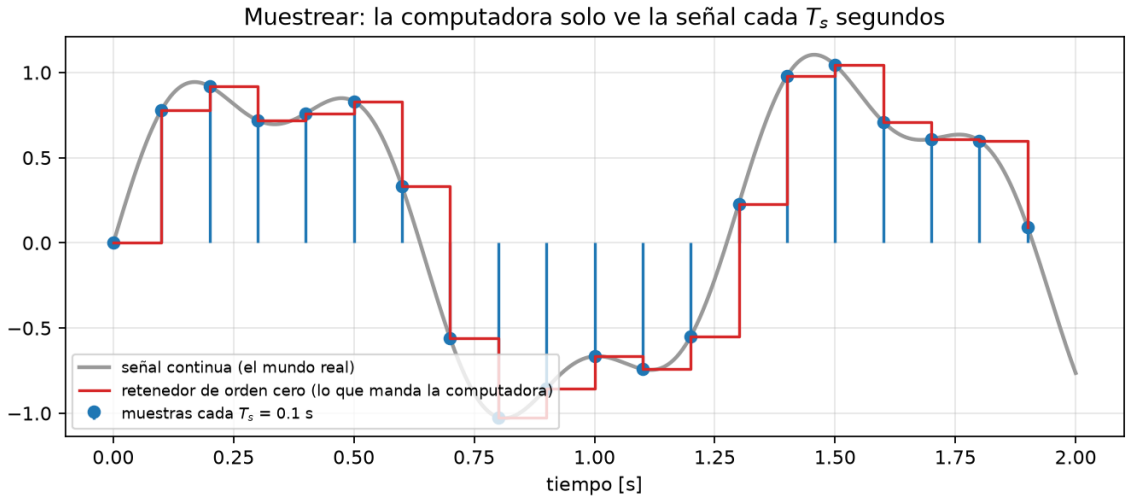



**Figura 8.1:** La computadora solo "ve" la señal en los instantes de muestreo y mantiene su orden constante entre ellos.

**Discretizar** es convertir una ley de control continua en una receta de pasos $k = 0, 1, 2, \ldots$ Hay muchas formas de hacerlo y no todas se portan igual.

## 8.2. Un ejemplo para entender: explícito contra implícito

Tomemos la ecuación más simple que se va a cero: $\dot{x} = -ax$ (por ejemplo, una taza de café que se enfría). Con paso $T_s$:

**Euler explícito** (usar el valor de *ahora*): $x_{k+1} = x_k - T_s a\, x_k = (1 - aT_s)\, x_k$.

**Euler implícito** (usar el valor *que va a tener*): $x_{k+1} = x_k - T_s a\, x_{k+1} \;\Rightarrow\; x_{k+1} = \dfrac{x_k}{1 + aT_s}$.

Si el paso es pequeño, los dos dan casi lo mismo. Pero si $aT_s > 2$, el explícito multiplica por un número de valor absoluto mayor que 1 en cada paso: **explota**, aunque la ecuación original sea perfectamente estable. El implícito divide siempre por algo mayor que 1: **nunca explota**.

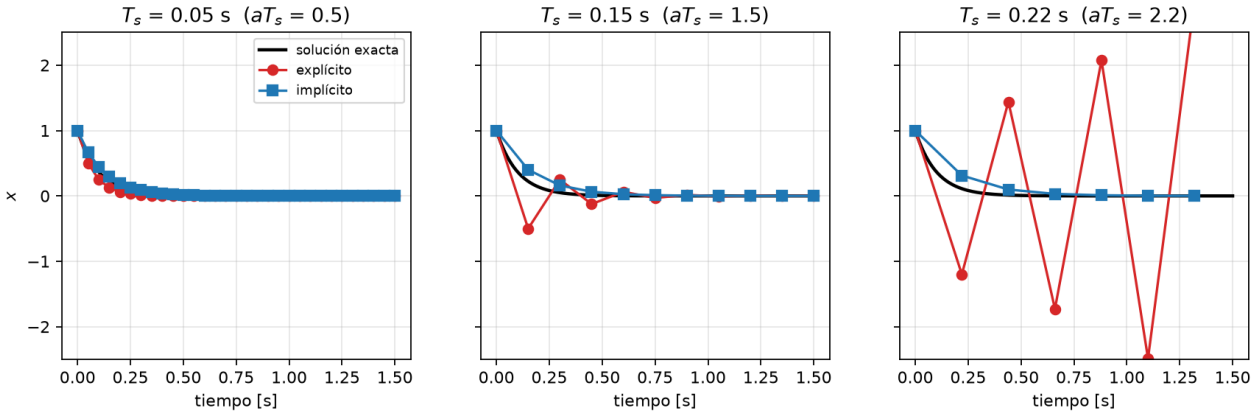



**Figura 8.2:** La misma ecuación $\dot{x} = -ax$ con $a = 10$, resuelta con pasos cada vez mayores. El implícito siempre se va a cero; el explícito oscila y, con $aT_s > 2$, diverge.

**Recuerda**

**Explícito**: calculo el siguiente paso con lo que sé ahora; es simple pero puede pasarse. **Implícito**: planteo una ecuación con el valor futuro y la resuelvo; cuesta un poco más pero respeta mejor el comportamiento del sistema continuo.

## 8.3. El problema con los modos deslizantes

Con la función signo pasa algo parecido, y peor. En la versión explícita del super-twisting:

$$u_k = -k_1 |s_k|^{1/2} \operatorname{sgn}(s_k) + \nu_k, \qquad \nu_{k+1} = \nu_k - T_s k_2 \operatorname{sgn}(s_k). \tag{8.1}$$

Cerca de $s = 0$, el signo cambia en cada paso y $s$ queda "rebotando" alrededor de cero con una amplitud que crece con la ganancia: es el **chattering numérico**, producido por la discretización y no por la física. Y como el STA adaptativo *sube* la ganancia cuando $s$ no está quieto, el chattering puede alimentar a la adaptación.

## 8.4. La discretización implícita del super-twisting

La idea (Brogliato, Polyakov y Efimov, 2020) es preguntar: "¿qué control lleva a $s$ a donde debería estar en el siguiente paso?" Se plantea una ecuación con el valor futuro $\tilde{s}$:

$$\tilde{s} + T_s k_1 |\tilde{s}|^{1/2} \xi + T_s^2 k_2 \, \xi = s_k + T_s \nu_k, \qquad \xi \in \operatorname{Sgn}(\tilde{s}), \tag{8.2}$$

donde Sgn es el signo "como conjunto": en $\tilde{s} = 0$ puede valer cualquier número entre $-1$ y $1$. Esa libertad es la clave: si el estado está cerca de la superficie, la ecuación tiene la solución $\tilde{s} = 0$ exactamente, con $\xi$ intermedio, en lugar de saltar de $+1$ a $-1$. La solución se obtiene con una fórmula cerrada (sección 9.3.1).

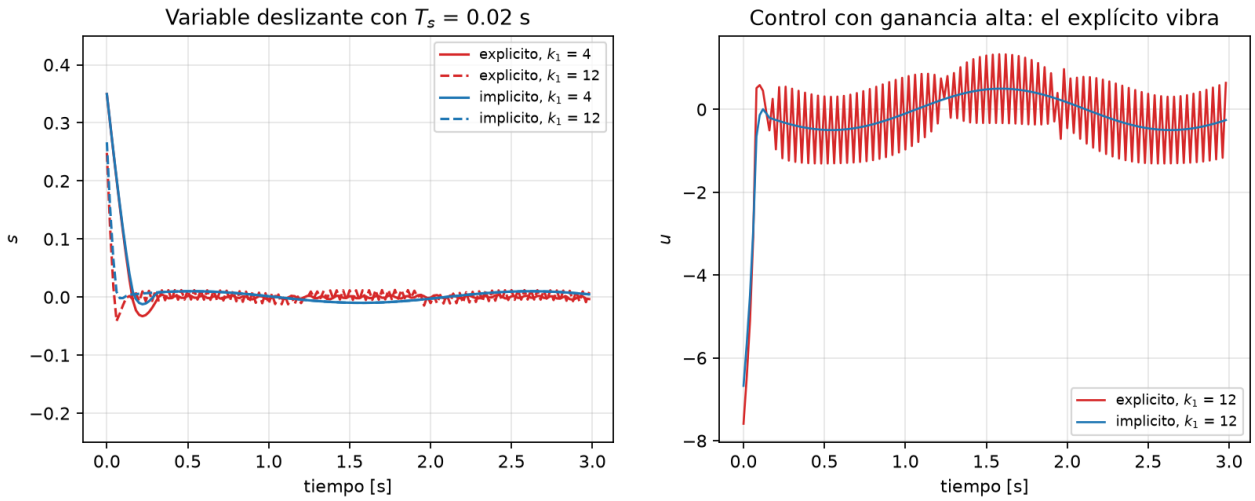



**Figura 8.3:** Super-twisting con $T_s = 20$ ms y una perturbación senoidal. Con la ganancia alta ($k_1 = 12$), el explícito vibra; el implícito da la misma señal suave que con la ganancia baja.

**Recuerda**

La hipótesis del proyecto: con la discretización implícita, el super-twisting adaptativo conserva su robustez y elimina el chattering numérico, y deja de ser sensible a que la ganancia sea demasiado grande. La simulación permite comprobarlo (y ya mostró matices, sección 9.4.1).

## 8.5. Tres versiones en el código

`sta.py` implementa las tres, y se elige al lanzar con `esquema:=`:

**`explicito`** la de arriba;

**`semi_implicito`** el signo se evalúa en el paso siguiente pero la raíz $|s|^{1/2}$ en el actual (más fácil de resolver);

**`implicito`** la ecuación completa.

**Pruébalo tú**

Corre la órbita con viento con los tres esquemas y un periodo de muestreo grande, y compara la banda de $s$ con `metricas.py`:

```
src/cuadrirotor_sim/scripts/corrida.sh ~/corridas/exp.csv 50 esquema=explicito Ts=0.02 k1_0=4.0
    adaptativo=false
src/cuadrirotor_sim/scripts/corrida.sh ~/corridas/imp.csv 50 esquema=implicito Ts=0.02 k1_0=4.0
    adaptativo=false
src/cuadrirotor_sim/scripts/metricas.py ~/corridas/*.csv
```

# Capítulo 9

# El control del dron en la simulación

**En este capítulo**

Cómo quedaron implementados, en la simulación, el PD en cascada y el super-twisting adaptativo con sus tres discretizaciones, con qué parámetros, y qué resultados dieron con el Tarot y con viento.

## 9.1. Modelo

Cuerpo rígido con posición $p \in \mathbb{R}^3$ (marco del mundo), orientación $R \in SO(3)$ y velocidad angular $\omega$ (marco del cuerpo):

$$m\ddot{p} = -mg\,e_3 + TRe_3 + d(t), \qquad J\dot{\omega} = -\omega \times J\omega + \tau, \qquad \dot{R} = R\,\hat{\omega}, \tag{9.1}$$

con $T$ y $\tau$ producidos por los motores a través del mezclador y $d(t)$ la perturbación (viento). Gazebo integra el modelo completo (incluidos el retardo de los motores, el arrastre de los rotores y el contacto con el suelo); el controlador usa los mismos $m$, $J$, $k_f$ y $k_m$.

## 9.2. Controlador PD en cascada (línea base)

`controlador_pd.py` corre cada $T_s$ (5 ms por omisión) de tiempo simulado y manda las velocidades de rotor con retenedor de orden cero.

**Lazo externo.** Con $e_p = p_d - p$ y $e_v = v_d - v$:

$$F = m\,(a + g\,e_3), \qquad a = a_d + K_p e_p + K_d e_v, \qquad T = F \cdot Re_3, \tag{9.2}$$

con $K_p = \mathrm{diag}(4, 4, 6)$ y $K_d = \mathrm{diag}(3, 3, 4)$.

**Actitud deseada.** A partir de la dirección de $F$ y la guiñada deseada $\psi_d$:

$$b_3 = \frac{F}{\|F\|}, \quad b_{1c} = (\cos\psi_d, \sin\psi_d, 0), \quad b_2 = \frac{b_3 \times b_{1c}}{\|b_3 \times b_{1c}\|}, \quad R_d = [\,b_2 \times b_3,\ b_2,\ b_3\,]. \tag{9.3}$$

**Lazo interno** (PD sobre SO(3)):

$$e_R = \tfrac{1}{2}\big(R_d^\top R - R^\top R_d\big)^\vee, \qquad \tau = -K_R e_R - K_\omega\,\omega + \omega \times J\omega. \tag{9.4}$$

## 9.3. Super-twisting adaptativo (STA)

`controlador_sta.py` reemplaza la aceleración del lazo externo. Por eje:

$$s = e_v + \lambda e_p, \qquad a = a_d + \lambda e_v - u \quad \Rightarrow \quad \dot{s} = u + d(t), \tag{9.5}$$

y $u$ es el super-twisting con ganancia adaptativa (Shtessel, Taleb y Plestan, 2012):

$$u = -k_1 |s|^{1/2} \operatorname{sgn}(s) + \nu, \qquad \dot{\nu} = -k_2 \operatorname{sgn}(s), \tag{9.6}$$

$$\dot{k}_1 = \begin{cases} w_1 \sqrt{\gamma_1/2}\ \operatorname{sgn}(|s| - \mu) & k_1 > k_{1,min} \\ \eta & \text{en otro caso} \end{cases}, \qquad k_2 = 2\varepsilon k_1. \tag{9.7}$$

### 9.3.1. Discretizaciones

Con periodo $T_s$ y $a_k = s_k + T_s \nu_k$, `sta.py` implementa:

**Explícita** $u_k = -k_1 |s_k|^{1/2} \operatorname{sgn}(s_k) + \nu_k, \quad \nu_{k+1} = \nu_k - T_s k_2 \operatorname{sgn}(s_k)$.

**Semi-implícita** el término discontinuo se evalúa en el paso siguiente y $|s|^{1/2}$ en el actual: $\tilde{s} + (T_s k_1 |s_k|^{1/2} + T_s^2 k_2)\, \xi = a_k$, con $\xi \in \operatorname{Sgn}(\tilde{s})$.

**Implícita** (Brogliato, Polyakov y Efimov, 2020):

$$\tilde{s} + T_s k_1 |\tilde{s}|^{1/2} \xi + T_s^2 k_2\, \xi = a_k, \qquad \xi \in \operatorname{Sgn}(\tilde{s}), \tag{9.8}$$

cuya solución única es

$$\begin{cases} \tilde{s} = 0, \quad \xi = a_k / (T_s^2 k_2) & \text{si } |a_k| \le T_s^2 k_2, \\ \tilde{s} = \operatorname{sgn}(a_k)\, w^2, \quad w = \dfrac{-T_s k_1 + \sqrt{T_s^2 k_1^2 + 4(|a_k| - T_s^2 k_2)}}{2} & \text{en otro caso.} \end{cases} \tag{9.9}$$

En los dos esquemas implícitos $\tilde{s}$ es la predicción nominal de $s_{k+1}$, $\nu_{k+1} = \nu_k - T_s k_2 \xi$ y $u_k = (\tilde{s} - s_k)/T_s$. El código de la solución implícita:

```
@staticmethod
def resolver_implicito(a, b, c):
    """Solución única de  x + b |x|^(1/2) xi + c xi = a,  xi en Sgn(x),  b, c > 0."""
    abs_a = np.abs(a)
    dentro = abs_a <= c
    w = (-b + np.sqrt(b**2 + 4.0 * np.maximum(abs_a - c, 0.0))) / 2.0
    x = np.where(dentro, 0.0, np.sign(a) * w**2)
    xi = np.where(dentro, a / np.where(dentro, c, 1.0), np.sign(a))
    return x, xi
```

### 9.3.2. Parámetros

En `config/parametros.yaml`, sección `controlador_sta`: $\lambda = (1.5, 1.5, 2.0)$, $k_{1,0} = 1$, $\varepsilon = 1$, $w_1 = 2$, $\gamma_1 = 1$, $k_{1,min} = 0.5$, $\eta = 0.1$. Los que dependen del vehículo van en su YAML: para el Tarot $\mu = (0.3, 0.3, 0.03)$ y $k_{1,max} = 3$. El esquema se elige al lanzar:

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque \
    controlador:=sta esquema:=implicito mision:=orbita viento:=true csv:=$HOME/corrida.csv
```

## 9.4. Resultados

La misión `orbita` despega a 3 m, se traslada y describe una órbita de radio 4 m alrededor del foco $(8, 5)$ con la cámara apuntando a él, con viento desde $t = 15$ s. Las métricas las calcula `scripts/metricas.py`.

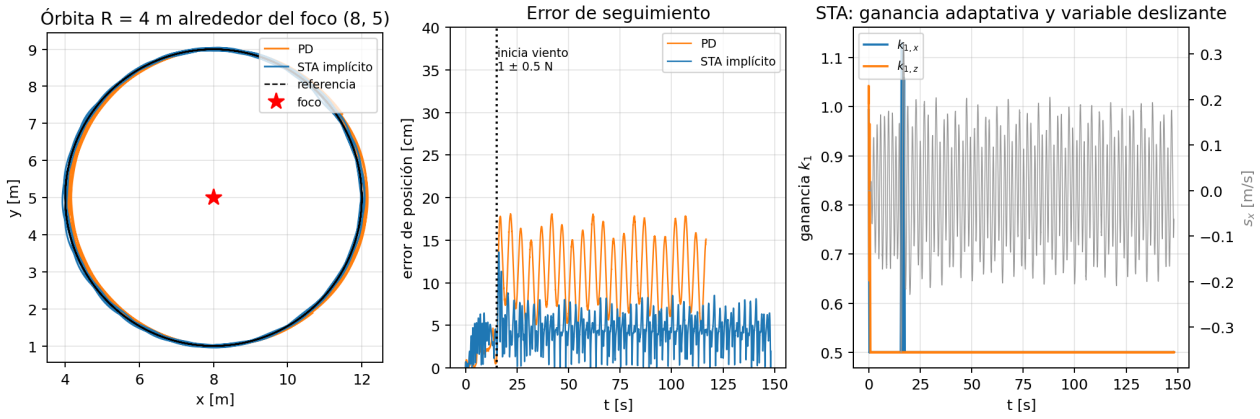



**Figura 9.1:** Tarot 650 en el mundo `bosque`, órbita con viento de $1 \pm 0.5$ N (corridas del 23 de octubre). El STA implícito reduce el error RMS con viento de 11.9 cm (PD) a 4.8 cm. La ganancia $k_1$ baja hasta $k_{1,min}$: el umbral $\mu$ del Tarot queda por encima del ciclo límite.

| Vehículo | Controlador | RMS sin viento | RMS con viento | Máx. con viento |
|---|---|---|---|---|
| cuadrirrotor | PD en cascada | — | 17.7 cm | 28.5 cm |
| cuadrirrotor | STA explícito | — | 2.4 cm | 4.7 cm |
| cuadrirrotor | STA semi-implícito | — | 2.1 cm | 4.0 cm |
| cuadrirrotor | STA implícito | — | 2.1 cm | 4.3 cm |
| Tarot 650 | PD en cascada | 2.8 cm | 11.9 cm | 18.1 cm |
| Tarot 650 | STA implícito | 3.3 cm | 4.8 cm | 8.5 cm |
| Tarot 650 | ArduCopter 4.7.2 | — | 13.7 cm | — |

**Tabla 9.1:** Error de seguimiento en la órbita ($T_s = 5$ ms). Las filas del cuadrirrotor y la de ArduPilot son del 21–22 de octubre; las del Tarot con PD y STA, de las corridas de la figura 9.1.

### 9.4.1. Observaciones

1. Con perturbación, el esquema implícito de las ec. (9)–(12) deja una banda $|s| \approx T_s|d|$, **independiente de la ganancia** (insensible a la sobreestimación), pero no proporcional a $T_s^2 k_2$ como plantea la hipótesis: la predicción nominal supone $d = 0$. Con `estimar_perturbacion:true` (estimación $\hat{d}$ del paso anterior; Acary, Brogliato y Orlov, 2012) la banda baja a $\approx T_s^2$ máx $|\dot{d}|$.
2. En $x$, $y$ el retardo del lazo de actitud y de los motores induce un ciclo límite ($|s| \approx 0.05$–$0.1$ m/s en el cuadrirrotor, 0.2–0.35 m/s en el Tarot). Si $\mu$ queda por debajo de él, la ganancia crece sin control y el lazo se inestabiliza; por eso $\mu$ es por eje y por vehículo.
3. La velocidad que publica Gazebo es un promedio de 10 muestras (≈4.5 ms de retraso a 1 kHz): forma parte de la dinámica no modelada.

## 9.5. Agregar un controlador

Se hereda de `ControladorPD`, se sobrescribe `aceleracion(ep,ev,ad)` y se registra el ejecutable en `setup.py`. Así quedó el STA:

```
def aceleracion(self, ep, ev, ad):
    self.s = ev + self.lam * ep
    return ad + self.lam * ev - self.sta(self.s)
```

# Parte III

# Ver el mundo: visión computacional

# Capítulo 10

# Qué es una imagen para una computadora

**En este capítulo**

Píxeles y canales de color, cómo forma una imagen una cámara (modelo estenopeico), qué es el campo de visión, qué ve la cámara del dron y cómo se pasa de un píxel a un punto del mundo.

## 10.1. Una imagen es una tabla de números

Para nosotros una foto es una escena; para la computadora es una **tabla de números**. La cámara del dron entrega imágenes de 640 columnas por 480 filas: 307 200 **píxeles** (de *picture element*). Cada píxel tiene tres números entre 0 y 255: cuánto rojo, cuánto verde y cuánto azul (RGB). $(0, 0, 0)$ es negro, $(255, 255, 255)$ blanco y $(230, 140, 55)$ un naranja de fuego.

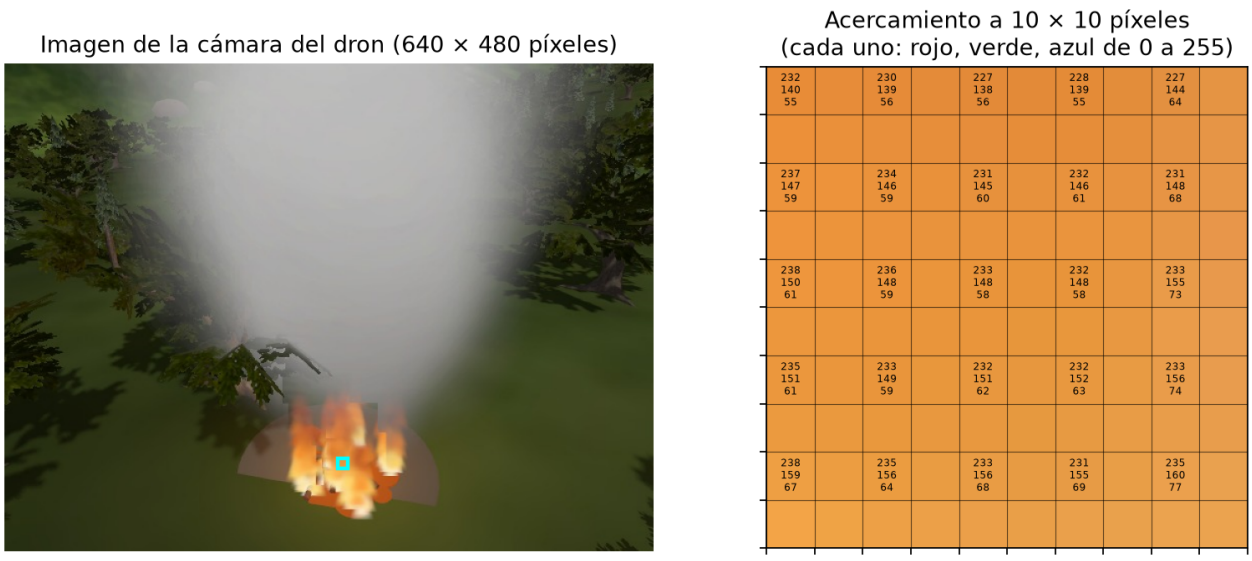



**Figura 10.1:** Una imagen de la cámara del dron y un acercamiento a 10×10 píxeles de las llamas: mucho rojo (230), algo de verde (145) y poco azul (60).

Las tres tablas (rojo, verde, azul) se llaman **canales**. Verlas por separado ayuda a entender por qué el fuego es fácil de distinguir: brilla mucho en el canal rojo y poco en el azul, mientras que el humo es claro en los tres.

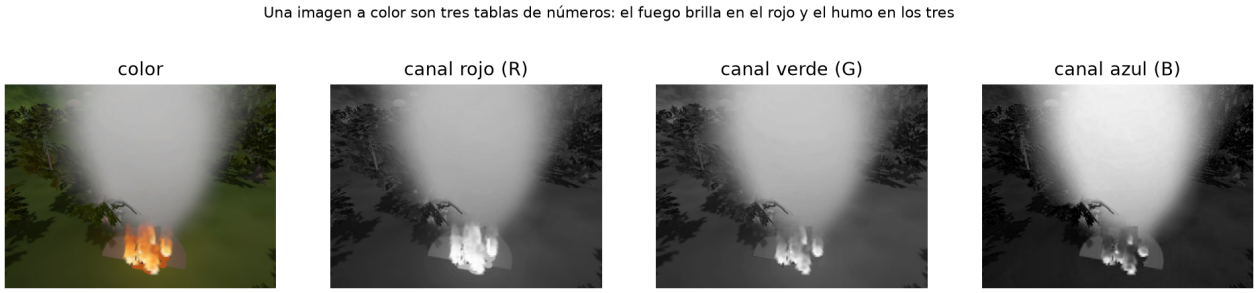



**Figura 10.2:** La misma imagen separada en sus tres canales (más claro = valor más alto).

**Ejemplo de la vida diaria**

Una imagen es como una hoja de cálculo gigante donde cada celda está pintada de un color. Un programa de visión "lee" esa hoja buscando patrones de números, igual que tú buscarías un patrón de colores.

Un video son imágenes una tras otra: la cámara simulada entrega 15 por segundo en el tópico `/cuadrirotor/camara`.

## 10.2. Cómo forma una imagen una cámara

El modelo más útil es el de la **cámara estenopeica** (*pinhole*): una caja oscura con un agujerito. La luz de cada punto del mundo entra en línea recta por el agujero y llega a un punto del sensor. Por

semejanza de triángulos, un punto a distancia $Z$ y desplazado $X$ del eje aparece en la imagen a

$$u = f\,\frac{X}{Z}, \tag{10.1}$$

donde $f$ es la **distancia focal** (en píxeles). Los objetos lejanos se ven más pequeños: dividir entre $Z$ es la *perspectiva*.

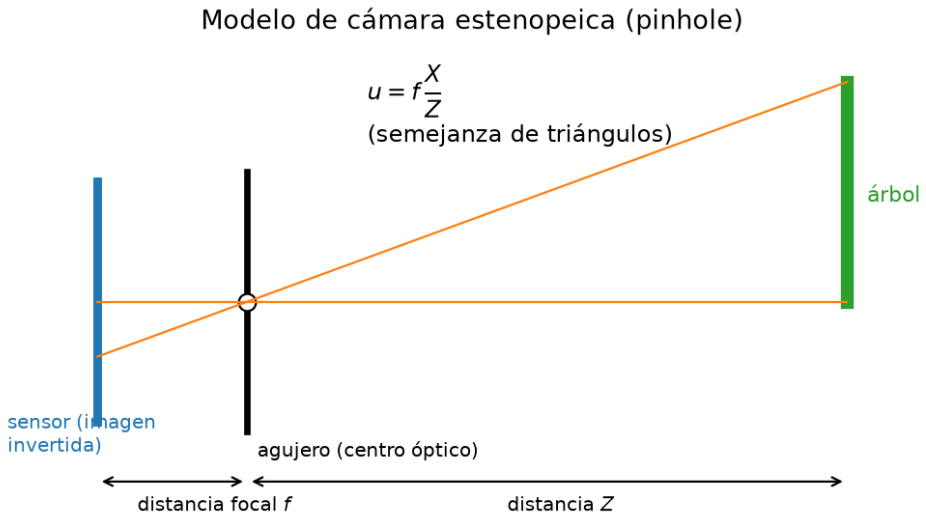



**Figura 10.3:** El modelo estenopeico: la imagen se forma por rayos rectos que pasan por el centro óptico.

## 10.3. Campo de visión

El **campo de visión** (FOV) es qué tan ancho ve la cámara. La del dron tiene un campo horizontal de 1.3 rad (74°); con una imagen de 640 píxeles de ancho eso da

$$f = \frac{640/2}{\tan(1.3/2)} \approx 418 \text{ píxeles.} \tag{10.2}$$

Va inclinada 30° hacia abajo: así ve el horizonte (para descubrir humo lejos) y también el suelo delante del dron.

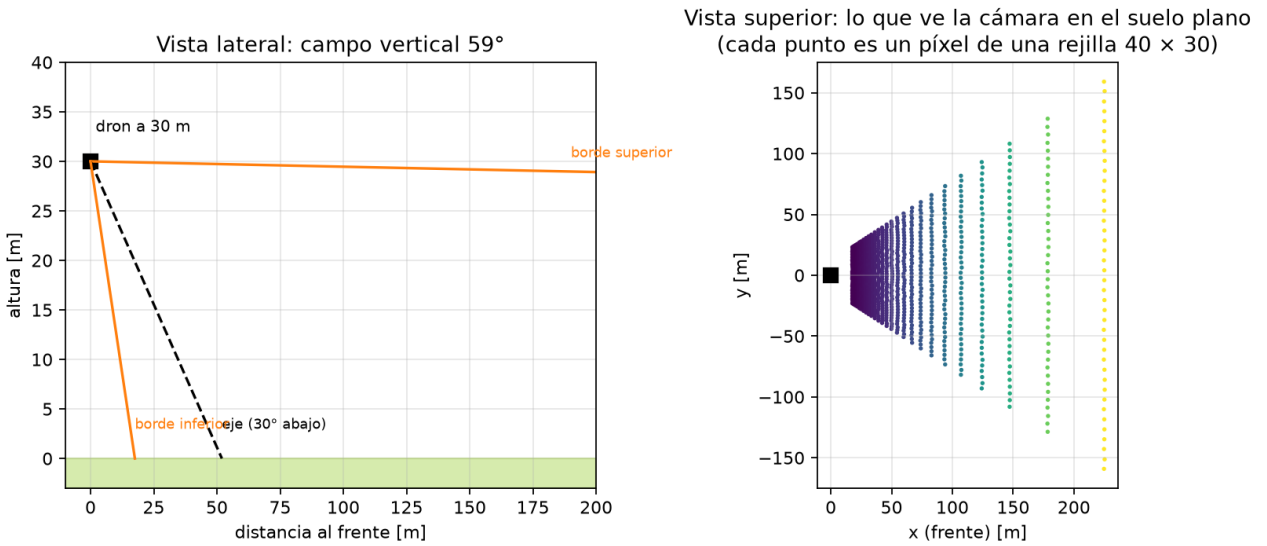



**Figura 10.4:** Qué ve la cámara desde 30 m de altura. Izquierda: el borde superior de la imagen casi toca el horizonte y el inferior cae a 18 m del dron. Derecha: la "huella" de la imagen en el suelo; los píxeles de arriba cubren mucho más terreno que los de abajo.

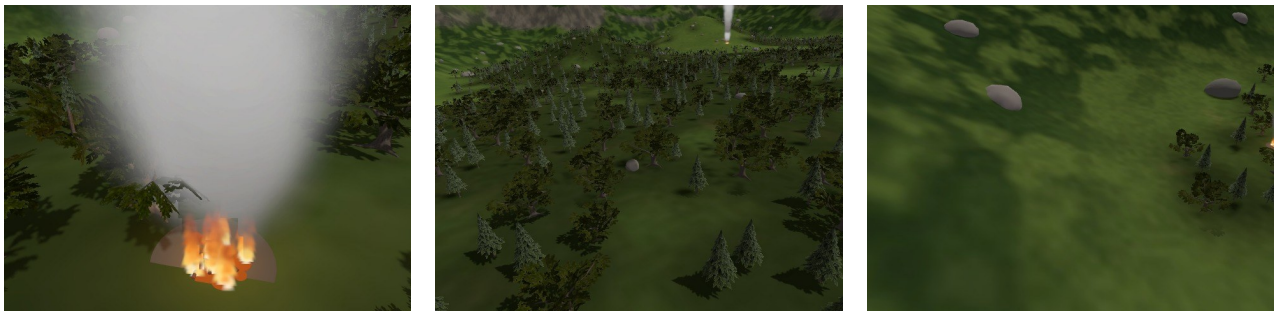


**Figura 10.5:** Imágenes de la cámara del dron simulado: fuego y humo de cerca, humo a lo lejos (la columna blanca arriba a la derecha) y una escena sin fuego con rocas.

## 10.4. Del píxel al mundo: rayos

La cámara pierde una dimensión: un píxel no dice a qué distancia está lo que muestra, solo en qué *dirección*. Cada píxel corresponde a un **rayo** que sale de la cámara. Invirtiendo el modelo estenopeico, el rayo del píxel $(u, v)$ en el marco de la cámara es

$$d_{cam} = \left(1,\ -\frac{u - W/2}{f},\ -\frac{v - H/2}{f}\right), \tag{10.3}$$

y se pasa al mundo con la orientación del dron y de la cámara. Para saber *dónde* está lo que se ve, hace falta otra información:

- un modelo del terreno: el punto está donde el rayo toca el suelo (así se estima el fuego, capítulo 17);
- o varias vistas desde lugares distintos: donde se cruzan los rayos (triangulación, como se hace con el humo).

**Recuerda**

Una cámara da direcciones, no distancias. Para ubicar algo en el mapa hay que cruzar su rayo con el terreno o con otros rayos.

# Capítulo 11

# Redes neuronales y YOLO sin complicaciones

**En este capítulo**

Qué es aprender a partir de ejemplos, cómo funciona una neurona artificial, qué hace una convolución, cómo detecta objetos YOLO, cómo se entrena (datos, etiquetas, épocas, pérdida) y cómo se mide qué tan bien funciona (IoU, precisión, recall, mAP).

## 11.1. Programar contra aprender

¿Cómo le explicarías a una computadora qué es el humo? Podrías intentar reglas: "muchos píxeles grises claros juntos". Pero las nubes también lo son, y el humo a contraluz es oscuro. Las reglas escritas a mano fallan pronto. El **aprendizaje automático** hace lo contrario: se le dan miles de ejemplos con la respuesta correcta ("aquí hay humo, en este rectángulo") y el programa ajusta sus propios parámetros hasta acertar. Eso es **entrenar**.

## 11.2. La neurona artificial

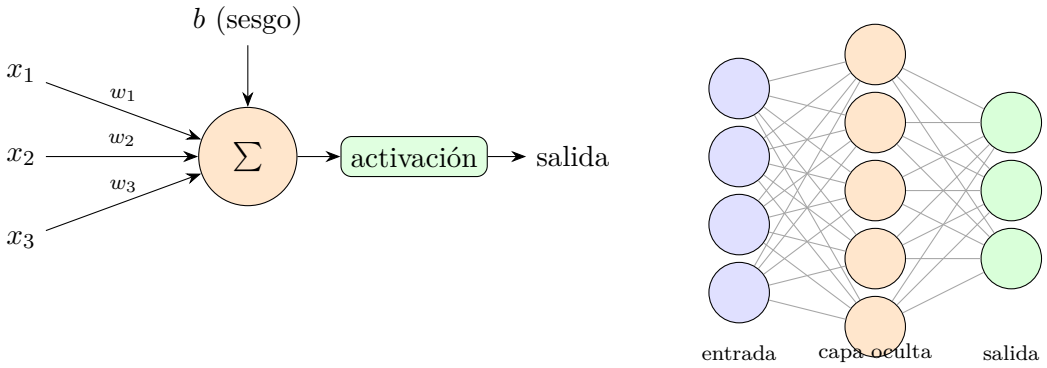



**Figura 11.1:** Izquierda: una neurona multiplica cada entrada por un *peso*, suma todo y pasa el resultado por una función de activación. Derecha: una red son muchas neuronas en capas.

Una neurona artificial hace una cuenta muy simple:

$$\text{salida} = \text{activación}\big(w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b\big). \tag{11.1}$$

Los **pesos** $w_i$ dicen qué tan importante es cada entrada. Sola, una neurona es poca cosa; millones de ellas en capas pueden reconocer caras, voces o fuego. Entrenar es buscar los pesos que hacen que la red acierte. YOLO26n tiene unos 2.5 millones de pesos.

## 11.3. Convoluciones: buscar patrones locales

Conectar cada píxel con cada neurona sería enorme. Las redes para imágenes usan **convoluciones**: un filtro pequeño (por ejemplo $3 \times 3$ números) que se desliza por toda la imagen; en cada posición multiplica sus números por los píxeles de abajo y suma. Según los números del filtro, resalta bordes, manchas, texturas...

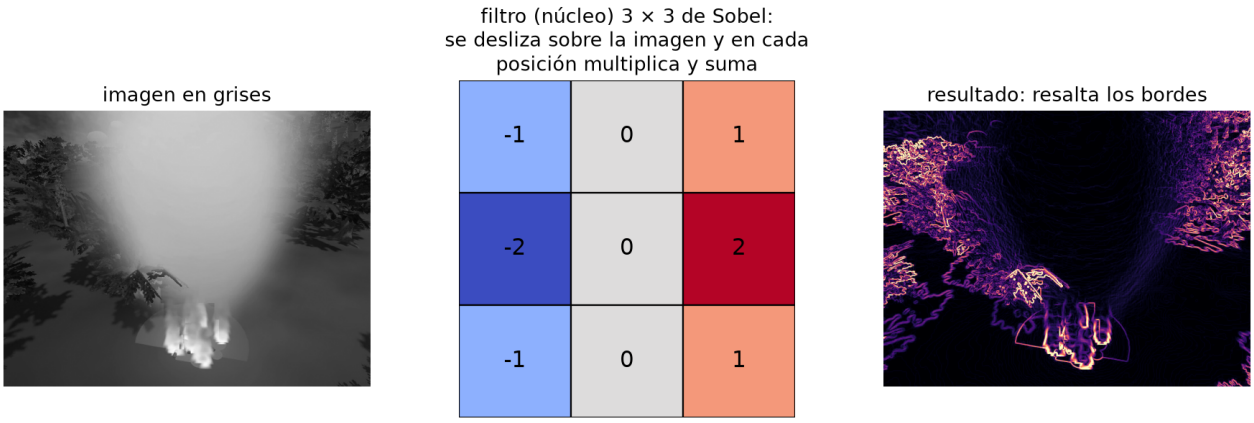



**Figura 11.2:** Un filtro de bordes (Sobel) aplicado a la imagen del incendio. En una red convolucional los números de los filtros no se eligen a mano: se aprenden.

Una **red convolucional** (CNN) apila muchas capas de filtros: las primeras detectan bordes y colores, las intermedias texturas (llamas, humo, hojas) y las últimas objetos completos.

## 11.4. Detectar objetos: cajas y clases

Hay varias tareas de visión: *clasificar* (¿hay fuego en la imagen?), *detectar* (¿dónde?) y *segmentar* (¿qué píxeles exactamente?). Necesitamos **detección**: el dron debe saber en qué parte de la imagen está el fuego para calcular hacia dónde está. Un detector entrega una lista de **cajas** (rectángulos), cada una con una **clase** ("fuego" o "humo") y una **confianza** entre 0 y 1.

## 11.5. YOLO

**YOLO** (*You Only Look Once*, "solo miras una vez") es una familia de detectores muy rápida: en lugar de revisar la imagen por pedazos, una sola pasada de la red predice todas las cajas a la vez. Por eso sirve en tiempo real, incluso en un dron. Usamos **YOLO26n**, la versión más reciente de Ultralytics en su tamaño más pequeño (*nano*); en la GPU procesa una imagen en pocos milisegundos.

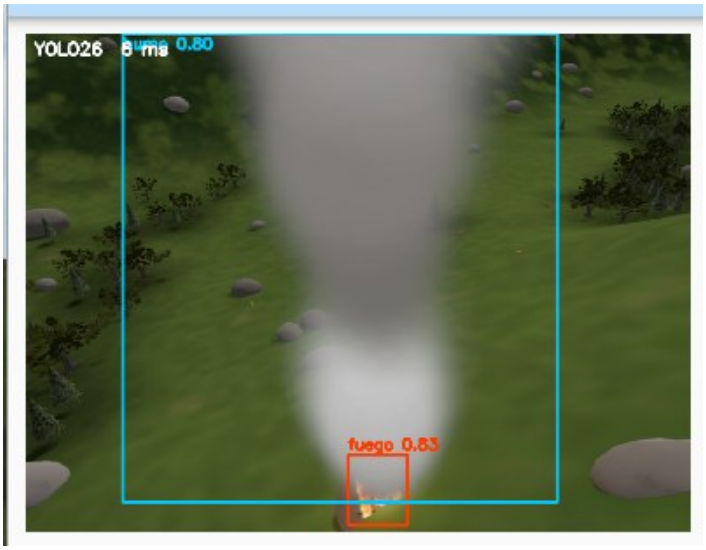



**Figura 11.3:** Lo que entrega YOLO: cajas con su clase y su confianza ("fuego 0.83").

## 11.6. Entrenar: datos, etiquetas y épocas

**Dataset:** el conjunto de imágenes de ejemplo. Se usaron 6000 imágenes del simulador.

**Etiquetas:** para cada imagen, las cajas correctas. En formato YOLO, un archivo de texto con una línea por objeto: clase y caja (centro, ancho y alto, como fracción de la imagen).

**Entrenamiento y validación:** el dataset se parte en dos. La red aprende con uno (*entrenamiento*) y se evalúa con el otro, que nunca ve al aprender (*validación*). Así se mide si generaliza o solo memoriza.

**Época:** una pasada completa por todas las imágenes de entrenamiento. Se hicieron 60.

**Pérdida (loss):** un número que mide qué tan mal predice la red. Entrenar es bajarla, ajustando los pesos un poquito en la dirección que la reduce (*descenso de gradiente*).

**Ajuste fino (fine-tuning):** no se empezó de cero, sino de una red ya entrenada con el dataset COCO (80 clases de objetos comunes). La red ya sabe "ver" y solo aprende las clases nuevas.




**Figura 11.4:** Un lote de imágenes de entrenamiento con sus etiquetas. Las imágenes se alteran al azar (mosaicos, cambios de color y escala) para que la red no memorice.

**Para curiosos (te lo puedes saltar)**

Etiquetar 6000 imágenes a mano tomaría días. Como en el simulador sabemos dónde está el fuego, las etiquetas se calcularon solas: se proyecta con el modelo estenopeico una nube de puntos del volumen de las llamas y del humo, se descartan los puntos tapados por árboles o por el terreno, y la caja es el rectángulo que encierra los puntos visibles (capítulo 12).

## 11.7. Cómo se mide si funciona

### 11.7.1. IoU: ¿le atinó a la caja?

Una predicción cuenta como acierto si su caja se parece lo bastante a la verdadera. Se mide con la **intersección sobre unión** (IoU):

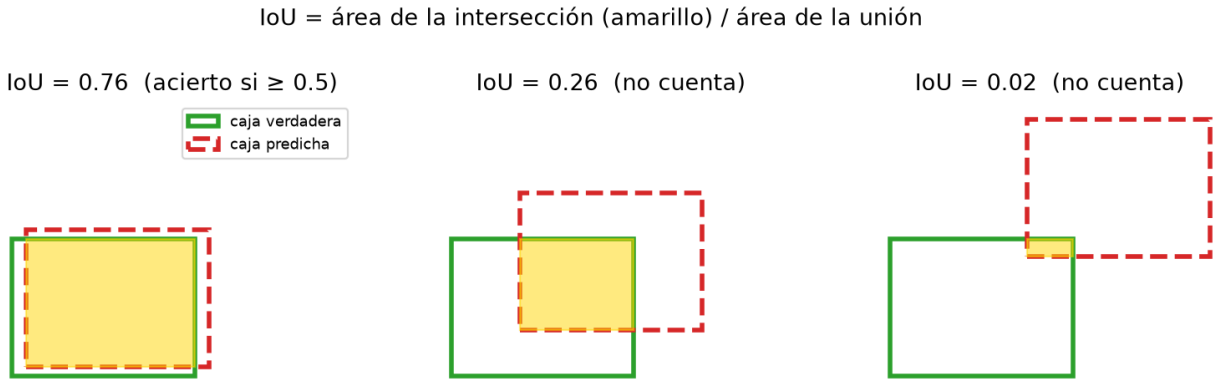



**Figura 11.5:** IoU: 1 es perfecta, 0 es no tocarse. Normalmente se exige al menos 0.5.

### 11.7.2. Precisión y recall

**Verdadero positivo (VP):** dijo "fuego" y había fuego.

**Falso positivo (FP):** dijo "fuego" y no había (por ejemplo, una roca oscura).

**Falso negativo (FN):** había fuego y no lo vio.

$$\text{precisión} = \frac{VP}{VP + FP} \qquad \text{(cuando dice "fuego", ¿cuántas veces acierta?)} \tag{11.2}$$

$$\text{recall} = \frac{VP}{VP + FN} \qquad \text{(de los fuegos que hay, ¿cuántos encuentra?)} \tag{11.3}$$

**Ejemplo de la vida diaria**

Una alarma de humo muy sensible tiene buen recall (no se le escapa ningún incendio) pero mala precisión (suena cuando quemas pan tostado). Una muy poco sensible, al revés. Para un dron que vigila un bosque importan las dos: no queremos ignorar un incendio, pero tampoco mandar a los bomberos a ver una roca.

### 11.7.3. mAP

La **precisión media** (AP) resume la curva de precisión contra recall al variar el umbral de confianza; promediada entre clases es la **mAP**. *mAP50* usa IoU $\geq 0.5$; *mAP50-95* promedia exigencias de IoU de 0.5 a 0.95 (cajas más ajustadas). Nuestro modelo final: mAP50 = 0.988 y mAP50-95 = 0.881, precisión 0.97 y recall 0.95.

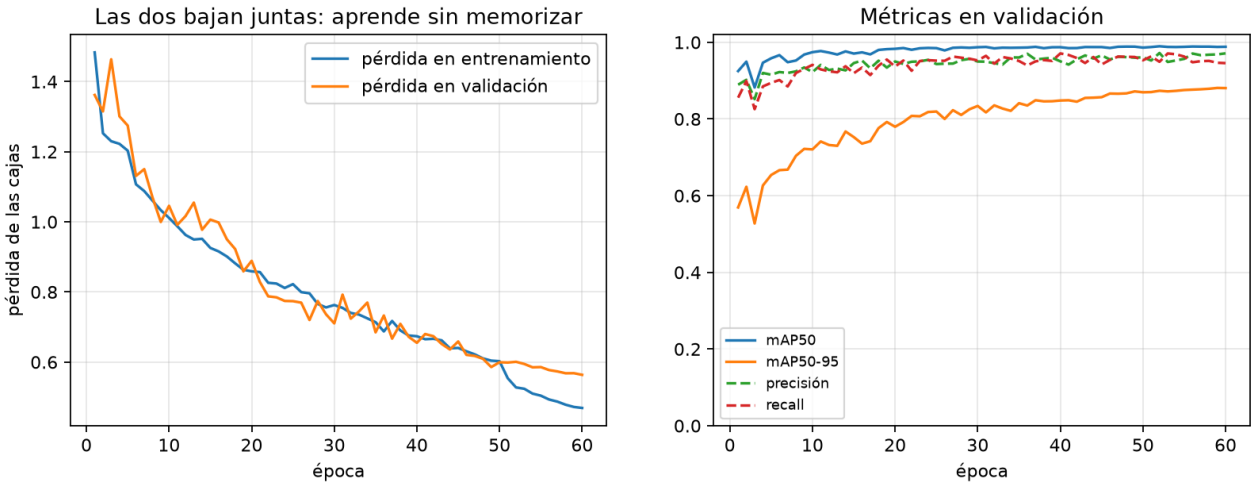



**Figura 11.6:** Izquierda: la pérdida baja en entrenamiento y en validación a la par (no hay sobreajuste). Derecha: las métricas suben y se estabilizan.

## 11.8. Sobreajuste y por qué importan los datos

Si la red memoriza las imágenes de entrenamiento en lugar de aprender el concepto, le va muy bien con ellas y mal con imágenes nuevas: eso es **sobreajuste**. Se detecta porque la pérdida de validación deja de bajar o sube. Más importante aún: **la red solo sabe lo que vio**.

**Problema encontrado y cómo se resolvió**

El primer modelo se entrenó solo con el mundo `bosque` (plano, sin rocas grandes). En el mundo grande confundió una roca oscura con fuego: nunca había visto una roca. Se agregaron 3000 imágenes del mundo grande, una de cada cinco apuntando a rocas o animales sin fuego, y el problema desapareció.

**Cuidado**

Un modelo entrenado solo con imágenes de simulación probablemente fallará con fuego real: los colores, la luz y el humo reales son distintos. Antes de volar el dron de verdad hay que reentrenar con imágenes reales (por ejemplo, el dataset D-Fire).

# Capítulo 12

# El detector de incendios de la simulación

**En este capítulo**

Cómo se generaron las 6000 imágenes etiquetadas, cómo se entrenó YOLO26n, qué tan bien quedó y cómo funciona el nodo que detecta fuego y humo durante el vuelo.

## 12.1. Por qué ajustar un modelo

YOLO26 es la última versión publicada de Ultralytics. Los modelos preentrenados en COCO no tienen clases de fuego ni de humo, así que se ajustó **YOLO26n** (el más rápido de la familia, adecuado para un dron) con imágenes del propio simulador, etiquetadas automáticamente. Las clases son *fuego* y *humo*.

## 12.2. Captura del dataset

`scripts/capturar_dataset.py` abre el mundo sin dron, con una cámara de la misma óptica que la del dron (640×480, HFOV 1.3 rad), y repite: mueve el foco y la cámara a posiciones al azar, toma una imagen y la etiqueta. Para etiquetar proyecta con el modelo de cámara estenopeica una nube de puntos del volumen de las llamas y de la columna de humo; descarta los puntos que tapan las copas de los árboles (y, en el mundo grande, el relieve), así que cada caja cubre solo la parte visible.

```
# Mundo 'bosque'
python3 scripts/capturar_dataset.py preparar /tmp/datos/bosque.sdf /tmp/cuadrirotor_sim_$UID/
    modelos
GZ_SIM_RESOURCE_PATH=/tmp/datos/modelos_captura:$PWD/models \
    gz sim -s -r --headless-rendering /tmp/datos/bosque.sdf &
python3 scripts/capturar_dataset.py capturar ~/datasets/incendio_bosque 3000

# Mundo 'bosque_grande' (relieve, rocas, animales, cielo y penacho alto)
```

```
python3 scripts/capturar_dataset.py preparar /tmp/datos/bg.sdf /tmp/cuadrirotor_sim_$UID/modelos
    bosque_grande
python3 scripts/capturar_dataset.py capturar ~/datasets/incendio_bosque_grande 3000 bosque_grande
```

En el mundo grande el foco se pone en cualquier punto del valle (se espera 30 s cada vez que se mueve para que el humo se forme), y una de cada cinco tomas apunta a una roca o a un animal: son ejemplos de lo que *no* es fuego. Se capturaron 3000 imágenes en cada mundo; en el grande, 989 con fuego, 2111 con humo y 836 vacías.

**Problema encontrado y cómo se resolvió**

Con el primer modelo, entrenado solo con el bosque plano, una roca oscura de 2.6 m en $(-129, -154)$ del mundo grande se detectaba como *fuego*: se parece al suelo quemado del foco. Se resolvió capturando el segundo dataset, con tomas dirigidas a rocas y animales, y reentrenando con los dos juntos.

## 12.3. Entrenamiento

```
~/.venvs/yolo/bin/python scripts/entrenar_yolo.py ~/datasets/incendio_bosque \
    ~/datasets/incendio_bosque_grande 60
```

El script combina los datasets (escribe `~/datasets/combinado_.../data.yaml`), ajusta YOLO26n partiendo de los pesos de COCO (60 épocas, lote de 32, imágenes de 640) en la GPU y copia el mejor modelo a `yolo/incendio_yolo26n.pt`. Resultado en el conjunto de validación: precisión 0.971, *recall* 0.946, **mAP50 = 0.988** y mAP50-95 = 0.881.

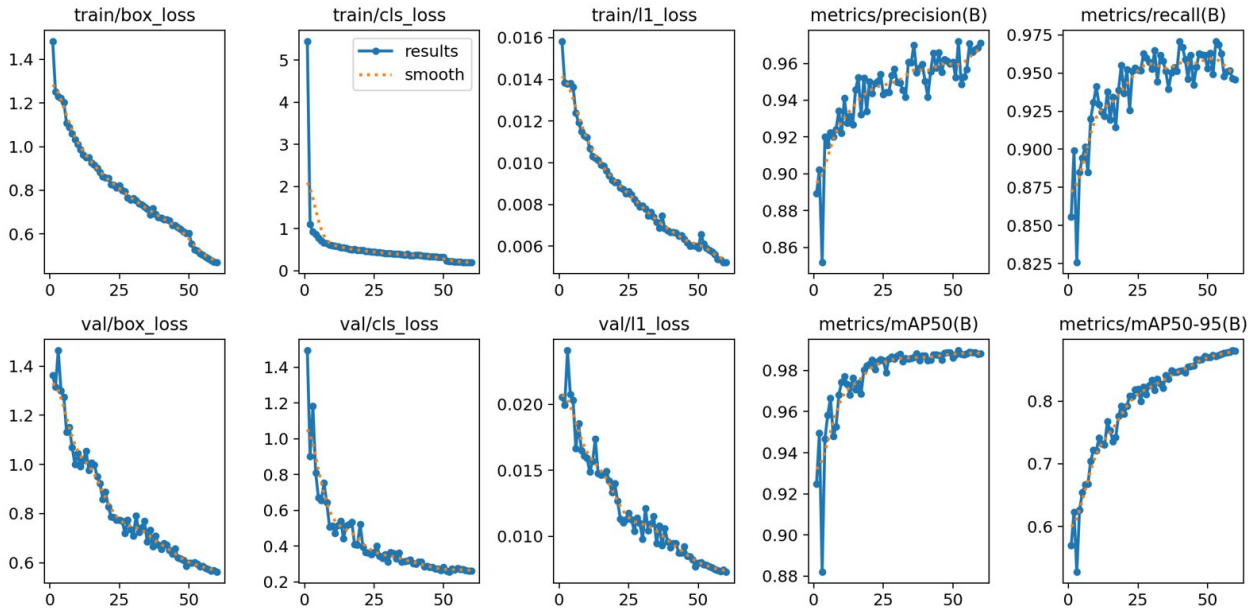



**Figura 12.1:** Curvas de entrenamiento (pérdidas y métricas por época).




**Figura 12.2:** Predicciones sobre imágenes de validación.

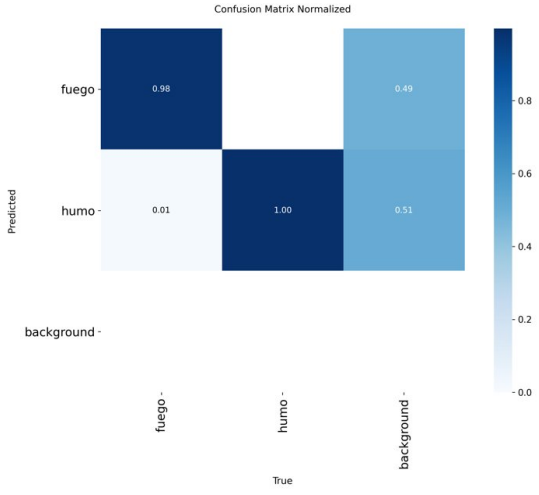



**Figura 12.3:** Matriz de confusión normalizada.

## 12.4. El nodo detector

`detector_yolo.py` corre con el Python de `~/.venvs/yolo`. Lee `/cuadrirotor/camara`, corre el modelo en la GPU a ≈10 Hz y publica:

- `/cuadrirotor/detecciones` (`vision_msgs/Detection2DArray`) con la marca de tiempo de la imagen, para emparejarla con la pose del dron en ese instante;
- `/cuadrirotor/camara_yolo`: la imagen con las cajas dibujadas, que se ve en el panel "Detección YOLO" de Gazebo (`config/gazebo_gui_yolo.config`).

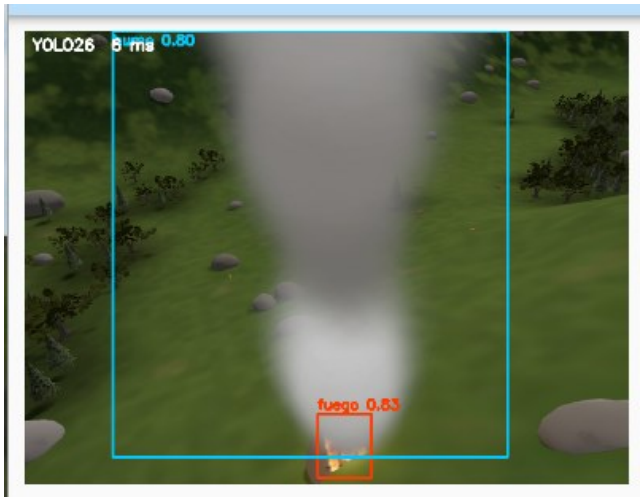



**Figura 12.4:** Panel "Detección YOLO" durante la órbita: caja de *humo* (cian) y de *fuego* (rojo) con su confianza.

**Nota**

Para volar con el dron real conviene volver a entrenar mezclando imágenes reales de incendios (por ejemplo, el dataset D-Fire) con las del simulador.

# Parte IV

# Construir la simulación

# Capítulo 13

# Instalación paso a paso

**En este capítulo**

Qué hay que instalar y en qué orden: ROS 2 y Gazebo, el espacio de trabajo con nuestros paquetes, ArduPilot, el entorno de YOLO y las herramientas de CAD. Cada paso es un script que ya quedó en la carpeta personal.

La instalación se hizo con cuatro scripts que quedaron en la carpeta personal (`~/instalar_ros2_gazebo.sh`, `~/instalar_ardupilot.sh`, `~/instalar_vision_yolo.sh` y `~/ardupilot_sitl.sh`). Esta sección explica qué hace cada uno y en qué orden se corren.

## 13.1. ROS 2 Jazzy y Gazebo Harmonic

```
sudo bash ~/instalar_ros2_gazebo.sh
```

El script:

1. agrega el repositorio *universe* y el paquete `ros2-apt-source` (que configura el repositorio oficial de ROS 2 para Ubuntu Noble);
2. instala `ros-jazzy-desktop`, `ros-dev-tools`, el puente `ros-jazzy-ros-gz` (trae Gazebo Harmonic), `ros-jazzy-actuator-msgs`, `ros-jazzy-xacro`, PlotJuggler, el almacenamiento MCAP de *rosbag2*, `colcon` y numpy, scipy, matplotlib y pandas;
3. inicializa `rosdep`.

Después, en cada terminal (o en `~/.bashrc`):

```
source /opt/ros/jazzy/setup.bash
```

Comprobación rápida: `gzsimshapes.sdf` debe abrir Gazebo con unas figuras, y `ros2topiclist` debe responder.

## 13.2. Espacio de trabajo

```
git clone https://github.com/jairoroke/Monitoreo-de-incendios-en-un-vehiculo-aereo-no-tripulado.git
    ~/ros2_ws
# Solo para controlador:=ardupilot (plugin de ArduPilot para Gazebo):
git clone https://github.com/ArduPilot/ardupilot_gazebo.git ~/ros2_ws/src/ardupilot_gazebo

cd ~/ros2_ws
export GZ_VERSION=harmonic
colcon build --symlink-install
source install/setup.bash
```

`--symlink-install` enlaza los archivos de Python, los *launch* y los YAML en lugar de copiarlos: un cambio en `src/` se ve sin volver a compilar (hay que recompilar solo si se agrega un ejecutable nuevo en `setup.py` o se cambia código C++). El espacio tiene dos paquetes propios:

**`cuadrirotor_sim`** (Python, *ament_python*): modelos, mundos, controladores, planificador, detector, *launch*, scripts y pruebas.

**`cuadrirotor_gz_plugins`** (C++): dos sistemas de Gazebo, `FuerzaViento` (fuerza externa de viento) y `ComandosMotores` (para ArduPilot, sección 20.1).

## 13.3. ArduPilot SITL

ArduCopter 4.7.2 se compila para SITL (*software in the loop*) en `~/ardupilot`:

```
git clone --recurse-submodules https://github.com/ArduPilot/ardupilot.git ~/ardupilot
cd ~/ardupilot && git checkout Copter-4.7.2 && git submodule update --init --recursive
bash ~/instalar_ardupilot.sh          # como usuario normal, NO con sudo
./waf configure --board sitl && ./waf copter
```

`instalar_ardupilot.sh` corre `Tools/environment_install/install-prereqs-ubuntu.sh` con dos variables que evitan conflictos con ROS: `DO_PYTHON_VENV_ENV=0` (no activar el entorno de Python de ArduPilot en cada terminal; chocaría con el `python3` de ROS) y `DO_AP_STM_ENV=0` (no instalar el compilador para placas STM32). Crea el entorno `~/venv-ardupilot` con MAVProxy y pymavlink, e instala las bibliotecas de GStreamer que necesita el plugin `ardupilot_gazebo`. El binario queda en `~/ardupilot/build/sitl/bin/arducopter`.

En `~/.bashrc` quedaron:

```
export GZ_VERSION=harmonic
export GZ_SIM_SYSTEM_PLUGIN_PATH=$HOME/ros2_ws/install/ardupilot_gazebo/lib/ardupilot_gazebo:${
    GZ_SIM_SYSTEM_PLUGIN_PATH}
export GZ_SIM_RESOURCE_PATH=$HOME/ros2_ws/install/ardupilot_gazebo/share/ardupilot_gazebo/models:
```

```
    $HOME/ros2_ws/install/ardupilot_gazebo/share/ardupilot_gazebo/worlds:${GZ_SIM_RESOURCE_PATH}
export PATH=$HOME/ardupilot/Tools/autotest:$PATH
```

Prueba del SITL con el Iris de ejemplo (sin nuestra simulación):

```
bash ~/ardupilot_sitl.sh              # iris_runway.sdf + sim_vehicle.py --console --map
```

Para la ventana que acomoda las ventanas del SITL (sección 20.4) se agregó `python-xlib` al entorno de ArduPilot:

```
~/venv-ardupilot/bin/pip install python-xlib
```

## 13.4. Visión: YOLO26

```
bash ~/instalar_vision_yolo.sh        # descarga ~3 GB
```

Crea `~/.venvs/yolo` con `--system-site-packages` (el detector necesita ver los paquetes de ROS 2) e instala `ultralytics` con PyTorch y CUDA. Fija tres versiones:

- `numpy<2`: los mensajes de ROS 2 Jazzy están compilados con numpy 1.x;
- `opencv-python<4.12`: las versiones nuevas exigen numpy 2;
- `setuptools<80`: lo que acepta `colcon`.

Al final comprueba que PyTorch vea la GPU y que `rclpy` se importe desde el entorno.

## 13.5. Herramientas de CAD y generación de mundos

La conversión del CAD del Tarot y la generación del terreno usan bibliotecas que no vienen con ROS. Van en un entorno aparte:

```
python3 -m venv --without-pip ~/.venvs/cad
curl -sSL https://bootstrap.pypa.io/get-pip.py | ~/.venvs/cad/bin/python
~/.venvs/cad/bin/pip install cascadio trimesh numpy fast_simplification pillow scipy matplotlib
```

**Nota**

Se intentó primero convertir el STEP con *gmsh*, pero tardaba más de 15 minutos; `cascadio` (teselador de OpenCASCADE) con `trimesh` lo hace en segundos.

## 13.6. Comprobación de la instalación

```
cd ~/ros2_ws/src/cuadrirotor_sim && python3 -m pytest -q test/     # 31 pruebas
ros2 launch cuadrirotor_sim simulacion.launch.py                    # cuadrirrotor en hover
```

# Capítulo 14

# Dónde está cada cosa: el espacio de trabajo

**En este capítulo**

Un mapa de las carpetas y archivos del proyecto, para saber dónde buscar cuando quieras cambiar algo.

```
~/ros2_ws/
|-- README.md                       descripción y órdenes (resumen de este manual)
`-- src/
    |-- cuadrirotor_sim/            paquete principal (Python)
    |   |-- cuadrirotor_sim/        nodos y bibliotecas
    |   |-- launch/simulacion.launch.py
    |   |-- config/                 parámetros (YAML), GUI de Gazebo, ArduPilot
    |   |-- models/                 vehículos, árboles, fauna, fuego, terreno
    |   |-- worlds/                 incendio.sdf, bosque.sdf, bosque_grande.sdf
    |   |-- scripts/                generadores, captura de datos, entrenamiento
    |   |-- yolo/incendio_yolo26n.pt   pesos del detector
    |   `-- test/                   pruebas (pytest)
    |-- cuadrirotor_gz_plugins/     sistemas de Gazebo en C++
    `-- ardupilot_gazebo/           plugin de ArduPilot (clonado)
```

## 14.1. Módulos del paquete `cuadrirotor_sim`

| Archivo | Qué hace |
|---|---|
| `dinamica.py` | Rotaciones, mezclador de rotores (empuje y pares → velocidades), referencia de las misiones *hover* y *órbita*. |
| `controlador_pd.py` | Nodo del PD en cascada: lazo de posición y PD sobre SO(3). Base de los demás controladores. |
| `sta.py` | Super-twisting adaptativo con discretización explícita, semi-implícita e implícita. |
| `controlador_sta.py` | Nodo que reemplaza el lazo de posición del PD por el STA. |
| `viento.py` | Perturbación $F(t) = F_0 + A\sin(2\pi ft)$ publicada en `/cuadrirotor/viento`. |
| `terreno.py` | Modelo digital de elevación (DEM): altura, máximo en un tramo y corte de rayos. |
| `bosque_grande.py` | Arma el SDF del mundo grande con el foco donde se pida. |
| `fauna.py` | Mueve a los animales (pastar, caminar, trotar) por gz-transport. |
| `detector_yolo.py` | YOLO26n sobre la cámara; publica cajas y la imagen anotada. |
| `busqueda_incendio.py` | Planificador autónomo: máquina de estados de la búsqueda. |
| `cobertura.py` | Rutas de barrido (franjas y serpentina) y mapa de cobertura. |
| `evasion.py` | Evasión reactiva de obstáculos con el LiDAR (histograma polar). |
| `geo.py` | Conversión del marco del mundo (ENU) a coordenadas GPS (WGS84). |
| `mision_ardupilot.py` | Misión por MAVLink para ArduCopter SITL. |
| `ventanas_sitl.py` | Acomoda las ventanas del SITL en la pantalla. |
| `mavproxy_trayectoria.py` | Módulo de MAVProxy: dibuja la trayectoria, la ruta y el foco en el mapa. |

## 14.2. Scripts

| Script | Uso |
|---|---|
| `cad_a_malla.py` | Convierte el STEP del Tarot en mallas STL por grupos de piezas. |
| `generar_modelo.py` | Escribe el SDF de cada vehículo y su YAML desde una sola tabla de parámetros. |
| `generar_fuego.py` | Texturas de las partículas y modelos `foco_incendio` y `columna_humo`. |
| `generar_bosque.py` | Mundo `bosque` (85 árboles, semilla fija). |
| `generar_bosque_grande.py` | Terreno, textura, árboles, rocas, fauna y DEM del mundo grande. |
| `preparar_fauna.py` | Descarga y adapta los modelos CC0 de venado, zorro y lobo. |
| `capturar_dataset.py` | Genera imágenes etiquetadas para YOLO desde Gazebo. |
| `entrenar_yolo.py` | Ajusta YOLO26n con uno o varios datasets. |
| `corrida.sh`, `metricas.py` | Campañas de simulación y métricas (RMS, máximo, *chattering*). |

## 14.3. Archivos de configuración

**`config/parametros.yaml`** misión por omisión (altura, foco, radio, $\omega$), ganancias del PD y del STA, y viento.

**`config/vehiculo_<vehiculo>.yaml`** masa, inercia, rotores, $k_f$, $k_m$, $\omega_{max}$, lazo de actitud y cámara. Lo genera `generar_modelo.py`: no se edita a mano.

**`config/mundo_bosque_grande.yaml`** parámetros de la búsqueda en el mundo grande.

**`config/puente.yaml`** tópicos que cruza `ros_gz_bridge` entre Gazebo y ROS 2.

**`config/ardupilot/tarot650.parm`** parámetros de ArduCopter para el Tarot.

**`config/gazebo_gui*.config`** distribución de la ventana de Gazebo (con el panel de la cámara o el de detección de YOLO).

**Cuidado**

Los `-pclave:=valor` de la línea de comandos **no** sobrescriben lo que esté en la sección de un nodo dentro del YAML (el YAML del nodo tiene prioridad). Para campañas se usa `scripts/corrida.sh`, que genera un YAML de ajustes.

# Capítulo 15

# Cómo se metió el Tarot 650 en Gazebo

**En este capítulo**

Cómo se pasó el dron de un archivo CAD a un modelo que vuela en Gazebo: las mallas, la tabla de parámetros, los motores, el mezclador, los sensores y el viento.



**Figura 15.1:** El Tarot 650 en vuelo estacionario sobre la base de despegue del mundo `bosque_grande`. Las hélices delanteras son naranjas.

## 15.1. Del CAD a las mallas

La geometría viene del modelo de GrabCAD *Tarot Ironman 650* (https://grabcad.com/library/tarot-ironman-650-2), con motores T-Motor MN3508 y hélices de $15 \times 5$". El archivo `Tarot.stp` es un ensamble de cientos de piezas. Se convirtió con `scripts/cad_a_malla.py` en dos pasos:

```
# 1) Ver las piezas: nombre, cuántas hay y su tamaño
~/.venvs/cad/bin/python scripts/cad_a_malla.py listar "Tarot 650/Tarot.stp"

# 2) Exportar grupos de piezas por nombre (expresiones regulares)
~/.venvs/cad/bin/python scripts/cad_a_malla.py exportar "Tarot 650/Tarot.stp" salida/ \
    --grupo placas='Base$|base|side bracket|lower support|bracket landing' \
    --grupo brazos='CarbonArm' \
    --grupo acentos='orange|clamp|pivot|clip|Motortoarm|ArmCopter|cap' \
    --grupo tren='Lower Tube|leg|foot|landing-gear|Landing' --grupo motores='bell' \
    --grupo marco=resto --excluir '94180|bearing|circlip|coil|axle|stator|15x5' \
    --ejes +z +x +y --origen 0.0142 0 0.0070

# Hélices, cada una por aparte y centrada:
~/.venvs/cad/bin/python scripts/cad_a_malla.py exportar "Tarot 650/Tarot.stp" salida/ \
    --grupo helice='15x5' --separar helice --caras 4000 --ejes +z +x +y --origen 0.0142 0 0.0070
```

- `--ejes+z+x+y` dice qué eje del CAD es el $x$ (frente), el $y$ (izquierda) y el $z$ (arriba) del modelo. El frente quedó paralelo a los patines del tren de aterrizaje.
- `--origen` es el punto del CAD (en metros) que queda en el origen del cuerpo.
- Cada grupo se exporta como un STL con su color en el SDF: placas y brazos de carbono, acentos naranjas, tren, campanas de los motores y el resto del marco.
- `helice_i.stl` se asigna al rotor $i$ por su posición.

**Cuidado**

No se deben simplificar `placas` ni `brazos`: las placas de carbono miden 1.6 mm y la simplificación de la malla las deshace. Sí se pueden reducir `acentos` (50 000 caras), `tren` (25 000) y `motores` (20 000).

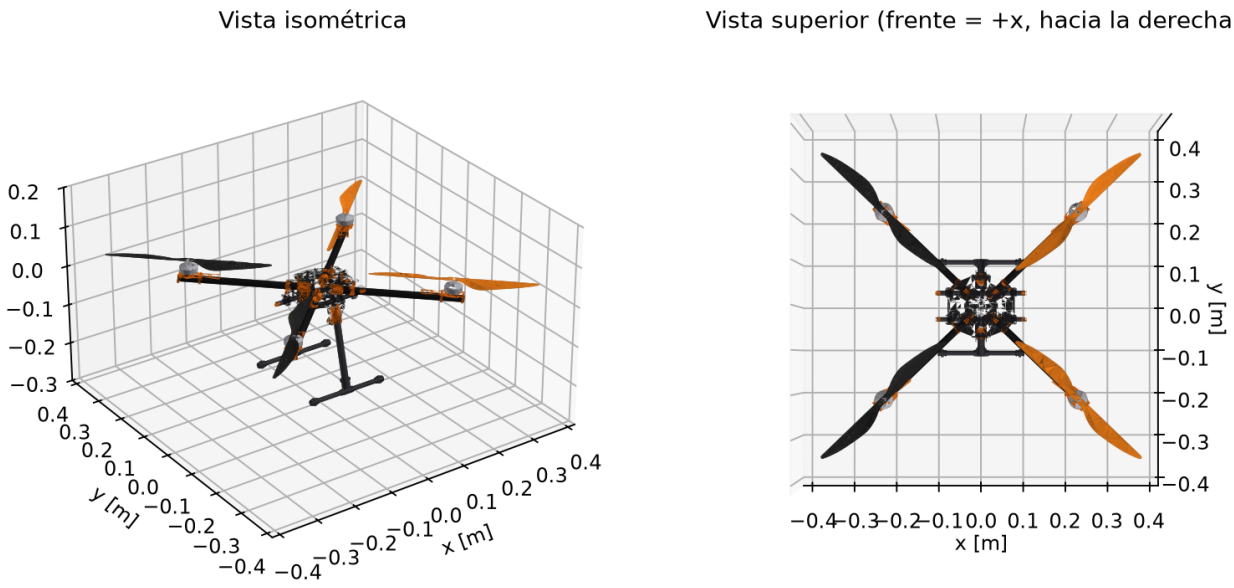



**Figura 15.2:** Mallas del Tarot convertidas del CAD (`models/tarot650/meshes`), en el marco del cuerpo.

## 15.2. Parámetros del vehículo

Todos los parámetros están en una sola tabla, `VEHICULOS` de `scripts/generar_modelo.py`. De ella salen la planta que simula Gazebo (`models/tarot650/model.sdf`) y los valores que usa el controlador (`config/vehiculo_tarot650.yaml`), así que el modelo del controlador coincide siempre con la planta. Para cambiar un parámetro se edita la tabla y se vuelve a correr:

```
python3 scripts/generar_modelo.py            # todos los vehículos
python3 scripts/generar_modelo.py tarot650   # solo uno
```

| Parámetro | Valor | Origen |
|---|---|---|
| Configuración | "X", 4 rotores | CAD |
| Posición de los rotores $(x_i, y_i)$ | $(\pm 0.2198,\ \pm 0.212)$ m | CAD |
| Altura de los rotores | 0.04 m sobre el origen | CAD |
| Tren de aterrizaje | dos patines a lo largo de $x$, 0.2205 m bajo el origen | CAD |
| Radio de la hélice | 0.1905 m (15") | CAD |
| Masa total | 2.2 kg (cuerpo 2.12 + 4 rotores de 0.02) | supuesto |
| Inercia total $J$ | diag(0.03412, 0.03609, 0.06596) kg m$^2$ | estimada |
| Constante de empuje $k_f$ | $4.24 \times 10^{-5}$ N/(rad/s)$^2$ | supuesto |
| Constante de par $k_m$ | 0.016 m | supuesto |
| Velocidad máxima $\omega_{max}$ | 680 rad/s ($\approx$ 19.6 N por motor) | supuesto |
| Constante de tiempo del motor | 30 ms al subir, 50 ms al bajar | supuesto |
| Velocidad en vuelo estacionario | 357 rad/s | calculada |
| Lazo de actitud $K_R$, $K_\omega$ | (3.4, 3.6, 1.65), (0.55, 0.58, 0.59) | ajustado |
| Cámara | 640×480, 15 Hz, HFOV 1.3 rad, 30° hacia abajo | diseño |
| LiDAR 3D | 360°×120 haces, 16 canales ($-35°$ a $+15°$), 30 m, 10 Hz | diseño |

**Tabla 15.1:** Parámetros del Tarot 650 en la simulación.

**Cuidado**

La geometría es la del CAD, pero la masa, las inercias, $k_f$, $k_m$ y los retardos son valores típicos supuestos para esta configuración. **Hay que reemplazarlos por los del dron real** (peso con batería, KV de los motores, hélice, número de celdas) antes de sacar conclusiones cuantitativas.

## 15.3. Motores y mezclador

Cada rotor es un `MulticopterMotorModel` de Gazebo: recibe una velocidad de referencia (`/cuadrirotor/command/motor_speed`, un mensaje `Actuators` con las cuatro velocidades), la filtra con un sistema de primer orden y produce el empuje $f_i = k_f\,\omega_i^2$ y el par de arrastre $k_m f_i$. Fragmento del SDF generado para el rotor 0:

```
<plugin filename="gz-sim-multicopter-motor-model-system"
        name="gz::sim::systems::MulticopterMotorModel">
  <robotNamespace>cuadrirotor</robotNamespace>
  <jointName>rotor_0_joint</jointName>
  <linkName>rotor_0</linkName>
  <turningDirection>ccw</turningDirection>
  <timeConstantUp>0.03</timeConstantUp>
  <timeConstantDown>0.05</timeConstantDown>
  <maxRotVelocity>680.0</maxRotVelocity>
  <motorConstant>4.24e-05</motorConstant>
  <momentConstant>0.016</momentConstant>
  <commandSubTopic>command/motor_speed</commandSubTopic>
  <motorNumber>0</motorNumber>
  <rotorDragCoefficient>8.06428e-05</rotorDragCoefficient>
  <rollingMomentCoefficient>1e-06</rollingMomentCoefficient>
  <motorSpeedPubTopic>motor_speed/0</motorSpeedPubTopic>
  <rotorVelocitySlowdownSim>10</rotorVelocitySlowdownSim>
  <motorType>velocity</motorType>
</plugin>
```

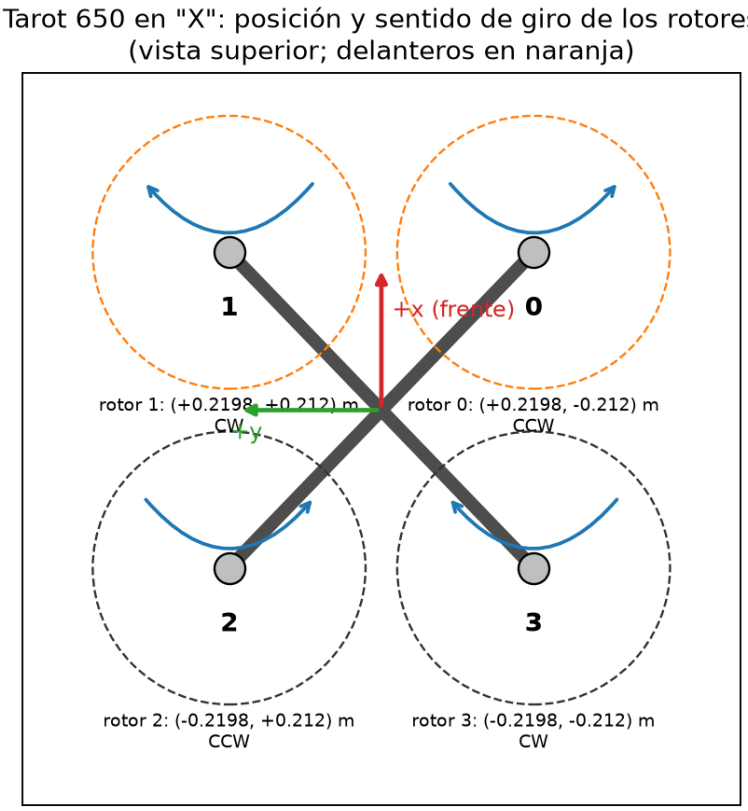



**Figura 15.3:** Numeración, posición y sentido de giro de los rotores del Tarot.

El controlador calcula un empuje total $T$ y un par $\tau = (\tau_x, \tau_y, \tau_z)$. El mezclador (`dinamica.Mezclador`) los convierte en velocidades de rotor. Con el rotor $i$ en $(x_i, y_i)$ y sentido $s_i$ ($+1$ antihorario, $-1$ horario):

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ y_0 & y_1 & y_2 & y_3 \\ -x_0 & -x_1 & -x_2 & -x_3 \\ -s_0 k_m & -s_1 k_m & -s_2 k_m & -s_3 k_m \end{bmatrix}}_{B} \begin{bmatrix} f_0 \\ f_1 \\ f_2 \\ f_3 \end{bmatrix}, \qquad \omega_i = \sqrt{\frac{\mathrm{sat}_{[0,\, f_{max}]}\left((B^{+}[T,\tau]^{\top})_i\right)}{k_f}}. \tag{15.1}$$

En vuelo estacionario cada motor carga un cuarto del peso:

$$f_h = \frac{mg}{4} = \frac{2.2 \cdot 9.81}{4} = 5.40 \text{ N}, \qquad \omega_h = \sqrt{\frac{f_h}{k_f}} = 357 \text{ rad/s}, \qquad \frac{f_h}{f_{max}} = \left(\frac{357}{680}\right)^2 = 0.28. \tag{15.2}$$

La relación empuje/peso es $4 \cdot 19.6/21.6 \approx 3.6$.

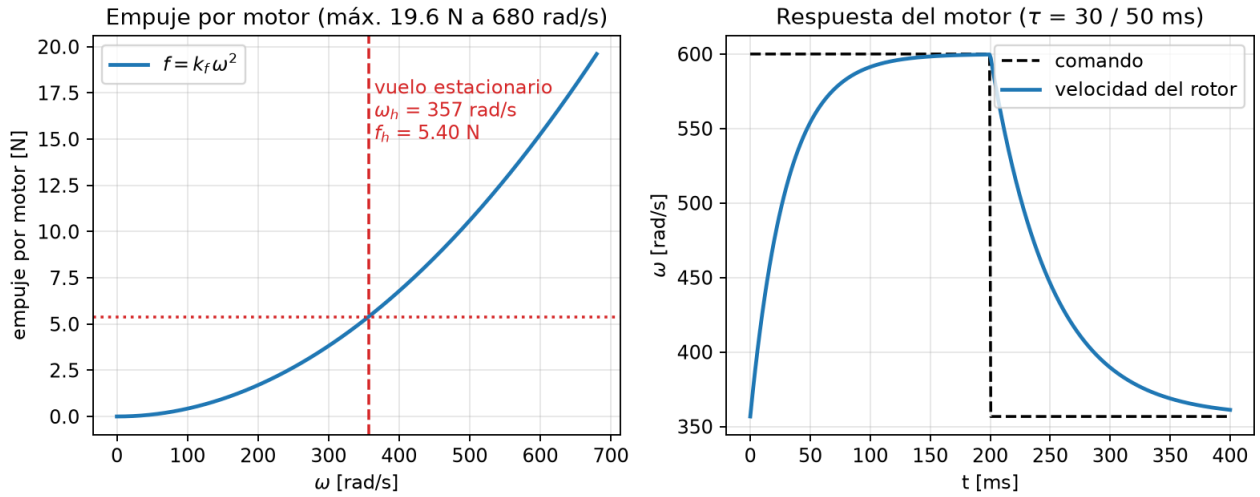



**Figura 15.4:** Curva de empuje de un motor y respuesta de primer orden a un escalón.

## 15.4. Sensores

**IMU** 200 Hz en `/cuadrirotor/imu`.

**Odometría** `OdometryPublisher` a 1 kHz en `/cuadrirotor/odometria` (posición y orientación verdaderas; velocidades en el marco del cuerpo).

**Cámara frontal** inclinada 30° hacia abajo: ve el horizonte para buscar el humo de lejos y mantiene el foco a la vista en la órbita. Rango de recorte hasta 1500 m (antes 200 m) para que se vean las montañas.

**LiDAR 3D** para la evasión de obstáculos (capítulo 18):

```
<sensor name="lidar" type="gpu_lidar">
  <pose>0 0 0.1 0 0 0</pose>
  <always_on>1</always_on>
  <update_rate>10</update_rate>
  <topic>cuadrirotor/lidar</topic>
  <lidar>
    <scan>
      <horizontal><samples>120</samples><min_angle>-3.14159</min_angle><max_angle>3.14159</
    max_angle></horizontal>
      <vertical><samples>16</samples><min_angle>-0.6109</min_angle><max_angle>0.2618</max_angle></
    vertical>
    </scan>
    <range><min>0.8</min><max>30</max><resolution>0.05</resolution></range>
  </lidar>
</sensor>
```

El alcance mínimo de 0.8 m evita que el LiDAR vea las propias hélices (las puntas quedan a ≈0.5 m del centro). La nube llega a ROS como `sensor_msgs/PointCloud2` en `/cuadrirotor/lidar/points`.

## 15.5. Perturbación de viento

El sistema `FuerzaViento` (`cuadrirotor_gz_plugins`) aplica sobre `base_link` una fuerza en el marco inercial y la mantiene en cada paso de física hasta que llega un valor nuevo; así la perturbación es exactamente la señal publicada, sin acumularse. El nodo `viento` publica

$$F(t) = F_0 + A\sin(2\pi f t), \qquad F_0 = 1 \text{ N},\ A = 0.5 \text{ N},\ f = 0.2 \text{ Hz}, \tag{15.3}$$

a partir de $t = 15$ s (con `viento:=true`).

# Capítulo 16

# Cómo se construyó el bosque

**En este capítulo**

Cómo se generaron por programa el terreno con montañas, su textura, los 1700 árboles, las rocas, los animales y el incendio con humo, y cómo los arma el *launch* al arrancar.

## 16.1. Mundos disponibles

**`incendio`** terreno plano, tres árboles simples y el foco en $(8, 5)$. Sirve para probar el control sin distracciones.

**`bosque`** como el del video de ArduPilot `iris_forest`: 50 robles y 35 pinos de OpenRobotics (Gazebo Fuel, licencia CC0, copiados en `models/roble` y `models/pino` para funcionar sin internet) y el foco en $(8, 5)$. Deja libres el despegue, el corredor al foco y el anillo de la órbita, así que la misión `orbita` funciona a 3 m entre los árboles. Se genera con `python3scripts/generar_bosque.pyworlds/bosque.sdf` (semilla fija).

**`bosque_grande`** el valle boscoso con montañas que se describe a continuación.

## 16.2. El mundo `bosque_grande`

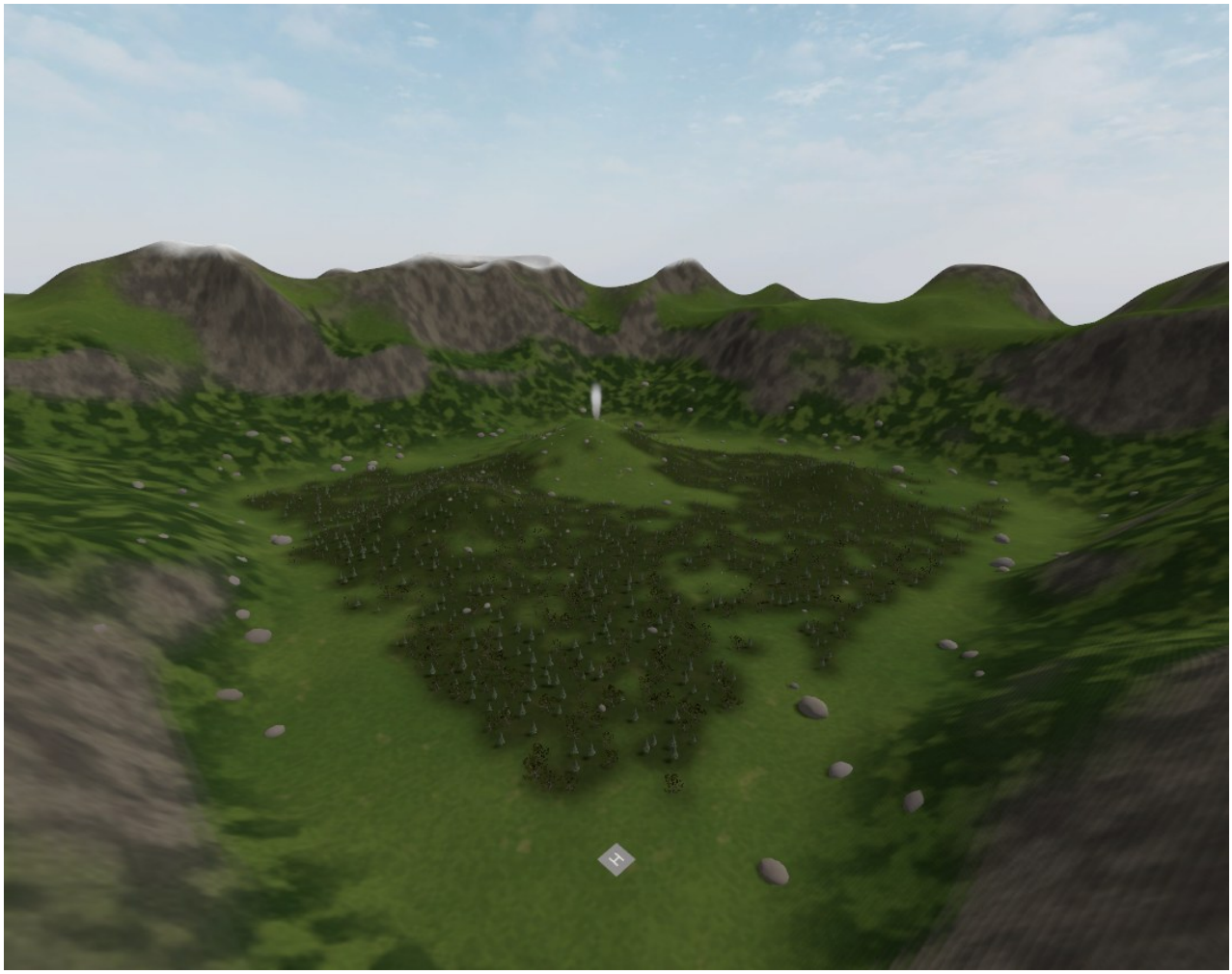


**Figura 16.1:** Vista aérea del valle desde el suroeste. Abajo al centro, la base de despegue; al fondo, el penacho de humo del incendio.

Se pidió "un mundo más grande, con más árboles, montañas, piedras grandes, cielo y animales, en el que el dron monitoree todo el bosque para encontrar el incendio". El resultado:

- Terreno de $1.2 \times 1.2$ km con malla de 4 m ($301 \times 301$ vértices). El valle mide $420 \times 420$ m, con lomas suaves y tres cerros de 18 a 42 m; alrededor, una cordillera de hasta $\approx$215 m con laderas de roca y nieve en las cumbres.
- Textura propia de $4096 \times 4096$ píxeles.
- 1700 árboles (956 pinos y 744 robles) de tamaños y orientaciones variables, con claros de pastizal; miden de 4.6 a 8.2 m.
- 140 rocas y peñascos.
- 21 animales (venados, lobos y zorros) que pastan y caminan.
- Cielo con nubes (`<sky>` de Gazebo).

- Base de despegue de $8 \times 8$ m con una "H" en $(-232, -232)$, fuera del bosque.
- Foco de incendio por omisión en una ladera al noreste, $(105, 135)$, con un claro quemado de 9 m y un penacho de humo de ≈30 m que se ve sobre las copas.

### 16.2.1. Generación del terreno

Todo lo genera una sola vez, con semilla fija (21), el script:

```
~/.venvs/cad/bin/python scripts/generar_bosque_grande.py
```

1. **Relieve** (`alturas`). Suma de: ruido fractal suave de amplitud 7 m (lomas), tres gaussianas deformadas por ruido (los cerros), y la cordillera, que crece con la distancia al borde del valle. El borde es un cuadrado redondeado de 250 m de semilado ($|x|^6 + |y|^6$) deformado por ruido para que no se vea artificial; las crestas salen de $1 - |\text{ruido}|$. Se suaviza con un filtro gaussiano ($\sigma = 1.2$ celdas) y se aplana la base de despegue.
2. **Textura** (`colores_textura`). Por cada píxel: dos verdes de pasto mezclados con ruido, manchas secas, sotobosque oscuro bajo las copas (según la densidad de árboles), monte denso en las faldas, roca donde la pendiente pasa de ≈0.8 o la altura de 140 m, y nieve por encima de 185 m.
3. **Árboles** (`distribuir_arboles`). Puntos al azar en la zona, descartando: cerca de las montañas (el bosque se ralea), en los claros (donde el ruido es bajo), a menos de 30 m de la base, con pendiente mayor a 0.75 o por encima de 95 m, y donde se encimarían con otro árbol. Hay más pinos cuanto más alto.
4. **Rocas.** Icoesferas deformadas, exportadas *con normales* (ver el recuadro).
5. **Fauna.** Grupos de animales en los claros.

Deja en `models/terreno_bosque_grande/`: la malla (`terreno.obj`, también como colisión), la textura (`suelo.jpg`), el DEM (`altura.npz`), las rocas, la lista de objetos (`objetos.json`) y el `model.sdf`.

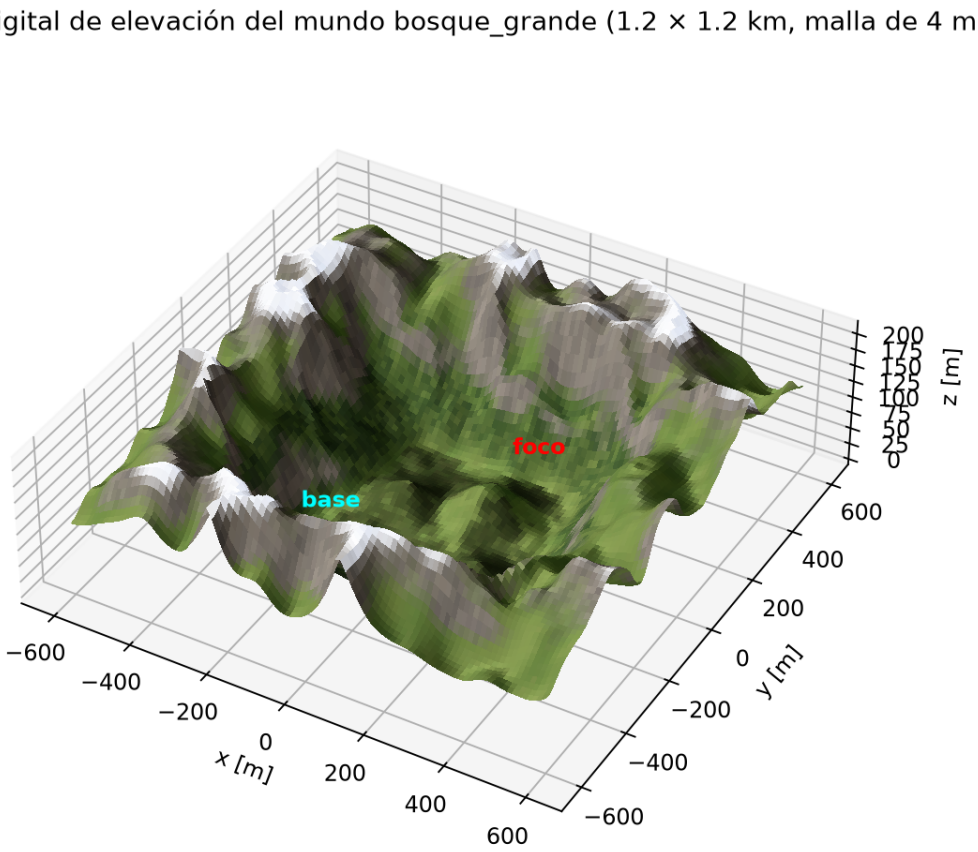



**Figura 16.2:** Modelo digital de elevación con la textura generada.

**Problema encontrado y cómo se resolvió**

La primera versión hacía que Gazebo se cayera con *segfault* en `OdeMesh::fillArrays` en cuanto el dron tocaba algo. La causa: las mallas OBJ de colisión de las rocas no traían normales. Se resolvió exportándolas con normales.

### 16.2.2. El DEM y la clase `Terreno`

`altura.npz` guarda $x_0$, $y_0$, el paso y la matriz $h[i_y, i_x]$. `cuadrirotor_sim/terreno.py` interpola por triángulos (con la diagonal $(i_x, i_y)$–$(i_x{+}1, i_y{+}1)$, igual que la malla de Gazebo) y ofrece:

- `altura(x,y)`: altura del suelo (vectorizada);
- `maximo_en_tramo(a,b)`: punto más alto entre dos puntos (para seguir el relieve);
- `cortar_rayo(o,d,alcance,desfase)`: dónde un rayo de la cámara toca el suelo.

Lo usan el planificador, la fauna (para caminar sobre el suelo, inclinados según la pendiente), el *launch* (para poner el dron y el foco sobre el terreno) y la captura de datos (para saber qué tapa el relieve). Sin DEM, el suelo es el plano $z = 0$.

### 16.2.3. Árboles en un solo modelo

Para que 1700 árboles corran a tiempo real, todos van en un único modelo estático `bosque`: cada árbol es un *visual* escalado (copa y corteza de la malla de OpenRobotics) y su colisión es solo un cilindro en el tronco. Las rocas van igual en un modelo `rocas`.

### 16.2.4. Fauna

Los modelos son CC0 de OpenGameArt: venado (crownjoshua), zorro y lobo (Quaternius). `scripts/preparar_fauna.py` los descarga, los gira a "$z$ arriba y la cabeza hacia $+x$", los escala (venado 1.55 m, zorro 0.55 m, lobo 0.95 m), les pone colores por parte y los exporta a `models/fauna_*` (dinámicos, sin gravedad ni colisión). Se descartó un águila porque traía las alas caídas.

El nodo `fauna` (sin ROS, solo gz-transport) mueve a todos los animales con el servicio `/world/bosque_grande/set_pose_vector` a 10 Hz de tiempo simulado: alternan pausas (pastar, olfatear) con caminatas a un punto al azar de su territorio; los lobos y zorros a veces trotan.

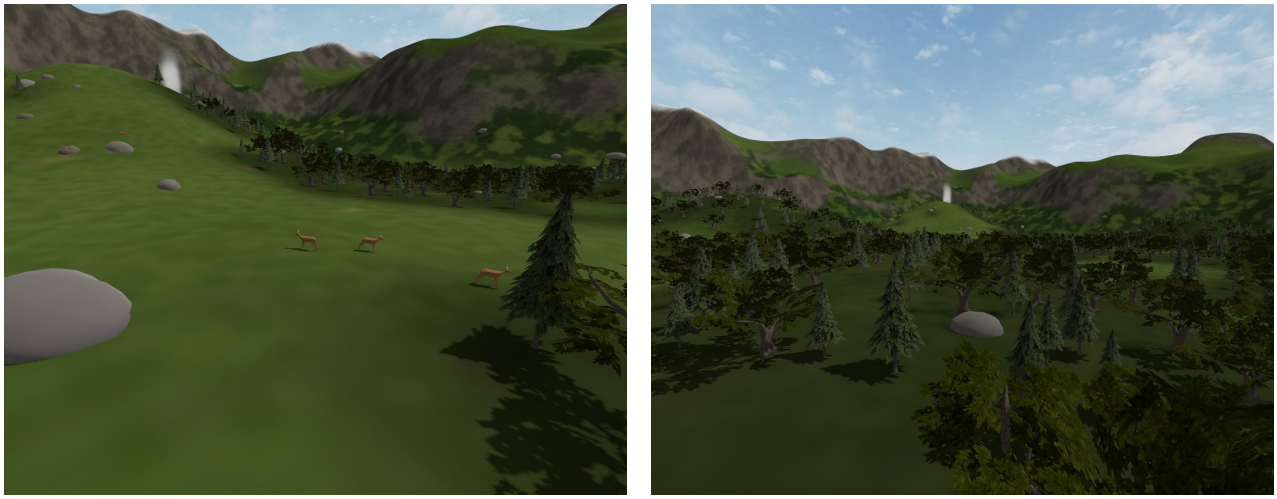


**Figura 16.3:** Venados pastando en un claro (izquierda) y el bosque visto a baja altura (derecha).

## 16.3. El incendio

`python3scripts/generar_fuego.py` genera las texturas de las partículas y dos modelos:

- `foco_incendio`: siete emisores de llama (blanco amarillento → naranja → rojo → transparente), una columna de humo que crece y se aclara al subir, brasas, troncos carbonizados, suelo quemado y una luz naranja;
- `columna_humo`: el penacho alto (≈30 m, vida de 26 s) para el mundo grande.

Usan el sistema `gz-sim-particle-emitter-system`; son solo visuales (no colisionan).

**Cuidado**

Gazebo solo encuentra la imagen de rango de colores de las partículas (`<color_range_image>`) con una ruta absoluta. Por eso el *launch* copia los dos modelos a `/tmp/cuadrirotor_sim_<uid>/modelos` con las rutas resueltas al arrancar.

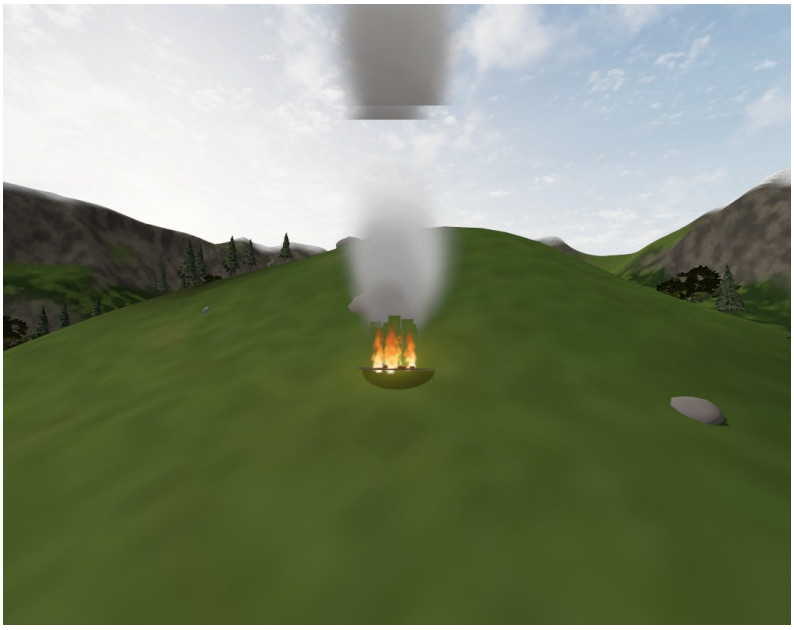


**Figura 16.4:** El foco de incendio en la ladera: llamas, claro quemado y columna de humo.

## 16.4. Cómo arma el mundo el *launch*

Con `mundo:=bosque_grande` el *launch* (`simulacion.launch.py`) escribe el SDF al arrancar en `/tmp/cuadrirotor_sim_<uid>/bosque_grande.sdf`, con el foco en (`foco_x`, `foco_y`) sobre el terreno y el claro quemado alrededor (`cuadrirotor_sim/bosque_grande.py`). También:

- pone el vehículo en la base, a la altura del DEM más 5 cm;
- lanza el nodo `fauna`;
- con GUI, arranca la cámara cerca de la base y a los 12 s llama a `/gui/follow` y `/gui/follow/offset` para que siga al dron;
- pasa al planificador `config/mundo_bosque_grande.yaml` y la ruta del DEM.

El mundo está georreferenciado para que el GPS de Gazebo y el de ArduPilot coincidan con el aviso del incendio (sección 18.8):

```
<spherical_coordinates>
  <surface_model>EARTH_WGS84</surface_model>
  <world_frame_orientation>ENU</world_frame_orientation>
  <latitude_deg>19.293</latitude_deg>
  <longitude_deg>-99.311</longitude_deg>
```

```
  <elevation>3000.0</elevation>
  <heading_deg>0</heading_deg>
</spherical_coordinates>
```

```
# Foco en otro lugar del valle:
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
    controlador:=sta mision:=busqueda foco_x:=-150 foco_y:=60
```

# Parte V

# La autonomía: buscar el incendio sin piloto

# Capítulo 17

# Conceptos de autonomía

**En este capítulo**

Qué significa que un dron sea autónomo, el ciclo percibir–decidir–actuar, las máquinas de estados, cómo se planea recorrer toda una zona, cómo se ubica algo que se ve, qué es el GPS y cómo se pasa de metros a latitud y longitud, qué es un LiDAR y cómo se esquivan obstáculos.

## 17.1. Percibir, decidir, actuar

Un dron autónomo repite sin parar tres pasos:

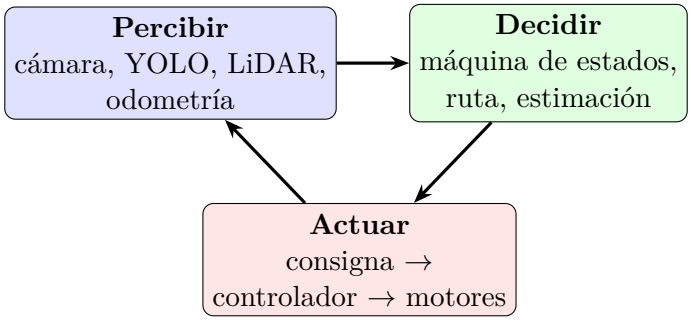



**Figura 17.1:** El ciclo de la autonomía. En nuestra simulación se repite 50 veces por segundo.

En nuestro caso: *percibir* es mirar con la cámara y el LiDAR; *decidir* es elegir a dónde ir (seguir la ruta, acercarse al humo, rodear un árbol); *actuar* es mandar la consigna al controlador, que mueve los motores.

## 17.2. Máquinas de estados

Un comportamiento complejo se organiza en **estados**: en cada momento el dron está en uno ("patrullando", "acercándose al foco"...), hace lo que ese estado indica y cambia a otro cuando se cumple una condición ("vi humo en tres imágenes seguidas").

**Ejemplo de la vida diaria**

Un semáforo es una máquina de estados: verde → (pasan 30 s) → amarillo → (pasan 3 s) → rojo → ... Una lavadora también: llenar, lavar, enjuagar, centrifugar.

La búsqueda del incendio tiene siete estados (figura 18.1): despegue, patrulla, hacia_humo, aproximación, verificación, descenso y órbita. Es fácil de entender, de probar y de explicar, y cuando algo falla el registro dice en qué estado estaba.

## 17.3. Planear la cobertura: recorrer todo el bosque

Si no sabemos dónde está el incendio, hay que mirar todo. Eso es un problema de **planificación de cobertura**: encontrar una ruta que haga pasar la "huella" de la cámara por toda la zona. La solución clásica es ir y venir en franjas paralelas, como al podar un jardín (en inglés, *boustrophedon*: "como gira el buey al arar"). Lo que importa es la **separación** entre franjas: tan grande como se pueda (ruta más corta) pero sin dejar huecos sin ver.

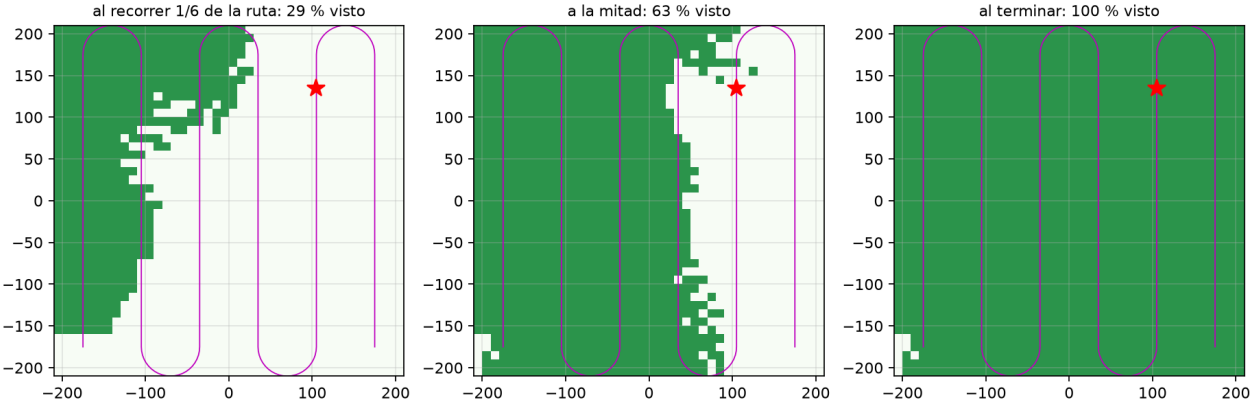



**Figura 17.2:** Simulación de qué tanto del bosque (celdas de 10 m) ha visto la cámara a lo largo de la serpentina, a 30 m sobre el suelo y viendo hasta 150 m. La estrella roja es el incendio.

Durante el vuelo, el planificador lleva este mismo mapa ("cobertura") y lo publica para verlo en RViz.

## 17.4. Ubicar lo que se ve

Del capítulo 10 sabemos que un píxel da una dirección. Para pasar a un punto del mapa hay dos caminos:

**Cortar el rayo con el terreno.** Si conocemos el relieve (un **modelo digital de elevación**, DEM: una tabla con la altura del suelo cada 4 m), avanzamos por el rayo hasta que queda por debajo del suelo. Así se estima la posición de las llamas.

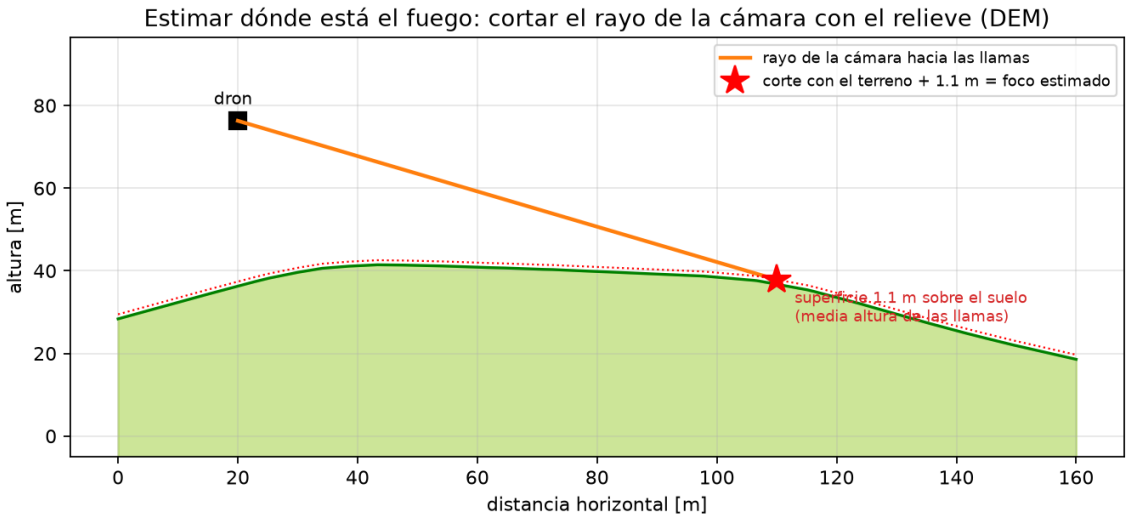



**Figura 17.3:** El rayo de la cámara hacia las llamas se corta con la superficie que está 1.1 m sobre el relieve.

**Triangular.** El humo sube en columna vertical: verlo da la dirección hacia el foco pero no la distancia. Con varias direcciones desde lugares distintos, el foco está donde se cruzan. Como las mediciones tienen ruido, las rectas no se cruzan en un solo punto y se busca el más cercano a todas (mínimos cuadrados).

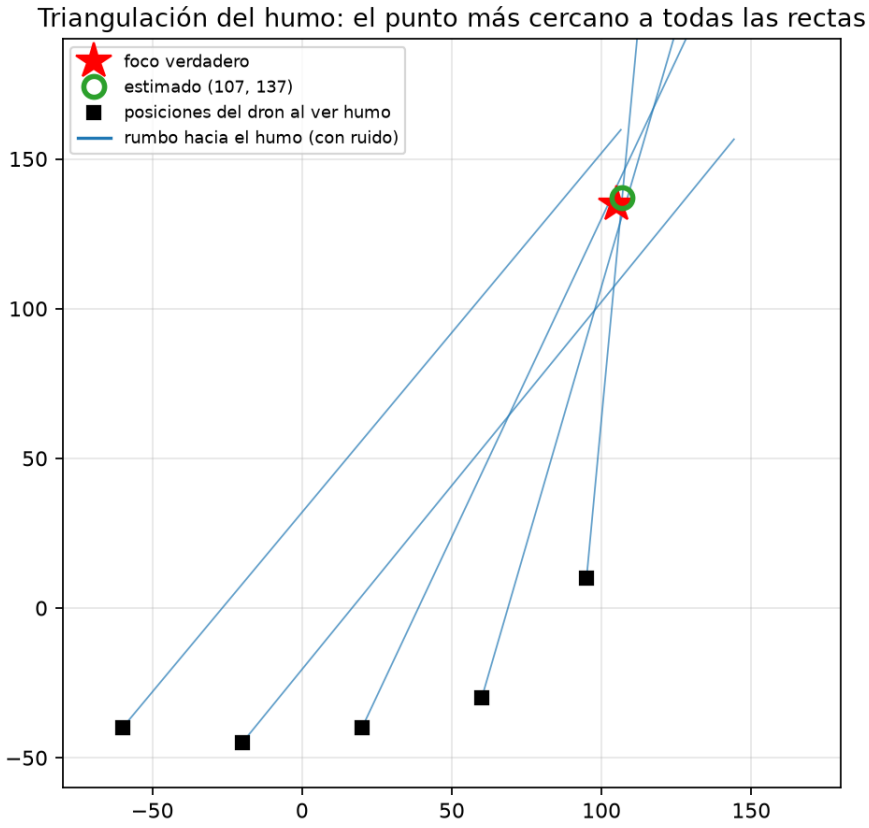



**Figura 17.4:** Triangulación del humo con cinco direcciones con ruido.

**Cuidado**

La triangulación funciona mal si las direcciones son casi paralelas (todas vistas desde puntos alineados en la misma dirección): un error pequeño en el ángulo mueve mucho el cruce. Por eso en la simulación una estimación hecha solo con humo no basta para confirmar un incendio: el dron se acerca hasta ver las llamas.

## 17.5. GPS: de metros a latitud y longitud

La simulación trabaja en metros sobre un plano (ENU), pero los bomberos necesitan **coordenadas GPS**: latitud (cuánto al norte o sur del ecuador), longitud (cuánto al este u oeste de Greenwich) y altitud. La Tierra no es una esfera sino un **elipsoide** un poco achatado; el GPS usa el modelo **WGS84**.

La Tierra (elipsoide WGS84) y el plano local ENU
donde vive la simulación (amarillo)

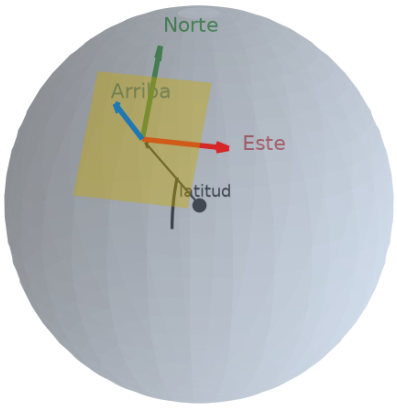



**Figura 17.5:** El plano local ENU de la simulación es tangente a la Tierra en el origen del mundo.

Para convertir se fija dónde está el origen del mundo (como supuesto, en el Parque Nacional Desierto de los Leones, CDMX: 19.293° N, 99.311° O, 3000 m) y se hace la cuenta exacta: ENU → coordenadas cartesianas centradas en la Tierra (ECEF) → latitud, longitud y altitud (sección 18.8).

**Para curiosos (te lo puedes saltar)**

A esta latitud, un grado de latitud mide unos 110.8 km y uno de longitud unos 105.1 km. Por eso un error de $10^{-6}$ grados en las coordenadas equivale a unos 10 cm: con seis decimales basta.

## 17.6. LiDAR: medir distancias con luz

Un **LiDAR** lanza pulsos láser en muchas direcciones y mide cuánto tardan en regresar; con eso calcula la distancia al primer obstáculo en cada dirección. El resultado es una **nube de puntos**. El del dron simulado lanza 120 rayos alrededor por 16 alturas distintas, 10 veces por segundo, hasta 30 m.

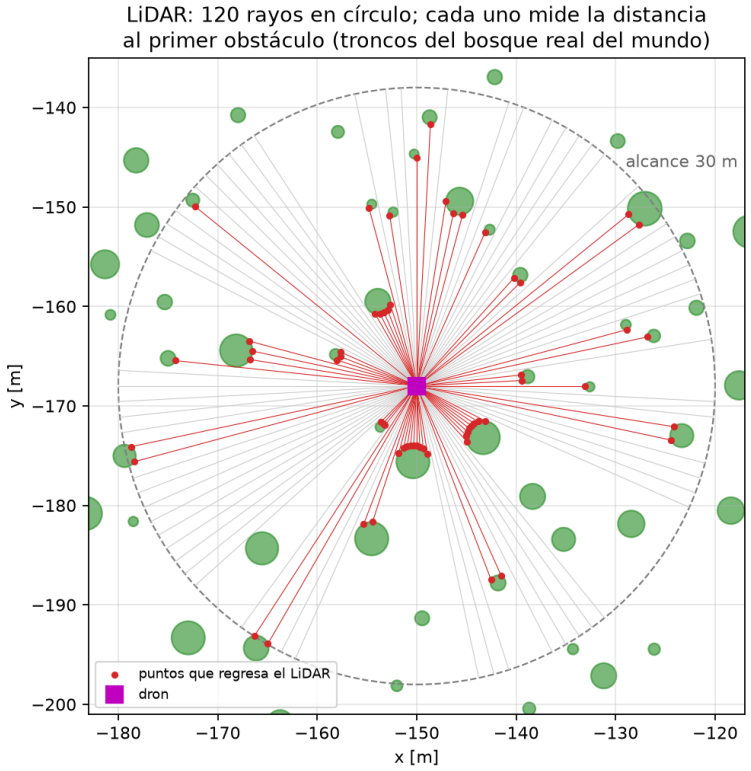



**Figura 17.6:** Una capa del LiDAR en el bosque del mundo grande: los rayos rojos tocan un tronco, los grises no encuentran nada en 30 m.

A diferencia de la cámara, el LiDAR sí da distancias, pero no sabe "qué" es cada punto. Se complementan: la cámara para reconocer el fuego, el LiDAR para no chocar.

## 17.7. Evitar obstáculos

Hay dos grandes familias de métodos:

**Planificación global:** con un mapa completo, se calcula de antemano una ruta sin choques (algoritmos como A* o RRT).

**Evasión reactiva:** con lo que se ve en ese momento, se corrige el rumbo sobre la marcha. Es lo que hacemos: no tenemos un mapa de los árboles, solo lo que ve el LiDAR.

Nuestro método es del tipo **VFH** (*vector field histogram*): se divide el círculo de direcciones alrededor del dron en 72 sectores, se marcan los que están bloqueados por algún obstáculo cercano y se elige el sector libre más parecido a la dirección a la que queremos ir.

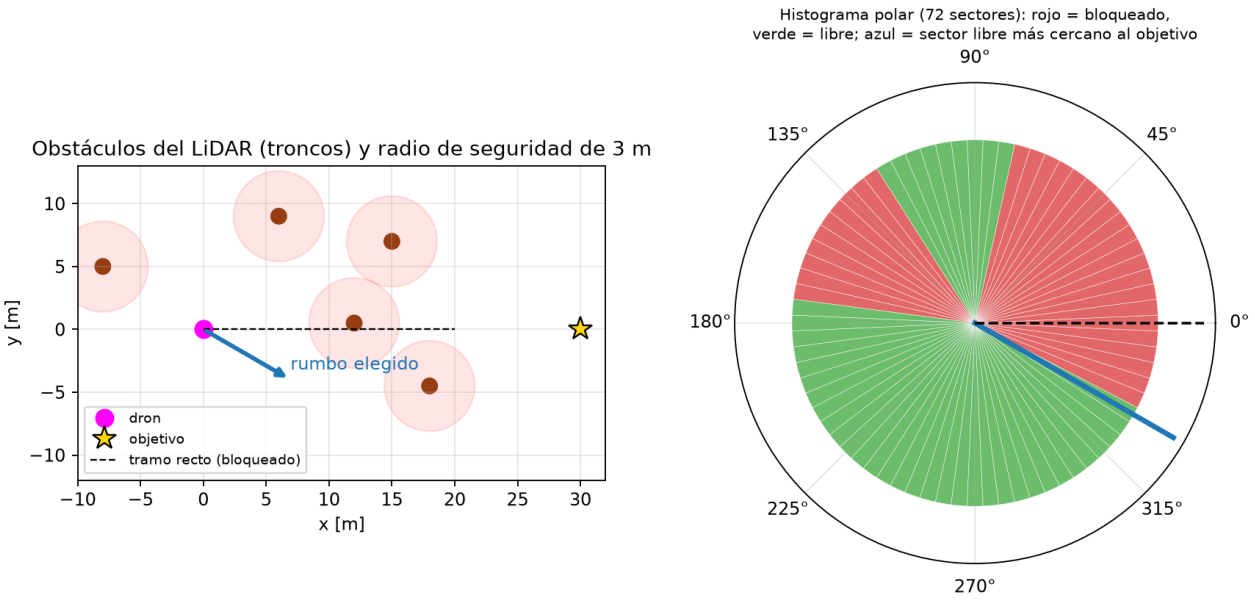



**Figura 17.7:** Evasión con histograma polar.

Para las montañas no hace falta el LiDAR: el relieve se conoce (el DEM), así que el dron sube antes de llegar a una ladera. El capítulo siguiente cuenta cómo se juntaron todas estas piezas.

# Capítulo 18

# La búsqueda del incendio, pieza por pieza

**En este capítulo**

Cómo quedó programado el planificador `busqueda_incendio`: la máquina de estados, la ruta en serpentina, el seguimiento del relieve, la estimación y confirmación del foco, el aviso GPS y la evasión de obstáculos, con sus fórmulas y parámetros.

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
    controlador:=sta mision:=busqueda
```

Con `mision:=busqueda` el *launch* agrega el detector y el planificador `busqueda_incendio`, y pone al controlador en modo `externa`: deja de generar su propia referencia y sigue la consigna que publica el planificador en `/cuadrirotor/consigna` (`trajectory_msgs/MultiDOFJointTrajectoryPoint`: posición, velocidad, aceleración y guiñada). El planificador corre a 50 Hz y limita la velocidad (5 m/s horizontal, 3 m/s vertical) y la aceleración (1.5 m/s$^2$) de la consigna.

## 18.1. Máquina de estados

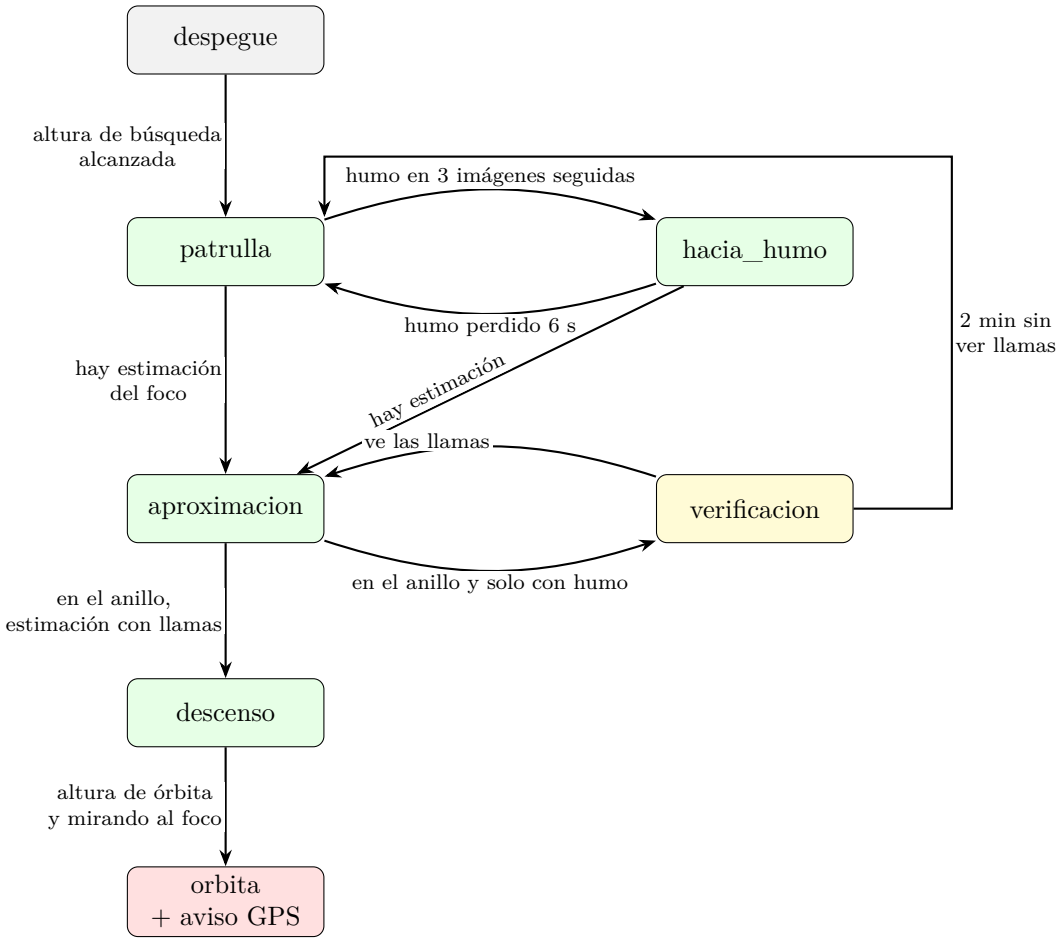



**Figura 18.1:** Estados de `busqueda_incendio`. En *patrulla*, al terminar la ruta sin ver el incendio, el dron la recorre de regreso y sigue vigilando.

## 18.2. Ruta de barrido en serpentina

La zona del bosque es el rectángulo $[-210, 210] \times [-210, 210]$ m. La ruta (`cobertura.py`, `ruta_serpentina`) avanza **hacia el frente del dron**: en el eje de la zona más parecido a su rumbo al despegar (en la base mira a $+x$). Los barridos son perpendiculares al avance, de lado a lado del bosque, separados $p \approx 70$ m y a $p/2$ de los bordes; cada uno se une con el siguiente por un semicírculo de radio $r = p/2 = 35$ m. Con $f$ la coordenada de avance, $l$ la lateral, $\sigma = \pm 1$ el lado de la vuelta y $\phi \in [0, \pi]$:

$$(f, l) = \big(f_k + r(1 - \cos\phi),\ l_{fin} + \sigma r \sin\phi\big). \tag{18.1}$$

La ruta se muestrea cada 5 m (2.6 km, ≈9 min a 5 m/s) y el dron persigue un punto 10 m adelante sobre ella (`anticipo_ruta`), así que no frena en las vueltas: a 5 m/s la vuelta pide $v^2/r = 0.7$ m/s$^2$. La versión anterior (`patron:franjas`) recorría franjas este-oeste con vueltas en ángulo recto, deteniéndose en cada vértice.

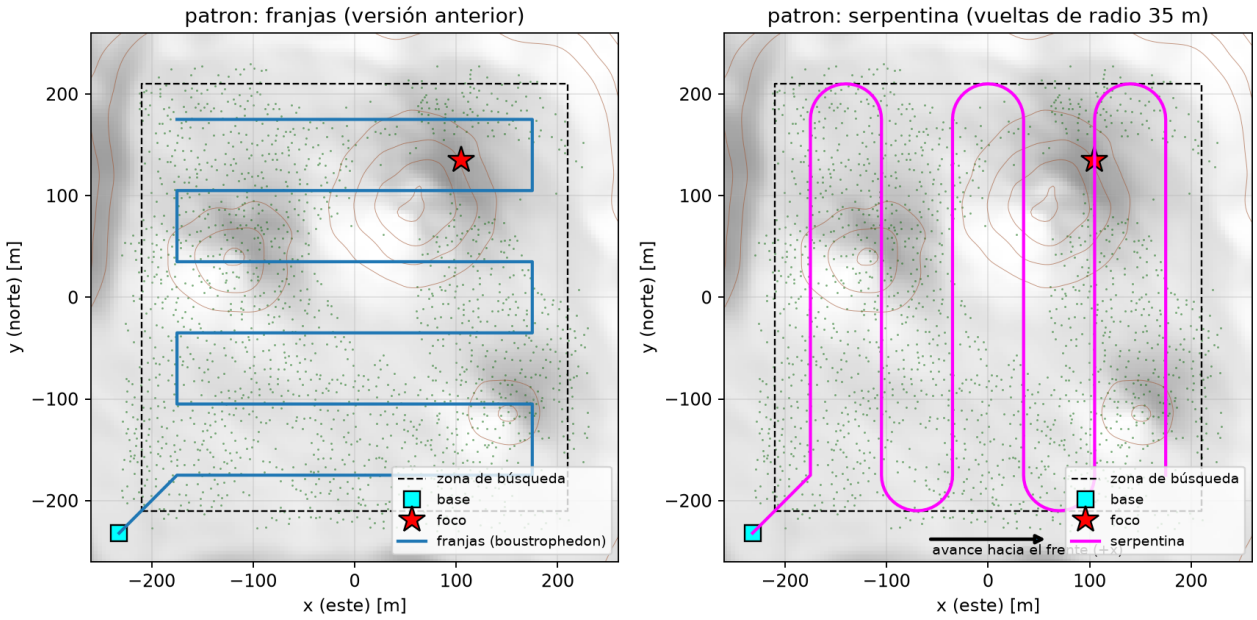



**Figura 18.2:** Rutas sobre el relieve (curvas de nivel cada 10 m) y los árboles (puntos verdes).

## 18.3. Seguimiento del relieve

Las alturas son *sobre el suelo*. La altura de la consigna es

$$z = \max_{q \in [c,\ c+L\hat{u}]} h(q) + h_{busq}, \qquad L = 50 \text{ m},\ h_{busq} = 30 \text{ m}, \tag{18.2}$$

el punto más alto del terreno entre la consigna $c$ y 50 m adelante (`anticipacion`), más un margen de 5 m a los lados. Así el dron empieza a subir antes de llegar a un cerro. El planificador conoce el relieve (como un dron real con un mapa de elevación), pero no dónde está el incendio.

## 18.4. Mapa de cobertura

Cada 0.5 s se proyecta una rejilla de $24 \times 18$ rayos de la cámara sobre el relieve (hasta 150 m) y se marcan las celdas de $10 \times 10$ m que ya se vieron. Se publica el porcentaje en `/cuadrirotor/busqueda/cobertura` y el mapa en `/cuadrirotor/busqueda/mapa_cobertura` (`nav_msgs/OccupancyGrid`, 0 = vista), junto con la ruta en `/cuadrirotor/busqueda/ruta`. Se ven en RViz con el marco `world`.

## 18.5. Estimación de la posición del foco

Cada detección se empareja con la pose del dron en el instante de la imagen. El rayo de la cámara por el centro de la caja $(u, v)$ es

$$d = R\,R_c \begin{bmatrix} 1 \\ -(u - W/2)/f \\ -(v - H/2)/f \end{bmatrix}, \qquad f = \frac{W/2}{\tan(\mathrm{HFOV}/2)}, \tag{18.3}$$

con $R_c$ la inclinación de 30° de la cámara.

**Con llamas.** El rayo se corta con la superficie 1.1 m sobre el suelo (media altura de las llamas) con `Terreno.cortar_rayo`, solo si el corte queda a menos de 100 m (`alcance_llamas`). La estimación es la mediana de los últimos 15 cortes (con al menos 5).

**Con humo.** La columna es vertical sobre el foco, así que cada detección da solo un rumbo $\hat{n}_i$ desde la posición $c_i$ de la cámara. Con rumbos desde puntos distintos (dispersión de más de 0.25 rad) se busca el punto más cercano a todas las rectas por mínimos cuadrados:

$$\hat{x} = \Big(\sum_i P_i\Big)^{-1} \sum_i P_i\, c_i, \qquad P_i = I - \hat{n}_i \hat{n}_i^{\top}. \tag{18.4}$$

**Problema encontrado y cómo se resolvió**

Antes del límite `alcance_llamas`, con el foco en $(-150, 60)$ la primera estimación caía en $(-137, 110)$: de lejos el rayo roza el suelo y un error pequeño en la caja da decenas de metros.

## 18.6. Verificación: solo con llamas se confirma

Si al llegar al anillo de órbita la estimación sigue saliendo solo del humo, el dron entra en *verificacion*: rodea el punto estimado a 35 m y a la altura de búsqueda, mirándolo, hasta ver las llamas; con ellas vuelve a *aproximacion* con la estimación corregida. Si no las ve en 2 min, descarta la estimación y sigue la patrulla. Solo con llamas baja a orbitar y avisa.

## 18.7. Órbita

Radio 14 m, $\omega = 0.12$ rad/s (con rampa de 4 s), cámara hacia el foco y altura

$$z_{orb} = \max\big(h(\text{foco}) + 18\ \text{m},\ \max_{\text{anillo}} h + 14\ \text{m}\big), \tag{18.5}$$

por encima de las copas (los árboles miden hasta 8.2 m). Si el LiDAR ve algo más alto bajo el anillo, sube (a lo más 1 m/s).



**Figura 18.3:** El dron llegando al foco: la cámara de Gazebo sigue al dron desde atrás.

## 18.8. Aviso con las coordenadas GPS

Al empezar la órbita (foco confirmado y visto de cerca) el planificador publica una sola vez:

- `/cuadrirotor/incendio/gps`: `sensor_msgs/NavSatFix` con latitud, longitud, altitud y la dispersión de los cortes de las llamas como incertidumbre;
- `/cuadrirotor/incendio/alerta`: el aviso de texto.

Los dos con QoS *transient local*: quien se suscriba después también lo recibe. Además lo escribe en la terminal y agrega una línea a `~/incendios_detectados.csv`:

```
INCENDIO CONFIRMADO 2026-10-23 04:26:47
  GPS: 19.294219, -99.310000  (19°17'39.2"N 99°18'36.0"W)
  Altitud del suelo: 3019 m   incertidumbre: ±1.0 m
  Mapa: https://www.google.com/maps?q=19.294219,-99.310000
  Local: (105.1, 135.0) m
```

Para recibirlo desde otra terminal: `ros2topicecho/cuadrirotor/incendio/alerta`.

**Conversión ENU → WGS84** (`geo.py`). El origen del mundo se puso, como supuesto, en el Parque

Nacional Desierto de los Leones (CDMX), un bosque de oyamel con incendios casi cada temporada seca: $\varphi_0 = 19.2930°$, $\lambda_0 = -99.3110°$, $h_0 = 3000$ m (el relieve del mundo no es el real; parámetro `origen_gps` para cambiarlo). La conversión es exacta sobre el elipsoide WGS84 ($a = 6378137$ m, $e^2 = 6.69437999014 \times 10^{-3}$). El origen pasa a ECEF con $N(\varphi) = a/\sqrt{1 - e^2 \sin^2 \varphi}$:

$$X_0 = (N + h_0)\cos\varphi_0 \cos\lambda_0, \quad Y_0 = (N + h_0)\cos\varphi_0 \sin\lambda_0, \quad Z_0 = (N(1 - e^2) + h_0)\sin\varphi_0, \quad (18.6)$$

el punto $(x, y, z)$ en ENU se rota al ECEF,

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X_0 \\ Y_0 \\ Z_0 \end{bmatrix} + \begin{bmatrix} -\sin\lambda_0 & -\sin\varphi_0\cos\lambda_0 & \cos\varphi_0\cos\lambda_0 \\ \cos\lambda_0 & -\sin\varphi_0\sin\lambda_0 & \cos\varphi_0\sin\lambda_0 \\ 0 & \cos\varphi_0 & \sin\varphi_0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad (18.7)$$

y se regresa a geodésicas iterando $\varphi \leftarrow \operatorname{atan2}\big(Z,\ p(1 - e^2 N/(N + h))\big)$, $h = p/\cos\varphi - N$, con $p = \sqrt{X^2 + Y^2}$ y $\lambda = \operatorname{atan2}(Y, X)$.

## 18.9. Evasión de obstáculos

Se pidió que el dron no choque con las montañas ni con los árboles. Toda consigna, salvo la de la órbita, pasa por dos filtros (`_seguro` del planificador y `cuadrirotor_sim/evasion.py`):

### 18.9.1. Relieve: primero subir

Con `altura_seguridad` $> 0$ (10 m en el mundo grande), si la consigna está a menos de esa altura sobre el punto más alto del terreno en los próximos 25 m, el objetivo horizontal se congela en la posición actual y solo se sube. El dron nunca avanza "hacia" una ladera más rápido de lo que puede subir.

### 18.9.2. Árboles, rocas y animales: LiDAR y histograma polar

1. **Memoria de obstáculos.** Cada nube del LiDAR pasa al marco del mundo con la pose del dron en ese instante, $p_w = p + R\,(q + r_{lidar})$. Se quitan los puntos a menos de 1.5 m del DEM (son el suelo) y el resto se guarda en vóxeles de 1 m, olvidando lo que queda a más de 60 m del dron. Se publican en `/cuadrirotor/evasion/obstaculos`.
2. **¿Está libre el camino?** Se toman los puntos a la altura del dron ($\pm 3$ m) y a menos de 20 m. Si ninguno queda a menos de $r = 3$ m del tramo recto hacia el objetivo, la consigna no se toca.
3. **Histograma polar** (como VFH). Si el tramo está bloqueado, se dividen las direcciones en 72 sectores de 5°. Un punto a distancia $d$ y ángulo $\theta$ bloquea los sectores dentro de $\theta \pm \big(\arcsin(r/d) + 2.5°\big)$, es decir, todo rumbo que pasaría a menos de $r$ de él.
4. **Elección.** Se toma el sector libre que minimiza $|\theta - \theta_{obj}| + 0.3\,|\theta - \theta_{prev}|$ (el segundo término evita oscilar) y el objetivo pasa a ser un punto a 8 m en esa dirección. Si todos están bloqueados, el objetivo es subir 4 m.
5. **Histéresis.** Mientras está desviado, solo vuelve al tramo recto cuando queda libre con $1.4\,r$.
6. **Al bajar** no entra a menos de $r$ de lo más alto que tenga debajo.

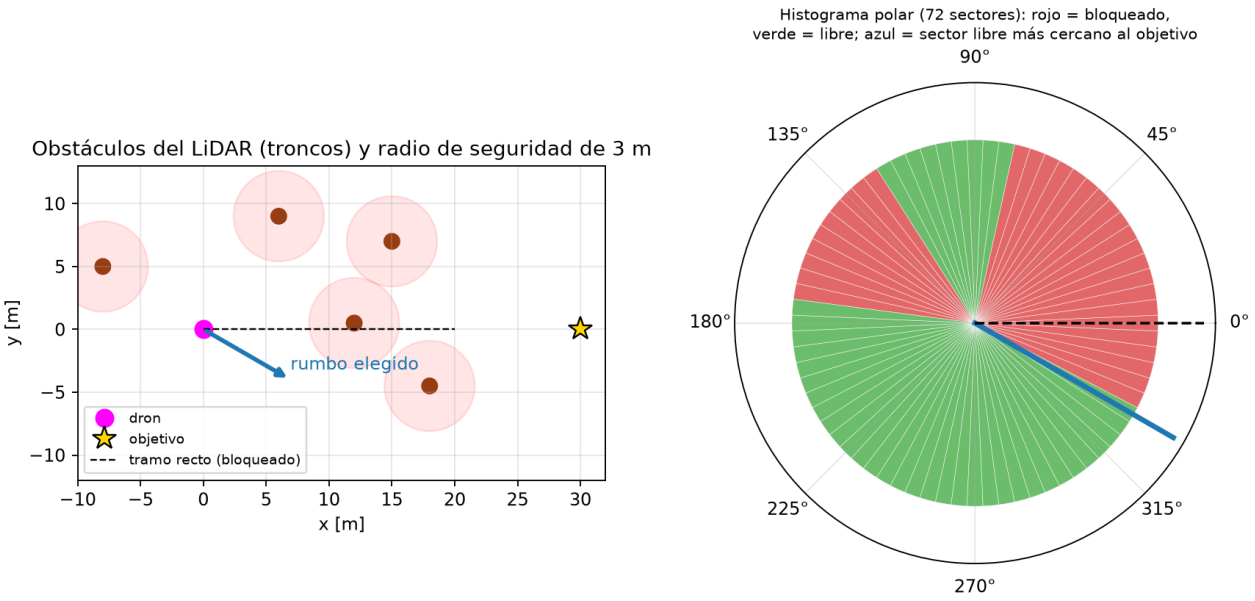



**Figura 18.4:** Ejemplo del algoritmo: el tramo recto al objetivo choca con un tronco; el histograma polar marca los sectores bloqueados y el dron toma el libre más cercano al rumbo del objetivo.

El núcleo de `Evasion.corregir`:

```
r = np.linalg.norm(Q, axis=1)                    # puntos (xy relativos) a la altura del dron
ang = np.arctan2(Q[:, 1], Q[:, 0])
ancho = np.arcsin(np.minimum(1.0, self.radio / np.maximum(r, 1e-3))) + self.medio_sector
bloq = (np.abs(envolver(self.th[:, None] - ang[None, :])) < ancho[None, :]).any(axis=1)
if bloq.all():                                   # sin salida: sube
    return np.array([c[0], c[1], c[2] + self.subida])
costo = np.abs(envolver(self.th - rumbo_g))
if self.rumbo_previo is not None:
    costo = costo + 0.3 * np.abs(envolver(self.th - self.rumbo_previo))
costo[bloq] = np.inf
rumbo = float(self.th[np.argmin(costo)])
```

**Prueba.** Para forzar la evasión se puso `altura_busqueda:5` (5 m sobre el suelo, entre árboles de hasta 8.2 m). En 540 m de patrulla el dron se desvió 12 veces y **ninguna muestra quedó dentro de una copa**: la holgura mínima a la copa más cercana fue de 0.5 m, a unos 2.2 m/s. A los 30 m de la búsqueda normal los árboles no estorban y la evasión casi nunca actúa.

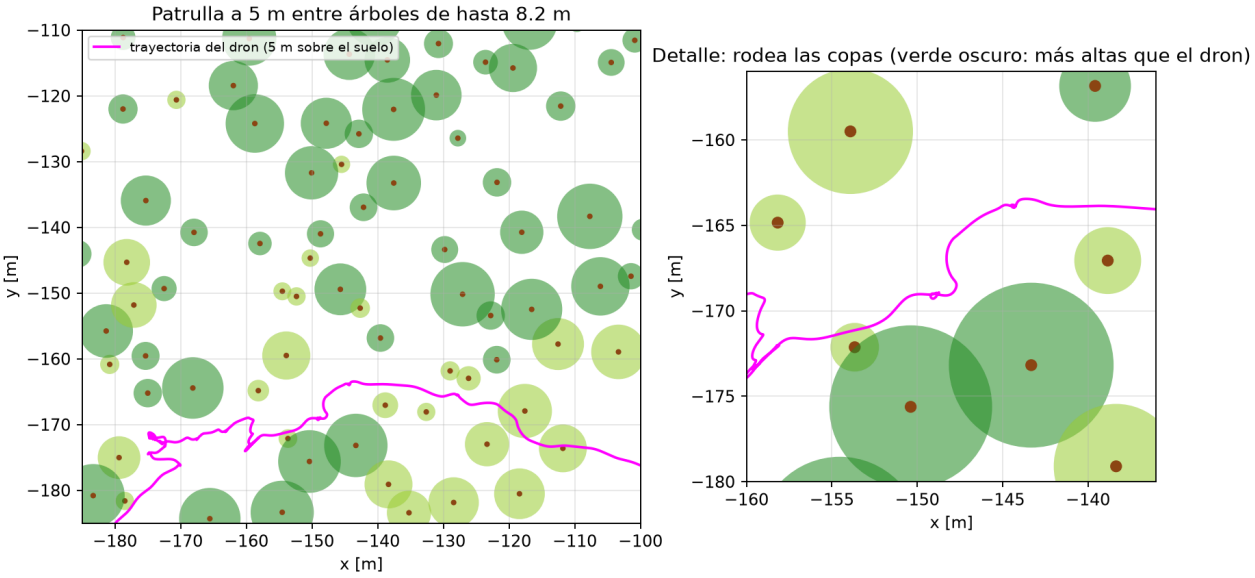



**Figura 18.5:** Trayectoria real del dron (odometría de Gazebo) en la prueba a 5 m sobre el suelo. Los círculos son las copas: verde oscuro, más altas que el dron.

## 18.10. Parámetros del planificador

| Parámetro | Valor | Significado |
|---|---|---|
| `zona` | $[-210, 210]^2$ | rectángulo del bosque que se recorre [m] |
| `patron` | serpentina | `serpentina` o `franjas` |
| `separacion` | 70 | entre barridos [m] |
| `anticipo_ruta` | 10 | distancia del punto que persigue sobre la ruta [m] |
| `altura_busqueda` | 30 | sobre el suelo [m] |
| `anticipacion` | 50 | mira el relieve esta distancia adelante [m] |
| `velocidad`, `velocidad_vertical` | 5, 3 | [m/s] |
| `aceleracion` | 1.5 | [m/s$^2$] |
| `alcance_vision` | 150 | para la cobertura [m] |
| `alcance_llamas` | 100 | solo usa cortes de llamas más cercanos [m] |
| `altura_orbita`, `altura_minima_copas` | 18, 14 | órbita sobre el foco y sobre el anillo [m] |
| `radio`, `omega` | 14, 0.12 | órbita [m], [rad/s] |
| `radio_verificacion`, `tiempo_verificacion` | 35, 120 | [m], [s] |
| `altura_seguridad` | 10 | sobre el relieve que tiene adelante [m] |
| `radio_evasion`, `alcance_evasion` | 3, 20 | [m] |
| `evasion` | true | activa la evasión con el LiDAR |
| `origen_gps` | (19.293, −99.311, 3000) | latitud, longitud y altitud del origen |

| Parámetro | Valor | Significado |
|---|---|---|
| `registro_incendios` | `~/incendios_detectados.csv` | CSV de avisos |

# Capítulo 19

# ¿Qué es un piloto automático? ArduPilot y MAVLink

**En este capítulo**

Qué hace el software de un controlador de vuelo, qué son los modos de vuelo, por qué el dron tiene que "armarse", qué es el EKF, cómo se comunica con tierra (MAVLink) y para qué sirve una estación de tierra.

## 19.1. El software del controlador de vuelo

Los drones reales no corren un controlador escrito desde cero: usan un **piloto automático** de código abierto ya probado en millones de horas de vuelo. Los dos más usados son **ArduPilot** (su versión para multirrotores se llama ArduCopter) y **PX4**. Hacen todo lo de los capítulos anteriores y más:

- estiman dónde está el dron y cómo está orientado combinando sensores;
- controlan posición, velocidad y actitud (con PID en cascada);
- ofrecen modos de vuelo, misiones con puntos de paso y regreso a casa;
- vigilan la seguridad: batería, señal de radio, GPS, geocercas.

Aquí se usa ArduCopter 4.7.2 como **referencia**: comparar el STA contra un piloto automático profesional es más convincente que compararlo solo contra un PD hecho en casa.

## 19.2. Modos de vuelo

ArduCopter tiene más de veinte modos. Los que aparecen en este proyecto:

**STABILIZE:** el piloto controla la inclinación con la palanca; el dron solo se mantiene nivelado.

**LOITER:** el dron se queda en su lugar usando el GPS.

**GUIDED:** el dron obedece órdenes de una computadora ("ve a este punto", "mantén esta velocidad"). Es el que usa nuestra misión.

**RTL** (*return to launch*): regresa a donde despegó y aterriza.

**LAND:** aterriza donde está.

## 19.3. Armar el dron

Por seguridad, los motores no giran hasta que el dron se **arma**. Antes de permitirlo, ArduPilot revisa (*pre-arm checks*) que los sensores estén calibrados, que el GPS tenga buena señal, que la batería esté bien y que el estimador esté listo. Por eso la misión con ArduPilot espera unos 30 segundos antes de despegar.

## 19.4. El EKF: adivinar bien dónde estoy

Ningún sensor es perfecto: el GPS da la posición con metros de error y pocas veces por segundo; la IMU es rápida pero sus errores se acumulan; el barómetro deriva con el clima. El **filtro de Kalman extendido** (EKF) los combina: predice con la IMU y corrige con el GPS, el barómetro y la brújula, pesando cada uno según su confianza. ArduPilot usa la versión EKF3.

**Ejemplo de la vida diaria**

Caminar con los ojos cerrados contando pasos (la IMU) te lleva más o menos a donde quieres, pero el error crece. Si de vez en cuando abres los ojos un instante (el GPS), corriges. El EKF hace eso cientos de veces por segundo.

## 19.5. MAVLink: el idioma de los drones

**MAVLink** es el protocolo con el que el piloto automático habla con el exterior: mensajes pequeños y bien definidos como `HEARTBEAT` (“sigo vivo”), `GLOBAL_POSITION_INT` (posición GPS), `COMMAND_LONG` (“despega”) o `SET_POSITION_TARGET_LOCAL_NED` (“ve a este punto”).

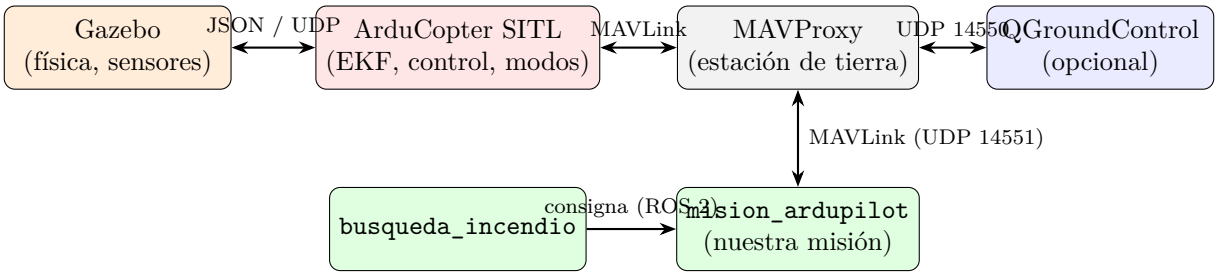



**Figura 19.1:** Cómo se conectan las piezas al volar con ArduPilot SITL.

## 19.6. Estación de tierra

Una **estación de tierra** (GCS) es el programa con el que una persona ve y manda al dron: mapa, telemetría, botones para cambiar de modo. Usamos **MAVProxy**, que trae consola, mapa y un

prompt donde se escriben comandos (`modeGUIDED`, `armthrottle`, `modeRTL`...). También se puede conectar QGroundControl por el puerto UDP 14550.

**Recuerda**

Con `controlador:=ardupilot`, nuestros controladores de ROS no se usan: el que pilota es ArduCopter. Nuestro planificador sigue decidiendo a dónde ir y se lo pide a ArduCopter en modo GUIDED, exactamente como se haría con el dron real conectado a una computadora a bordo.

# Capítulo 20

# Volar la búsqueda con ArduPilot

**En este capítulo**

Cómo se conectó ArduCopter a nuestra simulación: el modelo del dron para ArduPilot, sus parámetros, el nodo de misión, las ventanas, los comandos y la trayectoria en el mapa.

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
    controlador:=ardupilot mision:=busqueda
```

Con `controlador:=ardupilot` el dron lo pilota ArduCopter 4.7.2 en lugar de los controladores de ROS 2. Así se puede comparar el STA con un piloto automático real y probar la misión con la misma interfaz (MAVLink) que tendría el dron físico.

## 20.1. El vehículo para ArduPilot

`generar_modelo.py` genera también `models/tarot650_ardupilot`: el mismo Tarot con

- el plugin `ArduPilotPlugin` (de `ardupilot_gazebo`), que intercambia con el SITL el estado del dron y las salidas de los motores por UDP (puerto 9002, formato JSON, en *lock-step*) y lee una IMU girada al marco NED;
- el sistema `ComandosMotores` (`cuadrirotor_gz_plugins`), que junta las cuatro salidas (un `gz.msgs.Double` por motor, PWM de 1000–2000 µs escalado a 0–680 rad/s) en un solo `gz.msgs.Actuators`, que es lo que espera `MulticopterMotorModel`.

Así la física de los motores, el viento y el incendio son los mismos que con el PD y el STA.

```
<plugin filename="ArduPilotPlugin" name="ArduPilotPlugin">
  <fdm_addr>127.0.0.1</fdm_addr>
  <fdm_port_in>9002</fdm_port_in>
  <lock_step>1</lock_step>
  <modelXYZToAirplaneXForwardZDown degrees="true">0 0 0 180 0 0</modelXYZToAirplaneXForwardZDown>
```

```
  <gazeboXYZToNED degrees="true">0 0 0 180 0 90</gazeboXYZToNED>
  <imuName>base_link::imu_sensor</imuName>
  <control channel="0">
    <jointName>rotor_0_joint</jointName>
    <type>COMMAND</type>
    <cmd_topic>/cuadrirotor/ap/motor_0</cmd_topic>
    <multiplier>680.0</multiplier>
    <servo_min>1000</servo_min>
    <servo_max>2000</servo_max>
  </control>
  <!-- ... canales 1 a 3 ... -->
</plugin>
<plugin filename="ComandosMotores" name="cuadrirotor::ComandosMotores">
  <entrada>/cuadrirotor/ap/motor_0</entrada>  <!-- ... motor_1 a motor_3 -->
  <salida>/cuadrirotor/command/motor_speed</salida>
</plugin>
```

## 20.2. Parámetros de ArduCopter

`config/ardupilot/tarot650.parm` se carga sobre los del Iris de `ardupilot_gazebo`:

```
FRAME_CLASS     1        # quad
FRAME_TYPE      1        # X
MOT_PWM_MIN     1000
MOT_PWM_MAX     2000
MOT_THST_EXPO   1.0      # salida lineal en w, empuje en w^2: curva cuadrática pura
MOT_SPIN_ARM    0.08
MOT_SPIN_MIN    0.12
MOT_THST_HOVER  0.28     # (357/680)^2
MOT_HOVER_LEARN 2
ATC_RAT_RLL_P   0.08     # lazos de tasa más suaves que el Iris:
ATC_RAT_RLL_I   0.08     # empuje/peso ~3.6 y motores más lentos (30-50 ms)
ATC_RAT_RLL_D   0.002
ATC_RAT_PIT_P   0.08
ATC_RAT_PIT_I   0.08
ATC_RAT_PIT_D   0.002
ATC_RAT_YAW_P   0.2
```

```
ATC_RAT_YAW_I   0.02
```

## 20.3. El nodo `mision_ardupilot`

Corre con el Python de `~/venv-ardupilot` (pymavlink) y se conecta a MAVProxy por `udp:127.0.0.1:14551`. Espera a que el EKF esté listo, pone el modo GUIDED, arma y despega con `MAV_CMD_NAV_TAKEOFF`. Después manda a 20 Hz `SET_POSITION_TARGET_LOCAL_NED` con posición, velocidad y guiñada:

- con `mision:=orbita` o `hover`, la misma referencia que los controladores de ROS 2 (`dinamica.Referencia`); guarda el CSV con `csv:=` en el formato de `metricas.py`;
- con `mision:=busqueda`, despega a 5 m y después reenvía las consignas de `busqueda_incendio`.

El paso de ENU a NED es $N = y$, $E = x$, $D = -z$ y $\psi_{NED} = \pi/2 - \psi$.

**Problema encontrado y cómo se resolvió**

Con el backend JSON, el marco local de ArduPilot (`LOCAL_POSITION_NED`) **es** el mundo de Gazebo, y el GPS de ArduPilot es el *home* más la posición en Gazebo. La primera versión restaba además el punto de salida $(-232, -232)$: todas las consignas llegaban corridas ≈330 m al noreste. El dron patrullaba fuera de su ruta (de ahí unas detecciones de humo en rumbos extraños) y en una corrida se estrelló contra un cerro ("*Crash: Disarming*"). Se encontró comparando el tlog de MAVProxy con la odometría de Gazebo. Ahora:

- antes de armar se mide el desfase entre `LOCAL_POSITION_NED` y la odometría (sale ≈(0.02, 0.01, 0.01) m) y se descuenta, en lugar de suponerlo;
- el *home* del SITL (`-l`) es el origen del mundo (`geo.ORIGEN`), así que el GPS de ArduPilot coincide con el del aviso del incendio.

## 20.4. Las ventanas del SITL

Con GUI se abren cinco ventanas, que `ventanas_sitl.py` acomoda en columnas cuando aparecen (con `python-xlib`, por EWMH): la terminal **MAVProxy** arriba a la izquierda y al frente, la consola y la terminal de ArduCopter debajo, Gazebo al centro y el mapa a la derecha.

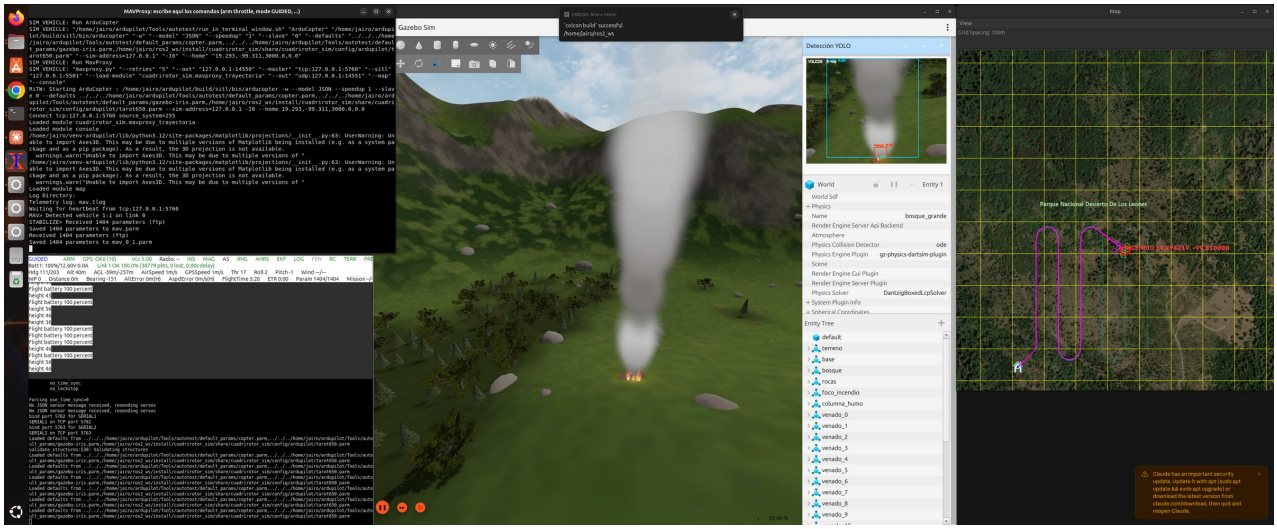


**Figura 20.1:** Pantalla durante la búsqueda con ArduPilot, al confirmarse el incendio.

**Terminal MAVProxy** sim_vehicle.py corre dentro de ella: ahí queda el prompt de MAVProxy para escribir comandos.

**Consola de MAVProxy** modo, armado, GPS, batería, altura y mensajes de ArduPilot.

**Terminal ArduCopter** el proceso del SITL (antes se abría minimizada; ahora, `SITL_RITW_TERMINAL` la abre a la vista).

**Mapa** imagen satelital con el dron, el *home* y lo que dibuja `mavproxy_trayectoria`.

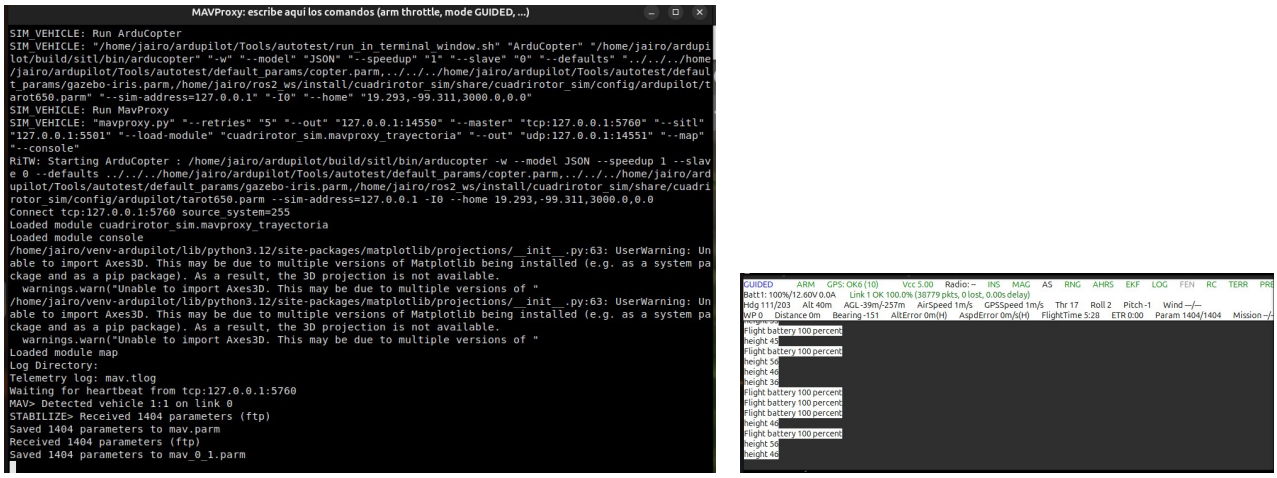



**Figura 20.2:** Terminal de MAVProxy con su prompt (izquierda) y consola (derecha).

### 20.4.1. Comandos útiles en el prompt de MAVProxy

| Comando | Efecto |
|---|---|
| `modeGUIDED` | modo guiado (el que usa la misión) |
| `armthrottle` | arma los motores |

| Comando | Efecto |
|---|---|
| `takeoff10` | despega a 10 m (en GUIDED y armado) |
| `modeLOITER` / `modeBRAKE` | se queda en el lugar |
| `modeRTL` | regresa a la base y aterriza |
| `modeLAND` | aterriza donde está |
| `disarm` | desarma (en tierra) |
| `paramshowMOT_*` / `paramsetNOMBREvalor` | ver y cambiar parámetros |
| `trayectoriaborrar` / `ocultar` / `mostrar` | controla el dibujo de la trayectoria en el mapa |

**Cuidado**

Si se toma el control desde MAVProxy (por ejemplo `modeRTL`), `mision_ardupilot` sigue enviando consignas, que solo surten efecto en GUIDED: al volver a `modeGUIDED` el dron retoma la búsqueda.

## 20.5. La trayectoria en el mapa

El módulo de MAVProxy `cuadrirotor_sim.mavproxy_trayectoria` (se carga con `--load-module` cuando hay GUI) dibuja:

- en magenta, **toda** la trayectoria recorrida (un punto cada 2 m de la posición GPS), en lugar de la estela de 12 s que trae MAVProxy;
- en cian, la ruta planeada (la serpentina);
- un círculo rojo con las coordenadas cuando se confirma el incendio.

La ruta y el foco los escribe `busqueda_incendio` en `/tmp/cuadrirotor_sim_<uid>/mapa_busqueda.json`; el módulo lo vuelve a leer cuando cambia.

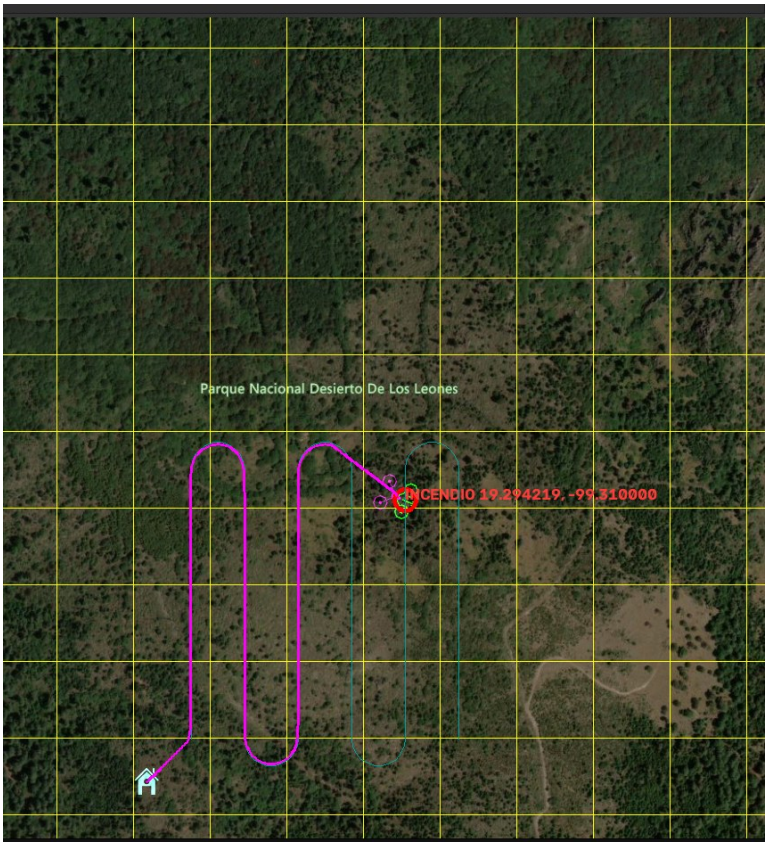



**Figura 20.3:** Mapa de MAVProxy al final de la búsqueda: trayectoria recorrida desde la base (H), ruta planeada y foco confirmado.

## 20.6. Cierre limpio

Al cerrar el *launch* con Ctrl-C, un manejador `OnShutdown` termina lo que no se cierra solo: el `gzsim` que queda detrás del envoltorio de ruby (que no reenvía la señal) y, con ArduPilot, MAVProxy y ArduCopter, que sim_vehicle no cierra cuando se cierra su terminal.

# Parte VI

# Manos a la obra

# Capítulo 21

# Tu primer vuelo, paso a paso

**En este capítulo**

Un recorrido guiado: despegar y mantener el dron en el aire, mirar qué pasa por dentro, hacer una órbita con viento y medirla, lanzar la búsqueda autónoma del incendio y, al final, volar con ArduPilot y darle órdenes desde MAVProxy. Supone que la instalación (capítulo 13) ya está hecha.

## 21.1. Paso 0: preparar la terminal

Abre una terminal y escribe:

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
```

Hazlo en *cada* terminal nueva (o agrégalo a `~/.bashrc`).

## 21.2. Paso 1: despegar

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande
    controlador:=sta mision:=hover
```

Se abre Gazebo. El Tarot aparece sobre la base, despega suavemente y se queda a 3 m. A los 12 segundos la cámara de la ventana empieza a seguirlo. En la terminal verás mensajes de los nodos.



**Figura 21.1:** Lo que debes ver: el Tarot en vuelo estacionario sobre la base.

**Consejo**

En Gazebo: rueda del ratón = acercar; botón izquierdo arrastrando = girar la vista; botón derecho (o Shift + izquierdo) = desplazar. El botón "play/pausa" de abajo a la izquierda congela la física.

## 21.3. Paso 2: mirar por dentro

Abre una segunda terminal (con el Paso 0) y explora:

```
ros2 node list                                              # ?`quién está corriendo?
ros2 topic list                                             # ?`qué datos circulan?
ros2 topic echo /cuadrirotor/odometria --field pose.pose.position   # la posición del dron
ros2 topic echo /cuadrirotor/error                          # el error de seguimiento
ros2 run rqt_image_view rqt_image_view /cuadrirotor/camara  # lo que ve la cámara
```



**Figura 21.2:** La imagen de la cámara del dron sobre la base: el borde de la losa y el pasto.

Para cerrar la simulación, regresa a la primera terminal y presiona **Ctrl+C**.

## 21.4. Paso 3: una órbita con viento, y medirla

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque \
    controlador:=sta esquema:=implicito mision:=orbita viento:=true csv:=$HOME/mi_orbita.csv
```

El dron despega, va junto al foco del incendio y le da vueltas a 4 m de radio mirándolo; a los 15 s empieza a soplar viento. Déjalo un par de minutos, ciérralo con Ctrl+C y mide:

```
python3 ~/ros2_ws/src/cuadrirotor_sim/scripts/metricas.py ~/mi_orbita.csv
```

Verás el error RMS sin viento y con viento (unos 3 y 5 cm con el STA). Repite con `controlador:=pd` y compara: es el experimento central del proyecto.

**Pruébalo tú**

Abre `~/mi_orbita.csv` en una hoja de cálculo o con Python y grafica `x` contra `y`: verás el círculo de la órbita. Las columnas `sx`, `sy`, `sz` son la variable deslizante y `k1x`, `k1y`, `k1z`, la ganancia adaptativa.

## 21.5. Paso 4: la búsqueda autónoma del incendio

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
    controlador:=sta mision:=busqueda
```

Ahora el dron va solo:

1. despega a 30 m sobre el suelo;
2. recorre el bosque en serpentina (en la terminal: *[patrulla] Recorriendo el bosque*);
3. cuando YOLO ve humo, se dirige hacia él (*[hacia_humo]*);
4. estima dónde está el foco y se acerca (*[aproximacion]*);
5. al confirmarlo con las llamas, baja y empieza a orbitar, y escribe el aviso:

```
INCENDIO CONFIRMADO 2026-10-23 04:26:47
  GPS: 19.294219, -99.310000  (19°17'39.2"N 99°18'36.0"W)
  Mapa: https://www.google.com/maps?q=19.294219,-99.310000
```

Tarda entre uno y tres minutos según dónde esté el fuego. En la ventana de Gazebo, el panel "Detección YOLO" muestra lo que ve la red neuronal.



**Figura 21.3:** El dron llegando al incendio.

**Pruébalo tú**

Pon el incendio en otro lugar y observa cuánto tarda en encontrarlo:

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
```

```
    controlador:=sta mision:=busqueda foco_x:=-150 foco_y:=60
```

Desde otra terminal: `ros2topicecho/cuadrirotor/busqueda/cobertura` muestra qué porcentaje del bosque ya vio.

## 21.6. Paso 5: volar con ArduPilot y darle órdenes

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
    controlador:=ardupilot mision:=busqueda
```

Se abren cinco ventanas acomodadas solas: arriba a la izquierda la terminal **MAVProxy**, debajo la consola y la terminal de ArduCopter, Gazebo al centro y el mapa a la derecha. Tras ≈30 s de revisiones, ArduPilot arma, despega y sigue la misma búsqueda. En el mapa verás la ruta planeada en cian y lo recorrido en magenta.

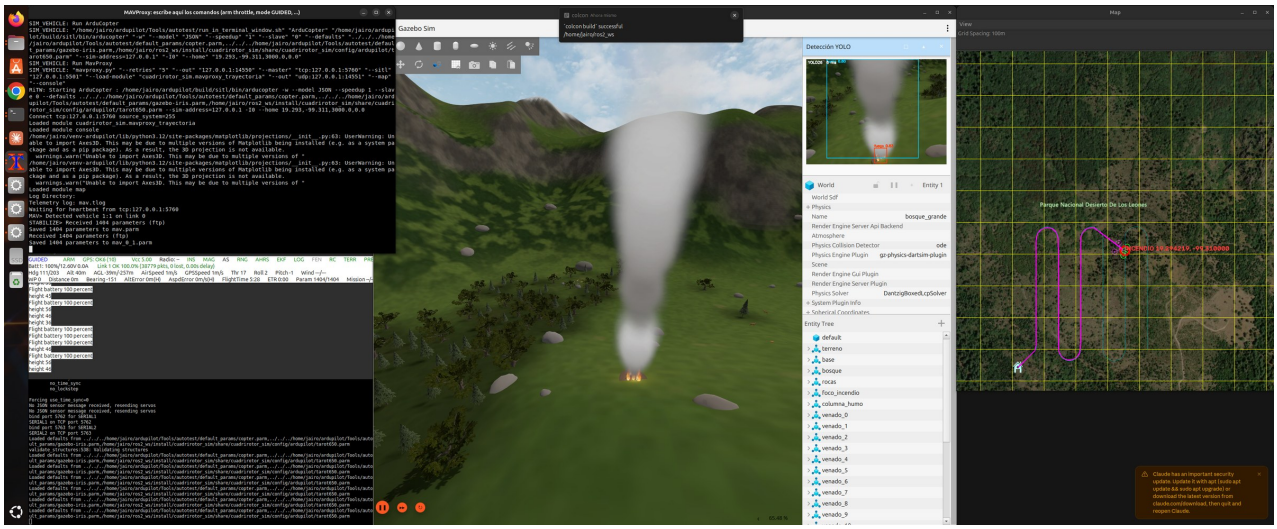


**Figura 21.4:** La pantalla con ArduPilot al confirmarse el incendio.

Escribe en la terminal MAVProxy (donde dice `MAV>` o `GUIDED>`):

```
mode LOITER          # el dron se detiene donde está
mode GUIDED          # retoma la búsqueda
mode RTL             # regresa a la base y aterriza
trayectoria borrar   # limpia la línea magenta del mapa
```

**Cuidado**

Si tomas el control con un modo distinto de GUIDED, la misión no puede mover al dron hasta que regreses a `modeGUIDED`. Y si aterriza con RTL y se desarma, hay que reiniciar la simulación.

## 21.7. Qué hacer si algo falla

| **Síntoma** | **Qué revisar** |
|---|---|
| *Package 'cuadrirotor_sim' not found* | Falta `source~/ros2_ws/install/setup.bash` en esa terminal. |
| Gazebo abre pero el dron no aparece | Mira la terminal: un error en rojo del *launch* dice qué falló. |
| La simulación va muy lenta | Cierra otros programas; prueba `gui:=false`. Revisa el factor de tiempo real abajo a la derecha de Gazebo. |
| YOLO no detecta nada | ¿Existe `~/.venvs/yolo`? ¿Tiene GPU? Mira los mensajes de `detector_yolo`. |
| Quedan procesos de una simulación anterior | Ciérrala con Ctrl+C y espera unos segundos; si algo queda, búscalo con `psaux\|grepgz` y termínalo por su número (PID). |
| ArduPilot no arma | Espera: el EKF tarda ≈30 s. La consola de MAVProxy dice qué revisión falla. |

# Capítulo 22

# Referencia de uso

**En este capítulo**

Todas las órdenes, argumentos y tópicos en un solo lugar, para consultar.

## 22.1. Antes de lanzar

```
cd ~/ros2_ws
colcon build --symlink-install      # solo si se agregaron ejecutables o se cambió C++
source install/setup.bash
```

## 22.2. Órdenes de lanzamiento

```
# Cuadrirrotor genérico: despegue y hover a 3 m
ros2 launch cuadrirotor_sim simulacion.launch.py

# Órbita alrededor del foco con viento, guardando un CSV
ros2 launch cuadrirotor_sim simulacion.launch.py mision:=orbita viento:=true csv:=$HOME/corrida.csv

# STA implícito, Tarot, bosque
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque \
    controlador:=sta esquema:=implicito mision:=orbita viento:=true

# Sin interfaz y con otro periodo de muestreo
ros2 launch cuadrirotor_sim simulacion.launch.py gui:=false Ts:=0.01

# Búsqueda autónoma del incendio en el valle (STA)
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
    controlador:=sta mision:=busqueda
#   ... con el foco en otro lugar
```

```
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
    controlador:=sta mision:=busqueda foco_x:=-150 foco_y:=60

# La misma búsqueda pilotada por ArduPilot SITL
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande \
    controlador:=ardupilot mision:=busqueda
```

## 22.3. Argumentos del *launch*

| Argumento | Por omisión | Valores |
|---|---|---|
| `vehiculo` | cuadrirotor | `cuadrirotor`, `tarot650` |
| `mundo` | incendio | `incendio`, `bosque`, `bosque_grande` |
| `controlador` | pd | `pd`, `sta`, `ardupilot`, `ninguno` (para lanzarlo aparte con `ros2run`) |
| `esquema` | implicito | STA: `explicito`, `semi_implicito`, `implicito` |
| `mision` | hover | `hover`, `orbita`, `busqueda` |
| `gui` | true | mostrar la interfaz de Gazebo (y las ventanas del SITL) |
| `Ts` | 0.005 | periodo de muestreo del controlador [s] |
| `viento` | false | aplicar la perturbación de viento |
| `csv` | | ruta del CSV de registro |
| `foco_x`, `foco_y` | | `bosque_grande`: posición del incendio [m] |
| `x0`, `y0` | | posición inicial del dron [m] |
| `modelo_yolo` | | pesos de YOLO (.pt) en lugar de `yolo/incendio_yolo26n.pt` |

## 22.4. Tópicos

| Tópico | Tipo y contenido | Publica |
|---|---|---|
| `/cuadrirotor/odometria` | `nav_msgs/Odometry` (velocidades en el cuerpo) | Gazebo |
| `/cuadrirotor/imu` | `sensor_msgs/Imu` | Gazebo |
| `/cuadrirotor/camara` | `sensor_msgs/Image` 640×480, 15 Hz | Gazebo |
| `/cuadrirotor/lidar/points` | `sensor_msgs/PointCloud2`, 10 Hz | Gazebo |
| `/cuadrirotor/motores` | `actuator_msgs/Actuators` [rad/s] | controlador |
| `/cuadrirotor/viento` | `geometry_msgs/Vector3` [N] | `viento` |
| `/cuadrirotor/referencia`, `/cuadrirotor/error` | `PoseStamped`, `Vector3Stamped` | controlador |
| `/cuadrirotor/consigna` | `MultiDOFJointTrajectoryPoint` | planificador |
| `/cuadrirotor/detecciones` | `vision_msgs/Detection2DArray` | detector |
| `/cuadrirotor/camara_yolo` | imagen con las cajas | detector |

| Tópico | Tipo y contenido | Publica |
|---|---|---|
| /cuadrirotor/foco_estimado | geometry_msgs/PointStamped | planificador |
| /cuadrirotor/busqueda/estado | std_msgs/String | planificador |
| /cuadrirotor/busqueda/cobertura | std_msgs/Float32 (% visto) | planificador |
| /cuadrirotor/busqueda/mapa_cobertura | nav_msgs/OccupancyGrid | planificador |
| /cuadrirotor/busqueda/ruta | nav_msgs/Path | planificador |
| /cuadrirotor/evasion/obstaculos | sensor_msgs/PointCloud2 (marco world) | planificador |
| /cuadrirotor/incendio/gps | sensor_msgs/NavSatFix (transient local) | planificador |
| /cuadrirotor/incendio/alerta | std_msgs/String (transient local) | planificador |

## 22.5. Herramientas de visualización

```
ros2 topic echo /cuadrirotor/incendio/alerta          # el aviso con las coordenadas
ros2 topic echo /cuadrirotor/busqueda/estado
ros2 run rqt_image_view rqt_image_view /cuadrirotor/camara_yolo
ros2 run plotjuggler plotjuggler                      # señales en tiempo real
rviz2                                                 # marco 'world': mapa de cobertura, ruta,
    obstáculos
```

## 22.6. Campañas y métricas

```
src/cuadrirotor_sim/scripts/corrida.sh ~/corridas/imp.csv 50 esquema=implicito Ts=0.02 k1_0=4.0
    adaptativo=false
src/cuadrirotor_sim/scripts/metricas.py ~/corridas/*.csv
```

`metricas.py` reporta el error RMS sin y con viento, el máximo con viento, el *chattering* y la ganancia $k_1$ final.

## 22.7. Regenerar los recursos

```
cd ~/ros2_ws/src/cuadrirotor_sim
python3 scripts/generar_modelo.py                     # vehículos y sus YAML
python3 scripts/generar_fuego.py                      # fuego y humo
```

```
python3 scripts/generar_bosque.py worlds/bosque.sdf       # mundo bosque
```

```
~/.venvs/cad/bin/python scripts/preparar_fauna.py ~/Descargas/fauna   # animales
```

```
~/.venvs/cad/bin/python scripts/generar_bosque_grande.py  # mundo grande
```

# Capítulo 23

# Cómo se verificó, qué falló y qué falta

**En este capítulo**

Las pruebas automáticas, los resultados de las búsquedas, la lista de problemas que aparecieron durante el desarrollo y lo que queda por hacer.

## 23.1. Pruebas unitarias

```
cd ~/ros2_ws/src/cuadrirotor_sim && python3 -m pytest -q test/
.............................                                    [100%]
31 passed
```

| Archivo | Qué comprueba |
|---|---|
| `test_dinamica.py` | mezclador, rotaciones, referencia de las misiones |
| `test_sta.py` | solución implícita, convergencia de los tres esquemas, banda con perturbación |
| `test_terreno.py` | interpolación del DEM, corte de rayos, rutas de barrido (franjas y serpentina), cobertura |
| `test_geo.py` | conversión ENU → WGS84 (origen, distancias norte y este, altura, grados-minutos-segundos) |
| `test_evasion.py` | camino libre sin cambios, descarte del suelo, rodear un árbol, cruzar entre árboles, subir sin salida, no bajar dentro de las copas, olvidar lo lejano |

## 23.2. Resultados de la búsqueda

Todas con el Tarot 650, sin viento y con el YOLO reentrenado. Altura sobre el suelo durante el vuelo con el STA: mediana 30.3 m, mínima 13.9 m (sobre la ladera del anillo de órbita); error de seguimiento 5 cm RMS (máximo 22 cm).

| Control | Foco | Ruta | Estimación final | Patrulla → órbita | Cobertura |
|---|---|---|---|---|---|
| STA | $(105, 135)$ | franjas | $(105.1, 135.0)$ | 2 min 42 s | 60 % |
| STA | $(-150, 60)$ | franjas | $(-150.0, 59.9)$ | 1 min 8 s | 29 % |
| STA + LiDAR | $(105, 135)$ | franjas | $(105.2, 135.0)$ | — | 53 % |
| ArduPilot | $(105, 135)$ | serpentina | $(105.1, 135.0)$ | — | 79 % |

**Tabla 23.2:** Búsquedas en `bosque_grande` (23 de octubre). En el mundo `bosque` la búsqueda estima $(8.3, 5.2)$ con el foco en $(8, 5)$.

## 23.3. Problemas encontrados

| Síntoma | Causa y solución |
|---|---|
| Gazebo se cae (*segfault* en `OdeMesh::fillArrays`) | Mallas OBJ de colisión sin normales. Exportarlas con normales. |
| YOLO detecta "fuego" en una roca oscura | El modelo solo conocía el bosque plano. Dataset del mundo grande con tomas de rocas y animales; reentrenar con los dos. |
| La primera estimación del foco se va hasta 50 m | De lejos el rayo de las llamas roza el suelo. Usar solo cortes a menos de 100 m. |
| Con ArduPilot, humo en rumbos extraños y choque contra un cerro | Se restaba el punto de salida a consignas que ya estaban en el marco del mundo. Medir el desfase con MAVLink; *home* = origen del mundo. |
| Se confirma un foco a 70 m del verdadero | Bastaba la triangulación del humo. Estado *verificacion*: solo con llamas se confirma. |
| La terminal de comandos de MAVProxy no se ve | Quedaba tapada por las demás ventanas. `ventanas_sitl.py` las acomoda. |
| Tras Ctrl-C quedan `gzsim`, MAVProxy y ArduCopter | El envoltorio de ruby no reenvía la señal. Manejador `OnShutdown` en el *launch*. |
| La evasión oscila cada 20 ms entre desviarse y no | Sin histéresis. Volver al tramo recto solo con $1.4\,r$ libre. |
| Conversión del STEP muy lenta | *gmsh* tardaba más de 15 min. `cascadio` + `trimesh`. |

**Cuidado**

Al detener simulaciones a mano no se debe usar `pkill -f` con un patrón que también aparezca en la propia orden: el comando se mata a sí mismo. Es mejor listar los procesos y terminarlos por PID. Y matar `gzsim` en general cierra también cualquier otra simulación abierta.

## 23.4. Pendientes

1. **Datos reales del Tarot**: masa con batería, inercias, $k_f$, $k_m$ (KV del motor, hélice, celdas) y retardos. Hoy son supuestos (tabla 15.1).
2. Modelo del error de detección, métricas de banda y de energía, y barridos de $T_s$ y de $k$.

3. Probar la evasión con obstáculos más altos que la altura de búsqueda y con viento.
4. Reentrenar YOLO con imágenes reales (por ejemplo, D-Fire) antes de volar el dron físico.

# Apéndice A

# Glosario

**Actitud**
Orientación del dron: alabeo, cabeceo y guiñada.

**Alabeo (roll)**
Giro alrededor del eje que va al frente.

**ArduPilot / ArduCopter**
Piloto automático de código abierto; ArduCopter es su versión para multirrotores.

**Armar**
Habilitar los motores para que puedan girar.

**Batería LiPo**
Batería de polímero de litio, ligera y de mucha energía.

**Brushless**
Motor eléctrico sin escobillas, el que usan los drones.

**Cabeceo (pitch)**
Giro alrededor del eje lateral; con él el dron avanza o retrocede.

**Caja (bounding box)**
Rectángulo que marca dónde está un objeto en una imagen.

**Campo de visión (FOV)**
Ángulo que abarca una cámara.

**Chattering**
Vibración de alta frecuencia del control por modos deslizantes.

**colcon**
Herramienta que compila los paquetes de ROS 2.

**Consigna / referencia**
Lo que se quiere que haga el sistema (posición, velocidad).

**Control en cascada**
Un lazo de control dentro de otro: posición afuera, actitud adentro.

**Convolución**
Operación que desliza un filtro pequeño sobre una imagen; base de las redes para imágenes.

**Cuaternión**
Cuatro números que representan una rotación sin singularidades.

**Dataset**
Conjunto de ejemplos para entrenar una red neuronal.

**DEM**
Modelo digital de elevación: tabla con la altura del terreno en una malla.

**Discretización**
Convertir una ley continua en pasos que una computadora puede ejecutar.

**EKF**
Filtro de Kalman extendido: combina sensores para estimar posición y orientación.

**Empuje**
Fuerza hacia arriba que produce una hélice; crece con el cuadrado de su velocidad.

**ENU / NED**
Marcos de referencia: Este-Norte-Arriba (ROS, Gazebo) y Norte-Este-Abajo (aviación).

**Época**
Una pasada completa por todos los datos de entrenamiento.

**Error RMS**
Raíz del promedio de los errores al cuadrado: resume qué tan bien sigue una trayectoria.

**ESC**
Controlador electrónico de velocidad de un motor.

**Explícito / implícito**
Formas de discretizar: con el valor actual o con el futuro.

**Gazebo**
Simulador de robots con física, render y sensores.

**GCS**
Estación de tierra: programa para ver y mandar al dron (MAVProxy, QGroundControl).

**GUIDED**
Modo de ArduPilot en el que obedece órdenes de una computadora.

**Guiñada (yaw)**
Giro alrededor del eje vertical; cambia hacia dónde mira el dron.

**Hover**
Vuelo estacionario: quieto en el aire.

**IMU**
Unidad de medición inercial: acelerómetros y giroscopios.

**IoU**
Intersección sobre unión: qué tanto coinciden dos cajas.

**Lazo cerrado**
Control que mide el resultado y corrige (retroalimentación).

**Launch**
Archivo que arranca varios nodos de ROS con sus parámetros.

**LiDAR**
Sensor que mide distancias con pulsos láser; produce nubes de puntos.

**mAP**
Precisión media promedio: métrica principal de un detector.

**MAVLink**
Protocolo de mensajes entre el dron y la estación de tierra.

**MAVProxy**
Estación de tierra de texto, con consola y mapa.

**Mezclador**
Convierte empuje y pares en velocidades de cada motor.

**Modos deslizantes (SMC)**
Control robusto que lleva al sistema a una superficie y lo hace deslizarse por ella.

**Momento de inercia**
Resistencia de un cuerpo a cambiar su giro.

**Nodo**
Programa de ROS que hace una tarea.

**Nube de puntos**
Conjunto de puntos 3D, como los que da un LiDAR.

**Odometría**
Estimación de la posición y velocidad del robot.

**Par (torque)**
Lo que hace girar un cuerpo: fuerza por distancia.

**Parámetro**
Ajuste de un nodo de ROS, normalmente en un archivo YAML.

**Periodo de muestreo ($T_s$)**
Tiempo entre dos cálculos del controlador.

**Perturbación**
Lo que empuja al sistema fuera de lo deseado (viento, carga).

**PID**
Controlador proporcional, integral y derivativo.

**Píxel**
Cada punto de una imagen digital, con su valor de rojo, verde y azul.

**Planta**
El sistema que se controla (aquí, el dron).

**Plugin**
Programa que se carga dentro de Gazebo para agregar un comportamiento.

**Precisión / recall**
De lo detectado, cuánto es correcto / de lo que hay, cuánto se detecta.

**QoS**
Calidad de servicio de un tópico de ROS (por ejemplo, guardar el último mensaje para quien llegue tarde).

**ROS 2**
Conjunto de herramientas para que los programas de un robot se comuniquen.

**RTF**
Factor de tiempo real: velocidad de la simulación respecto al reloj.

**SDF**
Formato de archivo con el que Gazebo describe robots y mundos.

**Servicio**
Comunicación de pregunta y respuesta en ROS.

**SITL**
El software del piloto automático corriendo en la computadora contra un dron simulado.

**Sobreajuste**
Cuando una red memoriza los ejemplos en lugar de aprender el concepto.

**Super-twisting (STA)**
Modo deslizante de segundo orden con señal de control continua.

**Superficie deslizante**
Conjunto donde $s = 0$; estando en él, el error se va a cero solo.

**Tópico**
Canal con nombre por donde viajan mensajes en ROS.

**Triangulación**
Ubicar un punto cruzando direcciones vistas desde lugares distintos.

**UAV**
Vehículo aéreo no tripulado.

**VFH**
Histograma de campo vectorial: método de evasión que elige un sector libre.

**WGS84**
Modelo de la Tierra que usa el GPS.

**Workspace**
Carpeta donde viven y se compilan los paquetes de ROS (`~/ros2_ws`).

**YAML**
Formato de texto para archivos de configuración.

**YOLO**
Familia de redes neuronales para detectar objetos en tiempo real.

# Apéndice B

# Referencia rápida

```
# Preparar la terminal
source /opt/ros/jazzy/setup.bash && source ~/ros2_ws/install/setup.bash

# Búsqueda autónoma con el STA / con ArduPilot
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande
    controlador:=sta mision:=busqueda
ros2 launch cuadrirotor_sim simulacion.launch.py vehiculo:=tarot650 mundo:=bosque_grande
    controlador:=ardupilot mision:=busqueda

# Aviso del incendio y estado
ros2 topic echo /cuadrirotor/incendio/alerta
ros2 topic echo /cuadrirotor/busqueda/estado
cat ~/incendios_detectados.csv

# Pruebas
cd ~/ros2_ws/src/cuadrirotor_sim && python3 -m pytest -q test/
```

# Apéndice C

# Configuración del mundo grande

config/mundo_bosque_grande.yaml:

```
# Búsqueda en el mundo 'bosque_grande' (valle de 420 x 420 m con lomas y cerros).
# El launch agrega 'terreno' (el DEM); las alturas son sobre el suelo.
busqueda_incendio:
  ros__parameters:
    zona: [-210.0, 210.0, -210.0, 210.0]   # xmin, xmax, ymin, ymax del bosque [m]
    patron: serpentina        # avanza hacia el frente del dron (franjas: boustrophedon)
    separacion: 70.0          # entre barridos [m]
    altura_busqueda: 30.0     # sobre el suelo; los árboles miden hasta ~8 m
    anticipacion: 50.0        # mira el relieve esta distancia hacia adelante [m]
    velocidad: 5.0
    velocidad_vertical: 3.0
    aceleracion: 1.5
    alcance_vision: 150.0     # para el mapa de cobertura y el corte del rayo [m]
    # Órbita por encima de las copas (el planificador no sabe dónde hay árboles).
    altura_orbita: 18.0
    altura_minima_copas: 14.0
    radio: 14.0
    omega: 0.12
    # Evasión: no avanza a menos de 10 m sobre el relieve que tiene adelante
    # (primero sube); el LiDAR rodea lo que esté a menos de 3 m del camino.
    altura_seguridad: 10.0
    radio_evasion: 3.0
    alcance_evasion: 20.0
```

# Apéndice D

# YAML del Tarot generado

`config/vehiculo_tarot650.yaml` (lo escribe `generar_modelo.py`):

```
/**:
  ros__parameters:
    masa: 2.2
    inercia: [0.03412, 0.03609, 0.06596]
    rotores_xy: [0.2198, -0.212, 0.2198, 0.212, -0.2198, 0.212, -0.2198, -0.212]
    rotores_sentido: [1, -1, 1, -1]
    kf: 4.24e-05
    km: 0.016
    w_max: 680.0
    kr_att: [3.4, 3.6, 1.65]
    kw_att: [0.55, 0.58, 0.59]
    z_inicial: 0.235
    camara_xyz: [0.12, 0.0, -0.06]
    camara_pitch: 0.5236
    camara_hfov: 1.3
    camara_ancho: 640
    camara_alto: 480

controlador_sta:
  ros__parameters:
    mu: [0.3, 0.3, 0.03]
    k1_max: 3.0
```

# Apéndice E

# Créditos de los recursos

- Tarot Ironman 650: modelo CAD de GrabCAD, https://grabcad.com/library/tarot-ironman-650-2.
- Pino y roble: OpenRobotics en Gazebo Fuel (CC0).
- Venado (crownjoshua), zorro y lobo (Quaternius): OpenGameArt, CC0. https://opengameart.org/content/cc0-3d-animals-creatures, https://opengameart.org/content/deer-low-poly-rigged, https://opengameart.org/content/5-low-poly-animals, https://opengameart.org/content/animated-animales-low-poly.
- YOLO26: Ultralytics. ArduPilot y `ardupilot_gazebo`: proyecto ArduPilot (GPLv3).
- Imagen satelital del mapa de MAVProxy: la que descarga MAVProxy para la ubicación supuesta.